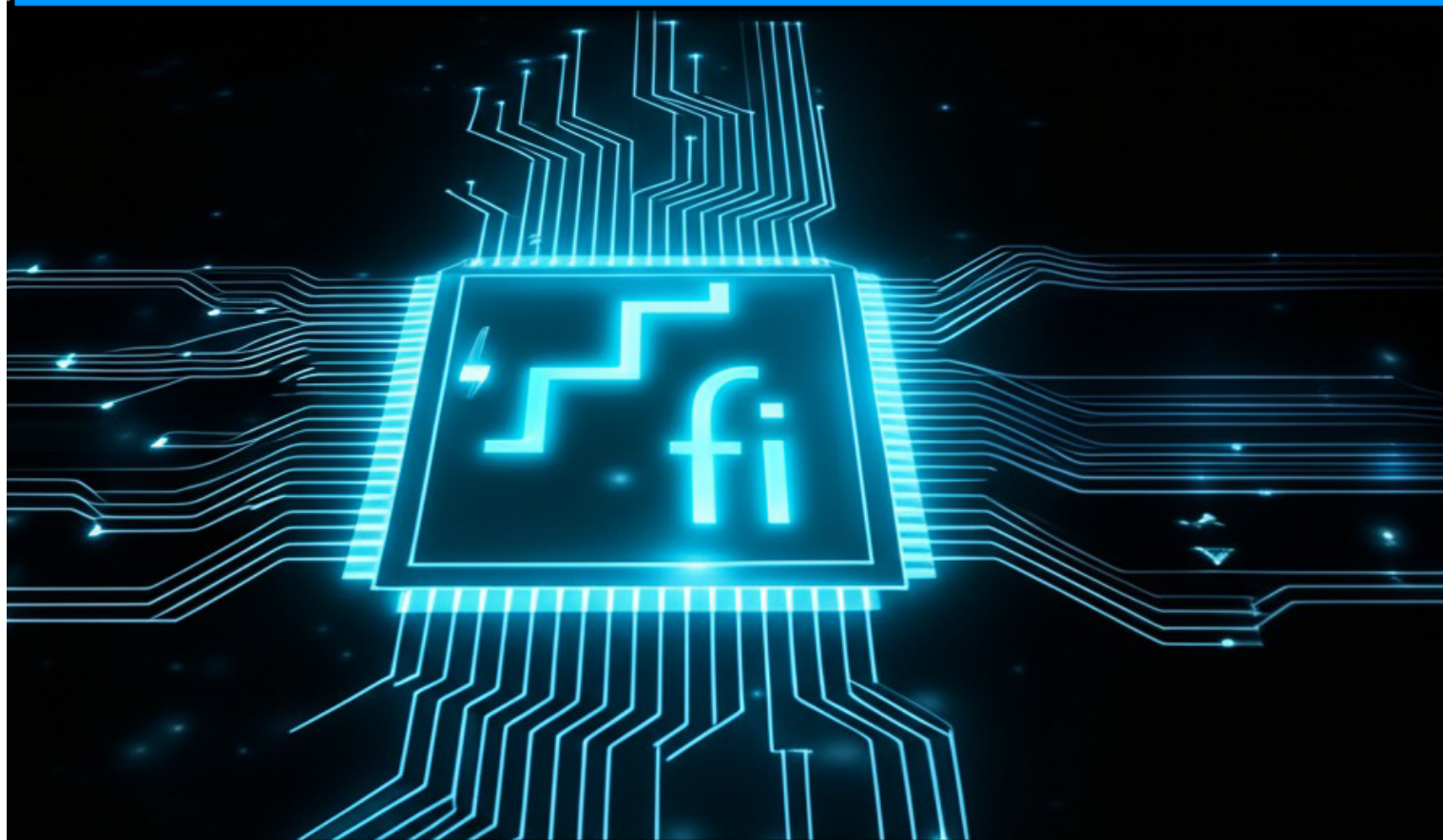


Fixed-Point Workshop



Tom Bryan, Brenda Zhuang, et al.

Fixed-Point MATLAB Workshop

Tom Bryan and Brenda Zhuang
with contributions from
Jenna Warren, Jim Joy, Bharath Venkataraman, and Rui Zhang

For MATLAB release R2025a
Last modified: 6 April 2026 at 15:43

© 2016–2026 The MathWorks, Inc.
All rights reserved.
First presented by Tom Bryan on 25 October 2016.
Updated by Brenda Zhuang on Monday 6th April, 2026.

Tom Bryan
`tom.bryan@mac.com`

Brenda Zhuang
`bzhuang@mathworks.com`

Contents

Fixed-Point MATLAB Workshop	2
Contents	7
Preface	9
1 Overview	13
1.1 Schedule	13
1.2 Workshop Examples	13
1.2.1 Fixed-Point Conversion Process using FIR Filter	14
1.2.2 Coordinate Rotation Digital Computer (CORDIC) Algorithm	15
1.2.3 Fixed-Point Polyphase DFT Channelized Receiver	16
1.2.4 Fixed-Point Solution of Linear Equations	20
1.2.5 Euler to North-East-Down (NED) Coordinate Transformation using CORDIC	22
1.2.6 Oblique Orthographic Projection Using CORDIC	23
1.3 Workflow	25
1.4 Rounding Errors in the Vancouver Stock Exchange	26
2 Numbers in Digital Hardware	31
2.1 Floating-Point Numbers	31
2.2 Fixed-Point Numbers	31
2.3 Scaling	34
2.4 Precision and Range	34

2.5	Quantization	34
2.6	Fixed-Point Numbers in MATLAB	35
2.7	Addition	36
2.7.1	Code generation example for addition	38
2.8	Multiplication	41
2.8.1	Code generation example for multiplication	44
3	Basics and Best Practices	47
3.1	Data types in MATLAB	47
3.2	<code>cast(x, 'like', T.x), zeros(m,n, 'like', T.z)</code>	53
3.3	Subscripted assignment <code>A(:)=B</code>	55
3.3.1	Subscripted assignment to retain datatype	57
4	Fixed-Point Conversion Process using FIR Filter	59
4.1	Workflow	59
4.2	Algorithm to Convert	60
4.2.1	Linear Buffer Implementation	62
4.2.2	Circular Buffer Implementation	63
4.3	Prepare Code for Instrumentation, Fixed-Point Conversion, and Acceleration	65
4.4	C++ Analogy: Template Functions	65
4.5	Templated MATLAB Function	69
4.6	Conversion Heuristic	70
4.7	(1) Separate algorithm from test script	71
4.7.1	Inlined algorithm and test script	71
4.7.2	Algorithm	72
4.7.3	Test script	72
4.8	(2) Types Table and Template Parameter	76
4.8.1	Create a Types Table in Original Double Types	76
4.8.2	Add Template Parameter	77
4.8.3	Update Test Script to use the Template Parameter	77
4.9	(3) Generate code	77

<i>CONTENTS</i>	5
4.10 (4) Single-Precision Floating-Point Types	81
4.10.1 Create a single-precision floating-point template parameter table	81
4.10.2 Change test script to run with single types	81
4.10.3 Assign in <code>acc(:)=...</code>	83
4.10.4 Output and accumulator are still double	83
4.10.5 Specify output and accumulator type	84
4.11 (5) Instrument Code and Visualize Types with Fixed-Point	85
4.11.1 Create Fixed-Point Types Table	85
4.11.2 Use Fixed-Point Types in Test Script	85
4.11.3 Instrument with Scaled Double Types	86
4.12 Choose Fixed-Point Types	91
4.12.1 Choose types based on theory	92
4.12.2 Choose types based on logged values from simulation data	102
4.13 More efficient <code>fimath</code>	107
4.13.1 Align product and sum	113
4.14 Verify Quantized Filter Design	115
5 Convert CORDIC Algorithm to Fixed Point	123
5.1 Derivation of the CORDIC Algorithm	125
5.2 The CORDIC Algorithm	126
5.3 CORDIC Kernel	128
5.4 CORDIC Datatypes	129
5.5 CORDIC Cartesian-to-Polar function	130
5.6 CORDIC Test Function	131
5.6.1 Generate C from the MATLAB function	135
5.6.2 Generate HDL from the MATLAB function	139
5.7 Combining MATLAB and Simulink for Efficient HDL	142
5.7.1 Generate HDL from Simulink Model	145
6 Convert DFT Polyphase Filter Bank to Fixed Point	149
6.1 MATLAB implementation	150

6.2	Test file	150
6.3	Types tables	151
6.4	DSP HDL Toolbox Channelizer	154
7	Linear Systems of Equations	155
7.1	Notation	156
7.2	Linear Equations	156
7.3	Datatypes for Linear Systems of Equations	157
7.4	QR Decomposition using CORDIC	158
7.4.1	Graphical Representation of CORDIC QR Algorithm	158
7.4.2	Rotate first element to real	161
7.4.3	Rotate Sub-Diagonal Elements to Zero	164
7.5	Solve $(A'A)X = B$ given A	165
7.5.1	Fixed-Point Code Generation	169
7.5.2	Minimum-Variance Distortionless Response (MVDR) Beamformer	170
7.6	Solve $(A'A)X = B$ given $(A'A)$	172
7.6.1	Fixed-Point Code Generation	172
7.6.2	MVDR Beamformer Solving $(A'A)X = B$ given $(A'A)$	173
7.7	Solve $AX = B$, Classical Least-Squares	176
7.7.1	Fixed-Point Code Generation	176
7.8	Tikhonov Regularization	177
7.9	Combine MATLAB and Simulink for efficient HDL	178
7.9.1	HDL-Optimized Linear Algebra Library	178
7.9.2	HDL-Optimized Linear Algebra Examples	179
7.10	Implementation Comparisons	179
7.10.1	Systolic: One cell for each zero ($\mathcal{O}(mn)$ cells). High area, High throughput, Low latency.	180
8	Euler to North-East-Down (NED) Coordinate Transformation	183
8.1	The Euler-NED Transformation	184
8.2	The CORDIC Technique	186

<i>CONTENTS</i>	7
8.3 Benchmarks	189
8.3.1 Resource Usage	189
8.3.2 Static Timing Analysis	189
9 Oblique Orthographic Projection Using CORDIC	191
9.1 The Oblique Orthographic Projection	191
9.2 The CORDIC Technique	193
9.3 Designing the Oblique Orthogonal Transformation	194
9.3.1 The Valid-Ready Interface	194
9.3.2 Architecture	194
9.3.3 Choosing Datatypes	196
9.4 Resource Usage	196
10 Future Horizons	199
A Derivation of the DFT Polyphase Filter Bank	201
B Equivalent Toolbox Functions and Blocks	207
C HEX Read and Write	211
D ac_fixed, sc_fixed, and fi	215
Bibliography	219
Author Biographies	223

Preface

Introduction

The simulation ran flawlessly. In the predictable world of double precision, your algorithm performed exactly as designed. Yet, on the target processor, the results are wrong. The output oscillates, values clip unexpectedly, and a subtle one-bit error in a scaling factor has cascaded into total system failure. This gap – between the elegant theory of algorithm and the unforgiving reality with its fixed-point implementation – is the often underestimated challenge in the embedded systems development. Somehow, issues like overflow, quantization error, and bit growth aren’t just academic—they’re the difference between a working product and a failed prototype.

Who is this book for? It’s for anyone who has ever brought an algorithm to life in MATLAB and wondered, “What’s next?” You are designing communication systems, developing DSP algorithms, or working on radar, audio, motor control, automotive, or aerospace projects. You are the embedded software engineer or hardware designer. You likely spend your days deep in C, C++ or wrestling with the intricacies of VHDL or Verilog. Your focus is on producing efficient code and moving seamlessly from simulation to silicon. You are the students and educators eager to bridge the gap between classroom concepts and real-world embedded systems.

This book presents a systematic, tool-driven and hands-on approach to fixed-point design. We will leverage the simulation and analysis power of MATLAB and Simulink to automate much of the conversion process, allowing engineers to focus on algorithm performance rather than manual implementation details, leading to faster development cycles and robust code.

The Roadmap

This book is designed as a guided journey—one that starts with the essentials and builds, step by step, toward mastery of fixed-point implementation in MATLAB and Simulink. Whether you are new to the topics or looking to deepen your expertise, you’ll find the path unfolds naturally, expanding your toolkit and confidence along the way.

We begin with the practicalities on how to set up your environment and get hands-on with the examples. You will quickly move from theory to action, experimenting with foundational

algorithms such as FIR filters and exploring the fixed-point conversion process.

As you progress, the topics expand to real-world examples, such as CORDIC algorithms, polyphase DFT channelized receivers, and coordinate transformations. You will learn how to select data types, manage scaling and quantization, and avoid pitfalls like overflow and loss of precision. The chapters weave these techniques into larger workflows, showing you how to structure your MATLAB code for hardware readiness, instrument your designs, and generate efficient C and HDL code for deployment on your target platform.

In the later chapters, we will tackle advanced topics: solving linear systems, optimizing architectures for resource and throughput, and comparing implementation strategies for real engineering tradeoffs. You'll see how to combine MATLAB and Simulink to create robust, hardware-efficient solutions, and benchmark your designs to understand their performance and limitations.

The appendices serve as a reference, offering derivations, toolbox comparisons, and practical guides for tasks like HEX file I/O.

Required Products

The following MathWorks' products are required for running the exercises from this book.

- MATLAB® release R2022a or later
- Simulink®
- DSP System Toolbox™
- Embedded Coder®
- Fixed-Point Designer™
- HDL Coder™
- MATLAB Coder™
- Signal Processing Toolbox™

Set Up for Hands-On Learning

In MATLAB, navigate to the directory accompanying this workshop

```
exercises
```

The MATLAB code and Simulink models accompanying the workshop are organized by directories numbered and named after the chapters in this workbook. Solutions for exercises are in directory

solutions

There are many ways to solve these problems, so please let us know if you find better solutions. You can reach us at

Tom Bryan
tom.bryan@mac.com

Brenda Zhuang
bzhuang@mathworks.com

If you encounter issues during the setup, consult the Appendix ?? with steps to configure your user environment.

Acknowledgements

A book of this technical nature is never the product of a single mind. We are deeply indebted to our contributors, Jenna Warren, Jim Joy, Bharath Venkataraman, and Rui Zhang, whose insightful input shaped the contents you will read. Our sincere thanks go to the engineering teams at MathWorks for creating a toolchain of such remarkable capabilities. These colleagues provided the inspiration for many of the real-world problems explored within these pages. Finally, to our family members, thank you for your unending patience and support throughout this endeavor.

Now, let's begin...

Our sincere hope is that the techniques and workflow presented in these chapters will do more than simply improve your code. We hope they will reclaim your most valuable asset as an engineer: your time. Time to innovate and solve new problems, rather than manually chasing down bit-level errors. The path to embedded efficiency of your product can be a clear and joyful one. We look forward to taking you on this journey.

Chapter 1

Overview

“For those of you that have not experienced MATLAB, we would like to try to show you what everybody is excited about ... The best way to appreciate PC-MATLAB is, of course, to try it yourself.”

— Jack Little and Cleve B. Moler, *A Preview of PC-MATLAB* (1985)

The examples for this chapter are in directory

`exercises/01overview`

1.1 Schedule

- Fixed-point basics and best practices
- Break
- Convert a system to fixed-point, featuring FIR filter and CORDIC algorithm

1.2 Workshop Examples

By the end of the workshop we will have worked through some of the following projects, depending on participant interest. If we have time, we can go through all of them.

Before working on these projects, we will go through the basics and best-practices of using fixed-point in MATLAB.

1.2.1 Fixed-Point Conversion Process using FIR Filter

We will go through all the steps of the fixed-point conversion process using a finite-impulse response (FIR) filter. FIR filters are a common feature of fixed-point embedded systems. They are simple enough for everyone to understand, even if you are not familiar with digital signal processing (DSP), and complicated enough to illustrate all of the procedures necessary to convert any system to fixed-point. The same conversion steps in this example can be used to build up to any size problem.

An FIR filter is a weighted moving average. Given a vector of coefficients $b = [b_0, b_1, b_2, b_3]$, and input at time n , $x(n)$, then the output at time n is:

$$y(n) = b_0x(n) + b_1x(n-1) + b_2x(n-2) + b_3x(n-3).$$

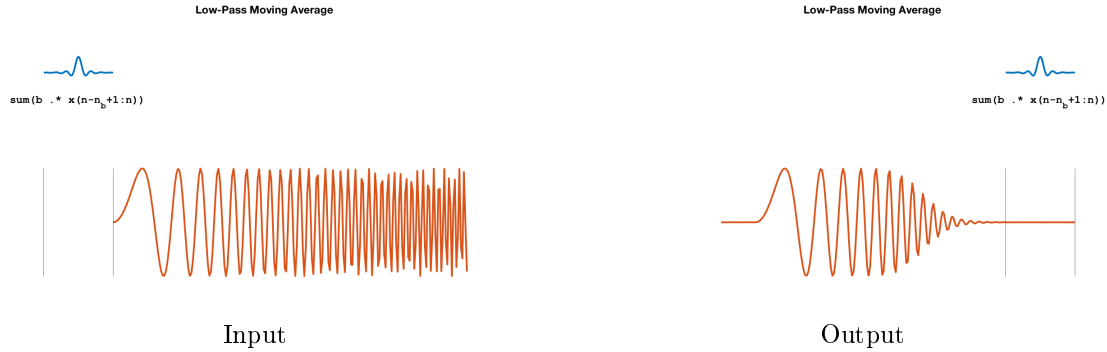
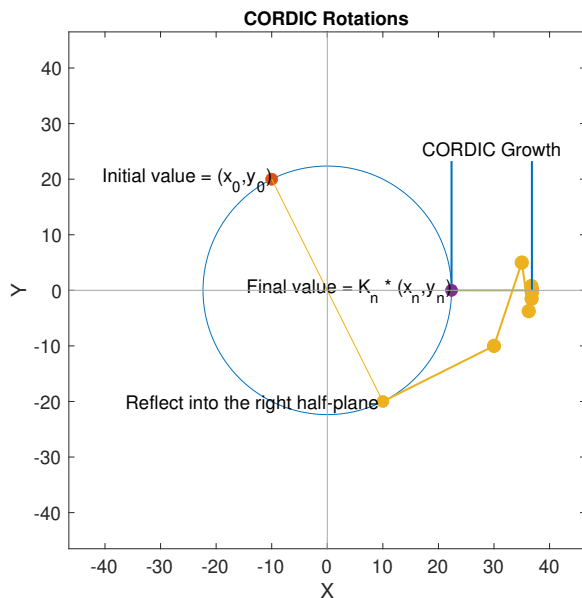


Figure 1.1: FINITE IMPULSE RESPONSE (FIR) FILTER IN ACTION

A low-pass moving-average filter passes low frequencies and attenuates high frequencies. Changing the coefficients changes which frequencies pass through the filter, and which frequencies are attenuated. (`filterMovieLow`)

1.2.2 Coordinate Rotation Digital Computer (CORDIC) Algorithm

The Coordinate Rotation Digital Computer (CORDIC) algorithm[24] is the backbone of many fixed-point algorithms from elementary functions (sine, cosine, square root, divide, etc.) to matrix factorizations (QR, SVD, etc.) to coordinate transformations (Givens rotations, Euler to north-east-down, oblique orthographic projection, etc.). It is efficient for fixed-point because it can be implemented using only shifts and adds.

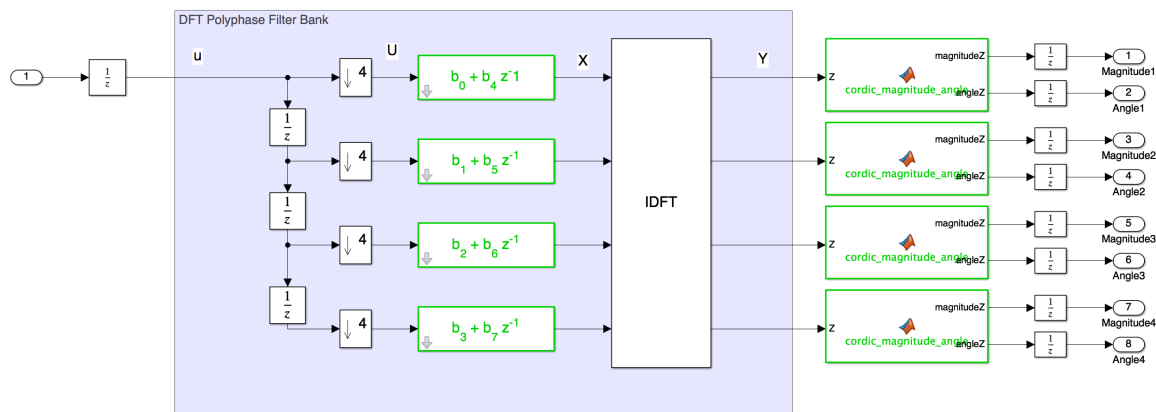


CORDIC ROTATIONS

`visualize_cordic(-10,20)`. The initial value is $(x, y) = (-10, 20)$.

1.2.3 Fixed-Point Polyphase DFT Channelized Receiver

A fixed-point channelized receiver in MATLAB is outlined by the following diagram.

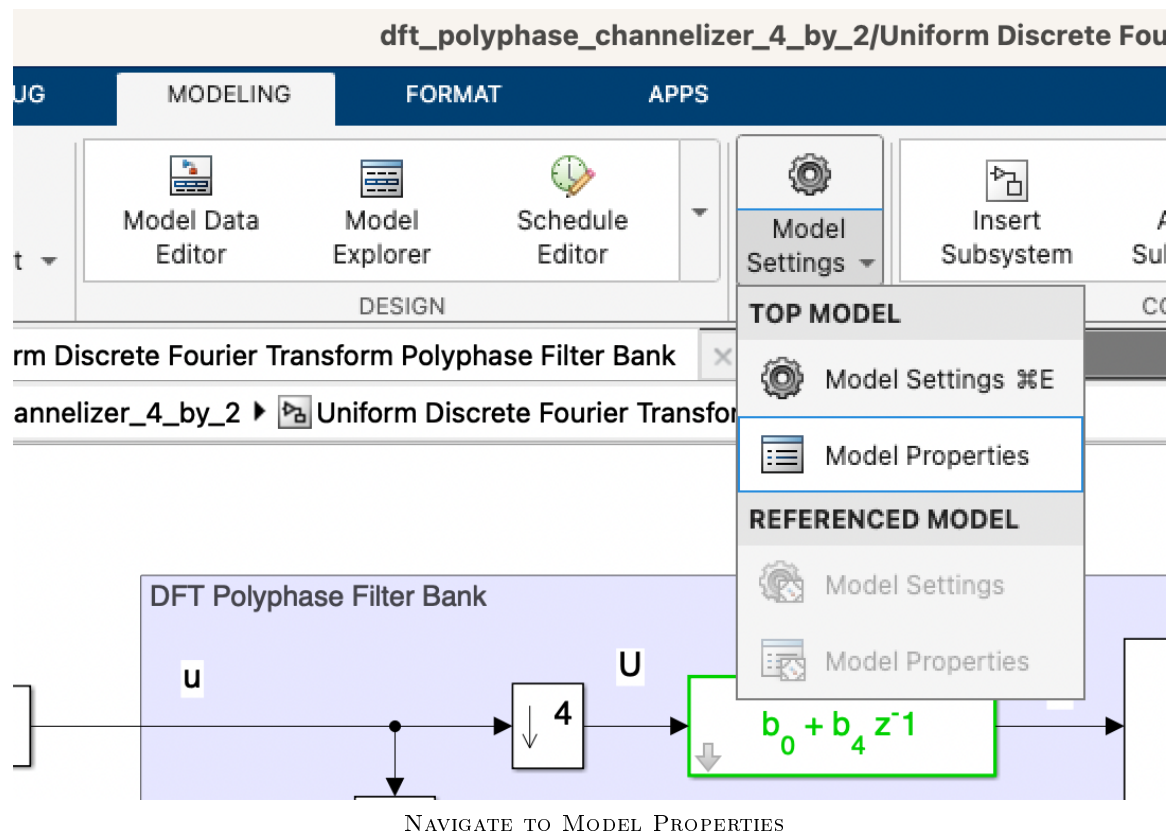


The shaded region in the diagram is a Discrete Fourier Transform (DFT) Polyphase Filter Bank [8], followed by a CORDIC implementation of converting a complex signal to magnitude and phase. If you pass a complex-valued chirp signal through it, you can see that each channel is a band-pass filter.

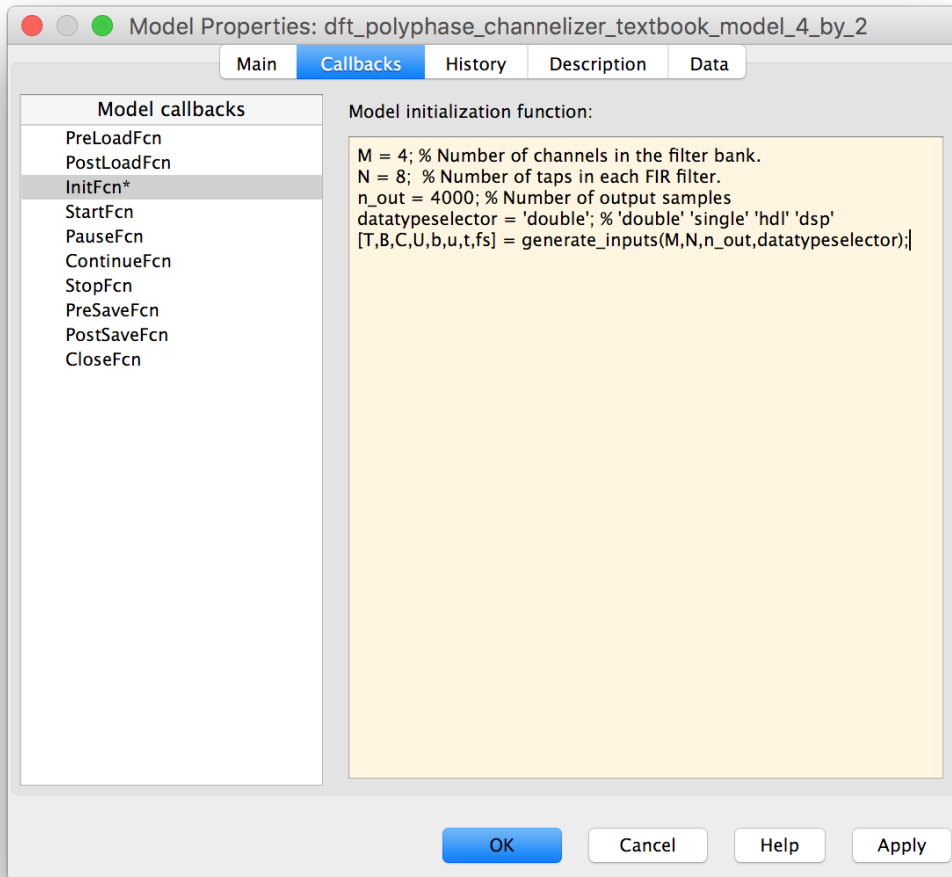
We're using this example because it ties together two algorithms we're interested in (FIR filters in a multi-rate system, and the CORDIC algorithm) into an end-to-end system. These functions are features of MATLAB and MathWorks toolboxes with HDL and C code generation support, but this workshop uses hand-written MATLAB code for these algorithms to teach you how to convert any MATLAB code to fixed point. Appendix B has a list of the equivalent toolbox functions and blocks.

Datatype Selection on the Model

In Simulink model `dft_polyphase_channelizer_4_by_2.slx`, to change the fixed-point settings and filter length, N , edit the settings in the model initialization function.



In the Modeling tab, select the down-arrow in Model Settings, and select Model Properties.



EDIT THE INITIALIZATION FUNCTION TO CHANGE DATA TYPES

Select Callbacks, then select InitFcn. Change the value of variable `datatypeselector` to one of `'double'`, `'single'`, `'hdl'`, or `'dsp'`, to change the data types of the model.

Alternatively, you can define parameters in a MATLAB script or load in the Model Workspace before running the model. Defining parameters in a MATLAB script or loading in the Model Workspace instead of in the model initialize function is better for running many different inputs through your model for testing. The workshop has the parameters defined in the model initialize function so each model has all its parameters defined when you press the play button on the model.

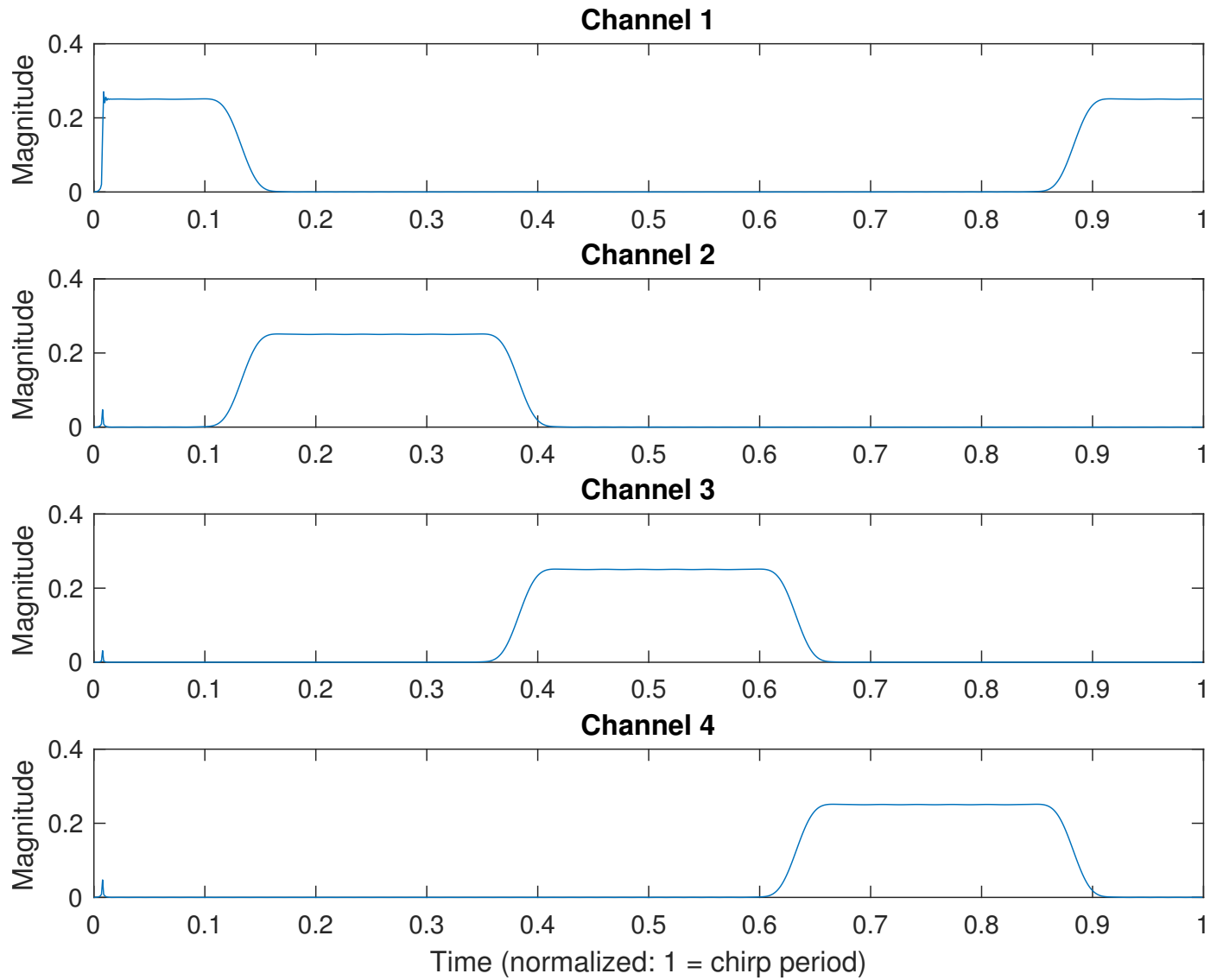
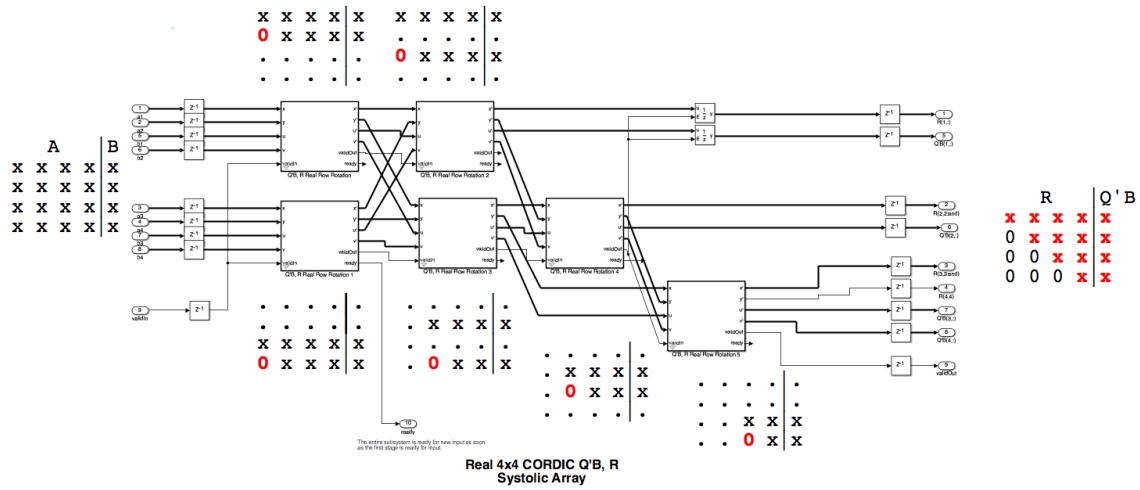


Figure 1.2: OUTPUT OF THE DFT POLYPHASE FILTER BANK IN RESPONSE TO A COMPLEX CHIRP INPUT WITH PERIOD EQUAL TO THE SIMULATION END TIME.

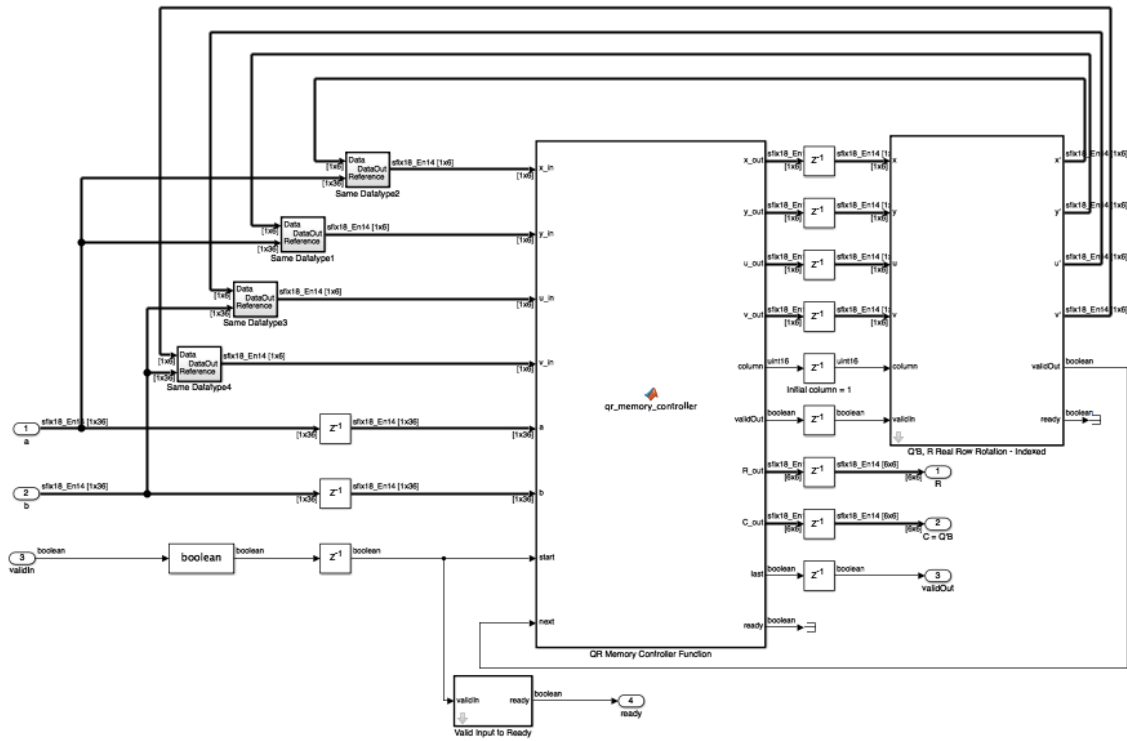
1.2.4 Fixed-Point Solution of Linear Equations

We will use the QR Decomposition to implement fixed-point code to solve systems of linear equations.

A fixed-point QR Decomposition in MATLAB implemented with CORDIC as a systolic array, and with shared resources are outlined in the following diagrams.



QR DECOMPOSITION IMPLEMENTED AS A SYSTOLIC ARRAY



QR DECOMPOSITION IMPLEMENTED WITH SHARED RESOURCES

1.2.5 Euler to North-East-Down (NED) Coordinate Transformation using CORDIC

We use the CORDIC technique to compute the Euler to North-East-Down (NED) transformation without explicitly performing any multiplications, or calculating any trigonometric functions directly. The NED transformation is used, for example, in aerospace coordinate systems [10].

$$R_\phi = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix}$$

$$R_\theta = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

$$R_\psi = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiplying these matrices together gives the total transformation given below:

$$A = R_\psi R_\theta R_\phi$$

$$= \begin{bmatrix} \cos(\psi) \cos(\theta) & \cos(\psi) \sin(\phi) \sin(\theta) - \cos(\phi) \sin(\psi) & \sin(\phi) \sin(\psi) + \cos(\phi) \cos(\psi) \sin(\theta) \\ \cos(\theta) \sin(\psi) & \cos(\phi) \cos(\psi) + \sin(\phi) \sin(\psi) \sin(\theta) & \cos(\phi) \sin(\psi) \sin(\theta) - \cos(\psi) \sin(\phi) \\ -\sin(\theta) & \cos(\theta) \sin(\phi) & \cos(\phi) \cos(\theta) \end{bmatrix}$$

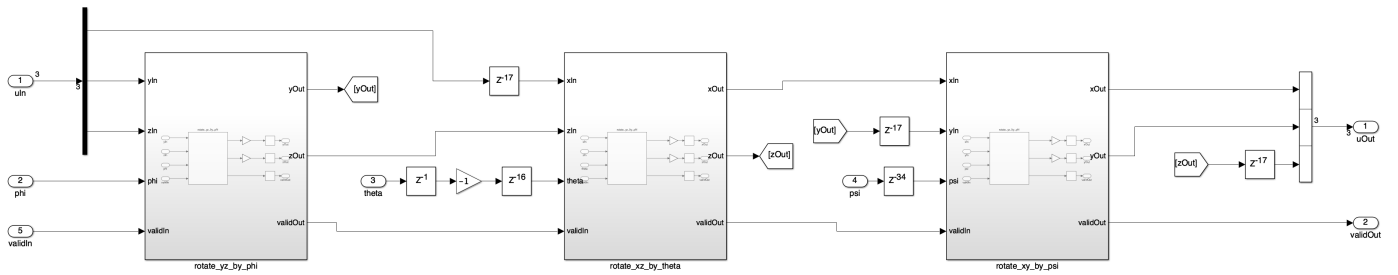


Figure 1.3: SIMULINK MODEL THAT CALCULATES THE EULER-NED TRANSFORMATION USING CORDIC.

The Oblique Orthographic Projection is a geometric transformation that maps a half-sphere onto the equatorial plane of the sphere that is used, for example, in Landsat and other near-polar-orbiting satellites [22].

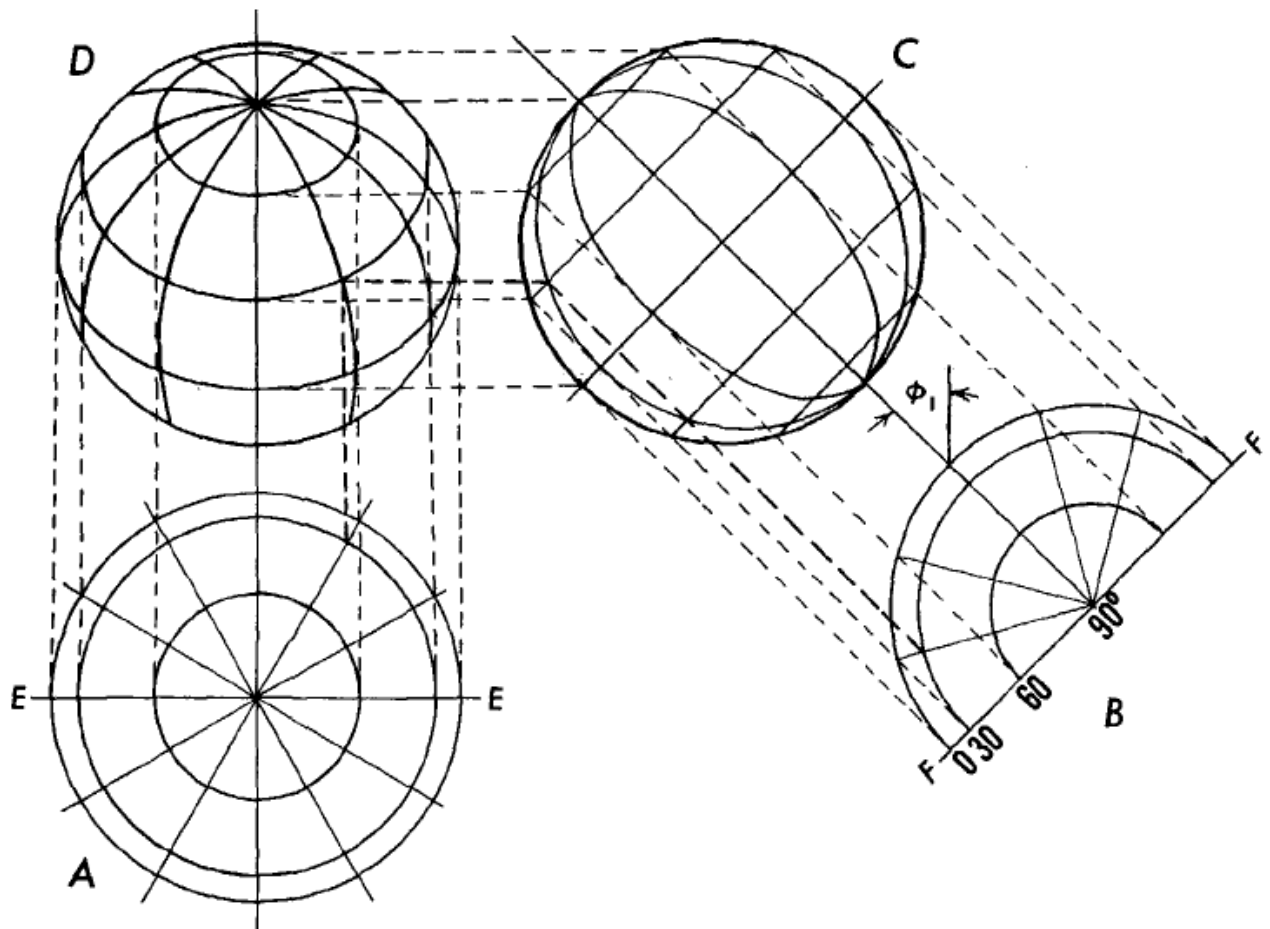


Figure 1.4: SCHEMATIC OF THE ORTHOGRAPHIC PROJECTION[22, p. 148].

$$\begin{aligned}
\begin{bmatrix} X \\ Y \\ H \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\pi/2 - \phi_0) & \sin(\pi/2 - \phi_0) \\ 0 & -\sin(\pi/2 - \phi_0) & \cos(\pi/2 - \phi_0) \end{bmatrix} \times \\
&\quad \begin{bmatrix} \cos(\lambda_0) & \sin(\lambda_0) & 0 \\ -\sin(\lambda_0) & \cos(\lambda_0) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \\
&\quad \begin{bmatrix} \cos(\lambda) & -\sin(\lambda) & 0 \\ \sin(\lambda) & \cos(\lambda) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \\
&\quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\pi/2 - \phi) & \sin(\pi/2 - \phi) \\ 0 & -\sin(\pi/2 - \phi) & \cos(\pi/2 - \phi) \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}
\end{aligned} \tag{1.1}$$

1.3 Workflow

Through experience, we found the following common workflow to be effective. Each step elaborates the design by adding more details about how the algorithm should be realized on embedded hardware. It breaks down the problem into tractable steps.

1. Develop algorithm in MATLAB.
2. Enable fixed-point data types in MATLAB code so that the code can run with different data types (double and single precision floating point, 14-bit fixed point, 18-bit fixed point, etc.).
3. Test MATLAB code with different fixed-point properties (dynamic range, different word lengths, saturate vs. wrap overflow, etc.) and pick the best set of properties for your application.
4. Experiment with different algorithm choices in the MATLAB code (e.g. different numbers of channels and different filter lengths).
5. Compare algorithm choices and fixed-point settings by running floating point and fixed point through the same code with different settings selected.
6. Look at tradeoffs of resources vs. speed using the MATLAB code.
7. If targeting HDL, use MATLAB code as a working design-document to create hardware-optimized Simulink models.
8. Use the MATLAB code to generate bit-faithful fixed-point input and output test vectors for testing the hardware.
9. Iterate until the algorithm meets your specifications.

In this workshop, we will cover everything it takes to accomplish this workflow.

1.4 Rounding Errors in the Vancouver Stock Exchange

The example in this section is a microcosm of the fixed-point MATLAB workflow that will be explained in detail in the following chapters. The example is in the following MATLAB Live Script.

`vancouver_stock.mlx`

In 1982, the Vancouver Stock Exchange started keeping a running index of all 1400 stocks in the exchange. Every time a stock price changed, which happened about 2800 times a day, the index was updated and truncated to 3 decimal digits (approximately 10 binary digits: $2^{10} = 1024$). The stock index was initialized to 1000. Instead of recomputing the index each time a stock price changed, the index y was updated by adding the price difference dx divided by the number of stocks in the exchange. After two years, the index was 480 points off [17]. The following is an example of the computation.

Put the random number generator in its default state for reproducible results.

```
rng('default')
```

n is the average number of price changes per day.

```
n = 2800;
```

dx is a vector of normally-distributed random price changes.

```
dx = 10*randn(n,1);
```

Coefficient a is the the inverse of the number of stocks.

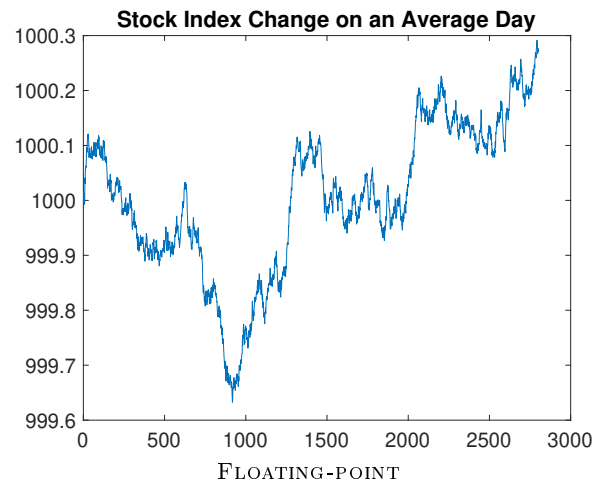
```
a = 1/1400;
```

Compute the stock index y as a running average by updating the stock index instead of averaging all the stock prices after every change in price. The computation is accurate in double-precision floating-point, which is MATLAB's default type. If you are familiar with signal processing or control theory, you will recognize this as a filter or controller with feedback, commonly known as an infinite impulse-response (IIR) filter or autoregressive (AR) model.

```
y = zeros(n,1);
y(1) = 1000;
for k = 2:n
    y(k) = y(k-1) + a * dx(k);
end

plot(y)
title('Stock Index Change on an Average Day')
```

Given the normally distributed random price changes, dx , you can see the stock index acts as expected—it goes up and down within the course of a day.

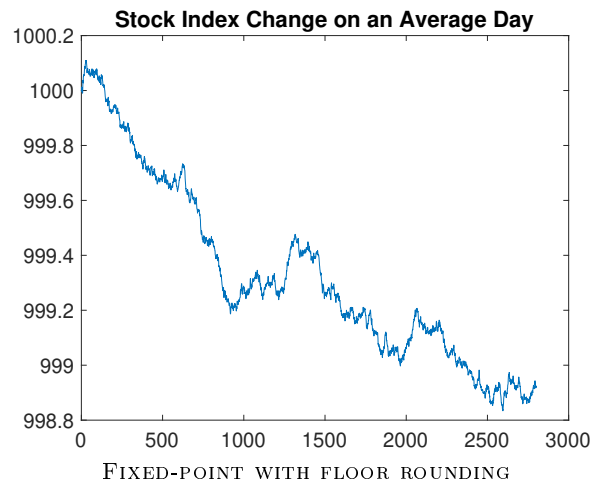


Now compute like it's 1982. Truncate the results to approximately 3 decimal digits (10 binary digits) by using a fixed-point type with 10 fractional bits. The method of creating a **struct** with a field (`T.y`), using it to create a variable 'like' it, and the effect of subscripted assignment (`y(k) = ...`) are described in detail in Chapters 2 and 4. Separating your datatypes from your algorithm enable you to keep one version of the algorithm for all datatypes. It is the keystone of the rest of this workshop.

```
% Type definition
T.y = fi([],1,32,10,'RoundingMethod','Floor');

% Algorithm
y = zeros(n,1,'like',T.y);
y(1) = 1000;
for k = 2:n
    y(k) = y(k-1) + a * dx(k);
end
```

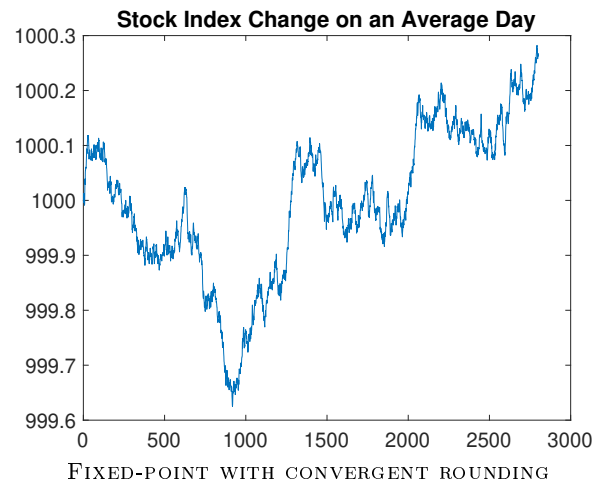
You can see that truncating to 10 binary digits causes all rounding to be down and the stock index starts to slide over the course of just one day. This was the cause of the Vancouver Stock Index to be off by 480 points over the course of two years.



More precision will not solve the problem. Biased rounding will eventually lead to large errors because of the feedback, like the screech of feedback in a microphone. More precision will just make the large errors be apparent at a later date. Even simple “round to nearest” rounding [16], [21] has a bias because the halfway points always round in the same direction. An unbiased rounding method in MATLAB is “convergent” rounding [6], which is a form of round-to-nearest where even ties go down and odd ties go up. Using convergent rounding keeps the computation of the stock index unbiased.

```
% Type definition
T.y = fi([],1,32,10,'RoundingMethod','Convergent');

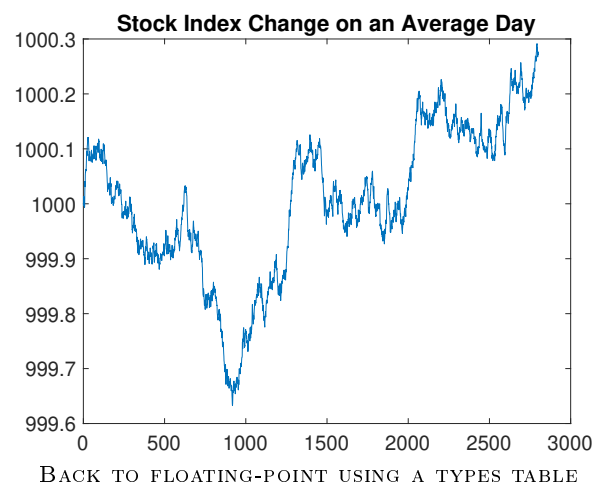
% Algorithm
y = zeros(n,1,'like',T.y);
y(1) = 1000;
for k = 2:n
    y(k) = y(k-1) + a * dx(k);
end
```



Using the types-table approach, we can return the algorithm to double-precision floating-point types by re-defining `T.y` without modifying the algorithm itself.

```
% Type definition
T.y = [];

% Algorithm
y = zeros(n,1, 'like', T.y);
y(1) = 1000;
for k = 2:n
    y(k) = y(k-1) + a * dx(k);
end
```



In 1982, only the stock index y was quantized. The number of stocks is an integer and the change in stock price is exact, so we leave them unquantized here with variables a and dx left in their original types. Also, we left the algorithm in-lined with the test file. Chapter 4 “Fixed-Point Conversion Process using FIR Filter” describes in detail the next steps of converting all data types to fixed point, and separating an algorithm from its test file.

Chapter 2

Numbers in Digital Hardware

The examples for this chapter are in directory

```
exercises/02numbers_in_digital_hardware
```

In digital hardware, numbers are stored in binary words. A binary word is a fixed-length sequence of binary digits (bits, ones and zeros). How hardware components or software functions interpret this sequence of ones and zeros is defined by the data type.

2.1 Floating-Point Numbers

Floating-point data types are characterized by a sign bit, a fraction (or mantissa) field, and an exponent field [13, 14]. The most common types of floating-point numbers are double-precision, which use 64 bits to represent the value, and single precision, which use 32 bits.

Because of the way numbers are represented in floating-point, floating-point data types can simultaneously represent both very large and very small values. However, due to the relatively large word size (64 and 32 bits) and details of floating-point arithmetic, computations using floating-point numbers are often more expensive on hardware than pure integer math.

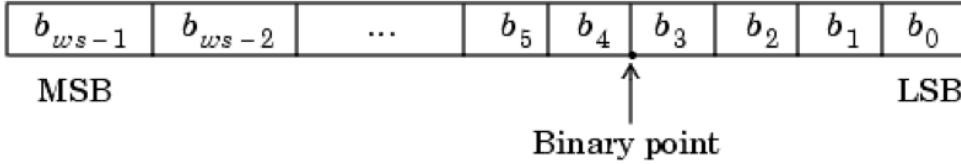
2.2 Fixed-Point Numbers

Fixed-point representation often offers advantages in terms of power consumption, size, memory usage, speed, and cost of the final product. However, fixed-point numbers are limited in that they cannot simultaneously represent very large or very small numbers using a reasonable word size.

A fixed-point data type is characterized by the word length in bits, the position of the binary point, and whether it is signed or unsigned. The position of the binary point is the means by

which fixed-point values are scaled and interpreted. The binary point is not physically represented in the hardware. The hardware manipulates purely integer data.

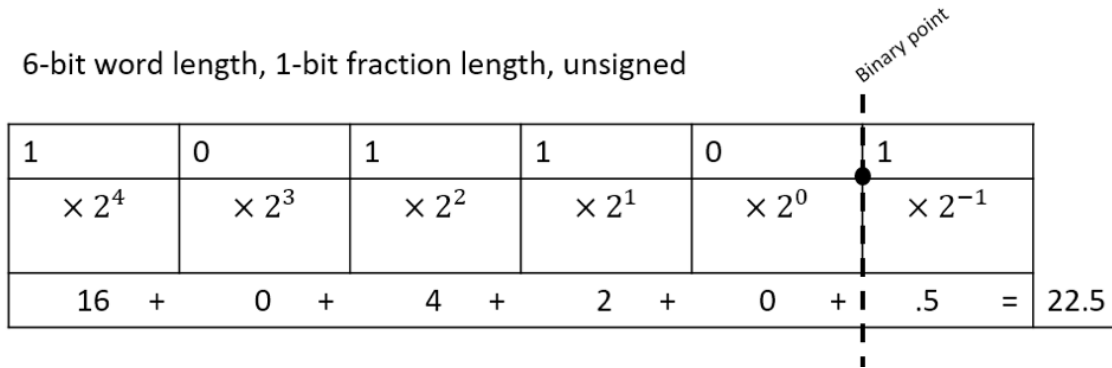
A common representation of a binary fixed-point number, either signed or unsigned, is shown in the following figure.



MSB = Most Significant Bit. LSB = Least Significant Bit. ws = word size

Signed data types can represent both positive and negative numbers, while unsigned data types can represent only positive numbers or zero.

The following figures show how to find the value of a number based on the binary representation of the number and its data type. The first figure shows a 6-bit number with an unsigned data type.



The following figure shows the same sequence of bits, but this time the data type is signed. The value represented is negative.

6-bit word length, 1-bit fraction length, signed

1	0	1	1	0	1	
$\times -2^4$	$\times 2^3$	$\times 2^2$	$\times 2^1$	$\times 2^0$	$\times 2^{-1}$	
-16	+	0	+	4	+	2
						+
						0
						+
						1.5
						=
						-9.5

The final figure shows another number with a signed data type. In this example the MSB (most significant, or first bit) is a zero, indicating that the value being represented is positive.

6-bit word length, 1-bit fraction length, signed

0	0	1	1	0	1	
$\times -2^4$	$\times 2^3$	$\times 2^2$	$\times 2^1$	$\times 2^0$	$\times 2^{-1}$	
0	+	0	+	4	+	2
						+
						0
						+
						1.5
						=
						6.5

When choosing a data type, you must consider these factors:

- The numerical range of the result
- The precision required of the result
- The associated quantization error (i.e., the rounding method)
- The method for dealing with exceptional arithmetic conditions such as overflows

These choices depend on your specific application, the computer architecture used, and the cost of development, among others.

2.3 Scaling

Binary-point-only or power-of-two scaling involves moving the binary point within the fixed-point word. The advantage of this scaling mode is to minimize the number of processor arithmetic operations.

The following equation describes how an integer is interpreted in binary-point scaling.

$$\text{real world value} = 2^{-\text{fraction length}} \cdot \text{stored integer}$$

2.4 Precision and Range

Because the dynamic range of fixed-point numbers is much less than floating-point numbers with equivalent word sizes, fixed-point numbers must be appropriately scaled.

The range of a number gives the limits of the representation, while the precision gives the distance between successive numbers in the representation. The range and precision of a fixed-point number depend on the length of the word and the scaling.

In general, hardware constraints result in trade-offs between the precision and range of a number.

2.5 Quantization

Quantization occurs when a data type cannot represent a value exactly. In these cases, the value must be rounded to the nearest value that can be represented by the data type.

In most cases, values are simply rounded to the nearest value representable by that data type. Some systems, however, are particularly sensitive to quantization error. This rounding introduces a small amount of error into the system that then accumulates and can ultimately have a much greater effect.

2.6 Fixed-Point Numbers in MATLAB

The default data type of variables in MATLAB is floating-point double. To specify a fixed-point data type, use the `fi` function.

```
>> a = fi(pi, 1, 16, 8)

a =

    3.1406

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 16
    FractionLength: 8
```

The first input to the `fi` constructor is the value, followed by the signedness (1 for signed, 0 for unsigned), then the word length, and finally the fraction length. If you do not specify any of the numeric properties the `fi` object uses the default values, and the fraction length that gives the best precision while avoiding an overflow.

```
>> a = fi(pi)

a =

    3.1416

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 16
    FractionLength: 13
```

2.7 Addition

The example in this section is in the following MATLAB Live Script.

`addition_example.mlx`

Whenever you add two fixed-point numbers, you may need a carry bit to correctly represent the result. For this reason, when adding two B-bit numbers (with the same scaling), the resulting value has an extra bit compared to the two operands used.

The addition of fixed-point numbers requires that the binary points of the addends be aligned. The addition is then performed using binary arithmetic so that no number other than 0 or 1 is used.

$$\begin{array}{r}
 010010.1 \quad (18.5) \\
 0110.110 \quad (6.75) \\
 \hline
 0011001.010 \quad (25.25)
 \end{array}$$

In the following example, note that the sum, c , of a and b has a word length (10) equal to one greater than the sum of the largest input fraction length (3), and the largest input integer length ($7 - 1 = 6$).

```

>> a = fi(18.5,0,7,1)

a =

    18.5000

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Unsigned
          WordLength: 7
       FractionLength: 1

>> a.bin

ans =

    '0100101'

```

```
>> b = fi(6.75,0,7,3)

b =

    6.7500

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Unsigned
    WordLength: 7
    FractionLength: 3

>> b.bin

ans =

    '0110110'
```

```
>> c = a + b

c =

    25.2500

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Unsigned
    WordLength: 10
    FractionLength: 3

>> c.bin

ans =

    '0011001010'
```

2.7.1 Code generation example for addition

You can concentrate on the high-level algorithm in MATLAB, and use MATLAB Coder and HDL Coder products to generate detailed implementations in C and HDL. Chapter 4 has methods to optimize C code.

Use variables a and b from the example, and the following MATLAB function.

```
function c = myplus(a,b)
    c = a + b;
end
```

C code generation

Generate C code using the following command.

```
codegen myplus -args {a,b} -config:lib -launchreport
```

```
unsigned short myplus(unsigned char a, unsigned char b)
{
    return (unsigned short)((int)((unsigned int)(a << 2) + b) & 1023);
}
```

HDL code generation

Generate HDL code using the following command.

```
codegen myplus -args {a,b} -config:hdl -launchreport
```

```
-- -----
--
-- Module: myplus
-- Source Path: myplus
-- Hierarchy Level: 0
--
-- -----
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY myplus IS
    PORT( a : IN      std_logic_vector(6 DOWNTO 0); -- ufix7_En1
          b : IN      std_logic_vector(6 DOWNTO 0); -- ufix7_En3
          c : OUT     std_logic_vector(9 DOWNTO 0)  -- ufix10_En3
    );
END myplus;

ARCHITECTURE rtl OF myplus IS

    -- Signals
    SIGNAL a_unsigned      : unsigned(6 DOWNTO 0); -- ufix7_En1
    SIGNAL b_unsigned      : unsigned(6 DOWNTO 0); -- ufix7_En3
    SIGNAL c_tmp           : unsigned(9 DOWNTO 0);  -- ufix10_En3
    SIGNAL myplus_add_cast : unsigned(9 DOWNTO 0); -- ufix10_En3
    SIGNAL myplus_add_cast_1 : unsigned(9 DOWNTO 0); -- ufix10_En3

BEGIN
    a_unsigned <= unsigned(a);

    b_unsigned <= unsigned(b);

    --HDL code generation from MATLAB function: myplus
    myplus_add_cast <= resize(a_unsigned & '0' & '0', 10);
    myplus_add_cast_1 <= resize(b_unsigned, 10);
    c_tmp <= myplus_add_cast + myplus_add_cast_1;

    c <= std_logic_vector(c_tmp);
END rtl;
```

SystemC code generation

Generate SystemC code using the following command.

```
cfg = coder.HdlConfig;  
cfg.TargetLanguage = "SystemC";  
codegen myplus -args {a,b} -config cfg -launchreport
```

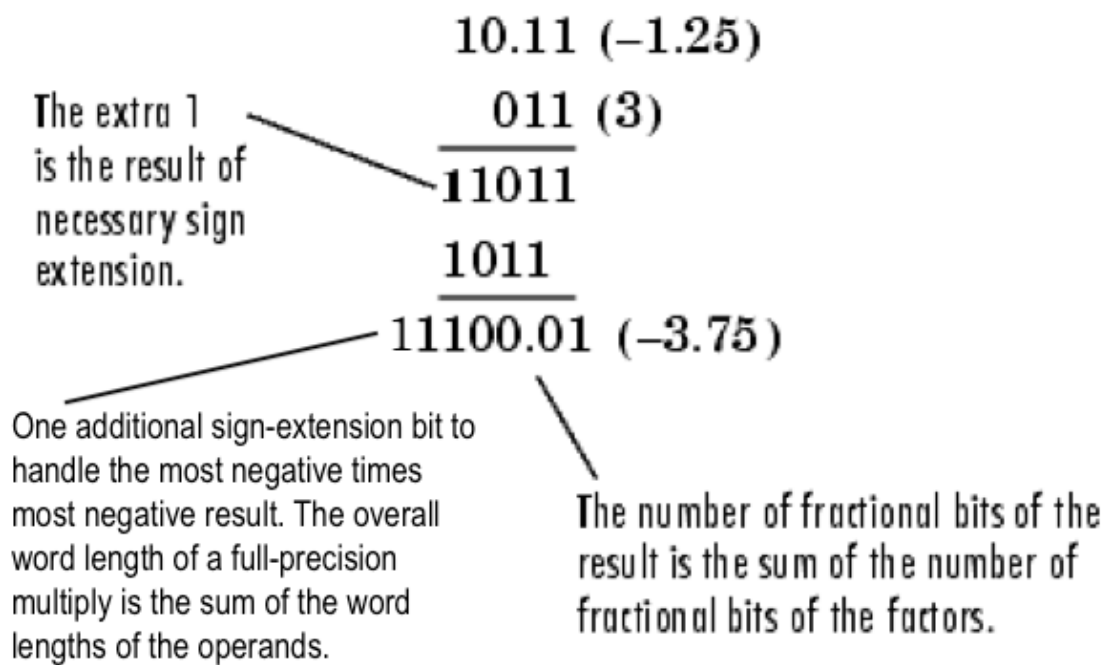
```
class myplusClass  
{  
    public:  
        sc_ufixed<10,7> myplus(sc_ufixed<7,6> a, sc_ufixed<7,4> b)  
        {  
            return (sc_ufixed<10,7>)a + (sc_ufixed<10,7>)b;  
        }  
};
```

2.8 Multiplication

The example in this section is in the following MATLAB Live Script.

`multiplication_example.mlx`

In general, a full precision product requires a word length equal to the sum of the word lengths of the operands.



In the following example, note that the word length of the product c is equal to the word length of a plus the word length of b . The fraction length of c is also equal to the fraction length of a plus the fraction length of b .

```
>> a = fi(-1.25,1,4,2)

a =

    -1.2500

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Signed
           WordLength: 4
      FractionLength: 2

>> a.bin

ans =

    '1011'
```

```
>> b = fi(3,1,3,0)

b =

     3

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Signed
           WordLength: 3
      FractionLength: 0

>> b.bin

ans =

    '011'
```

```
>> c = a*b

c =

    -3.7500

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 7
      FractionLength: 2

>> c.bin

ans =

    '1110001'
```

2.8.1 Code generation example for multiplication

You can concentrate on the high-level algorithm in MATLAB, and use MATLAB Coder and HDL Coder products to generate detailed implementations in C and HDL. Chapter 4 has methods to optimize C code.

Use variables a and b from the example, and the following MATLAB function.

```
function c = mytimes(a,b)
    c = a * b;
end
```

C code generation

Generate C code using the following command.

```
codegen mytimes -args {a,b} -config:lib -report
```

```
signed char mytimes(signed char a, signed char b)
{
    return (signed char)(a * b);
}
```

HDL code generation

Generate HDL code using the following command.

```
codegen mytimes -args {a,b} -config:hdl -report
```

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY mytimes IS
  PORT( a : IN      std_logic_vector(3 DOWNTO 0); -- sfix4_En2
        b : IN      std_logic_vector(2 DOWNTO 0); -- sfix3
        c : OUT     std_logic_vector(6 DOWNTO 0)  -- sfix7_En2
        );
END mytimes;

ARCHITECTURE rtl OF mytimes IS

  -- Signals
  SIGNAL a_signed : signed(3 DOWNTO 0); -- sfix4_En2
  SIGNAL b_signed : signed(2 DOWNTO 0); -- sfix3
  SIGNAL c_tmp    : signed(6 DOWNTO 0); -- sfix7_En2

BEGIN
  a_signed <= signed(a);

  b_signed <= signed(b);

  --HDL code generation from MATLAB function: mytimes
  c_tmp <= a_signed * b_signed;

  c <= std_logic_vector(c_tmp);
END rtl;
```

SystemC code generation

Generate SystemC code using the following command.

```
codegen mytimes -args {a,b} -config:hdl -report
```

```
class mytimesClass
{
public:
    sc_fixed<7,5> mytimes(sc_fixed<4,2> a, sc_int<3> b)
    {
        return a * b;
    }
};
```

Chapter 3

Basics and Best Practices

The best practices are summarized in the following webinar and article from MathWorks: *Best Practices for Converting MATLAB Code to Fixed Point*, [1], [2]. The best practices article is also included in the following folder in this workshop:

```
references/Best_Practices_for_Converting_MATLAB_Code_to_Fixed_Point.pdf
```

3.1 Data types in MATLAB

Let's start with a relatively new feature that makes using different data types in MATLAB a lot easier.

In R2013a, the 'like' syntax in `cast`, `zeros`, and `ones` was introduced. It allows you to separate your algorithm from your type definitions. This feature plays a key role in readable, maintainable fixed-point code in MATLAB.

Most people never have to think about MATLAB having different data types, and never have to think about how an algorithm is implemented. But anyone working with embedded systems has to care.

Since you don't declare types in MATLAB, it's easy to forget that by default MATLAB uses 64-bit double-precision floating point types.

```
>> clear
>> A = magic(3)

A =

     8     1     6
     3     5     7
     4     9     2

>> whos

  Name      Size      Bytes  Class  Attributes
  ----  -----  -
  A         3x3         72  double
```

But there are different types in MATLAB, like 32-bit single-precision floating-point,

```
>> A = single(magic(3))

A =

 3x3 single matrix

     8     1     6
     3     5     7
     4     9     2
```

and builtin integers.

```
>> A = int8(magic(3))

A =

 3x3 int8 matrix

     8     1     6
     3     5     7
     4     9     2
```

Starting in the R2019b release of MATLAB, you can write hexadecimal values as literals using the prefix 0x, and binary numbers using the prefix 0b (see documentation at [Hexadecimal and Binary Values](#)). By default, MATLAB stores the number as the smallest unsigned integer type that can accomodate it.

```
>> A = 0x2A

A =

    uint8

    42
```

The leading 1 in this signed binary number indicates that it is a negative value.

```
>> A = 0b11001100110011001100110011001100110011001100110011001100110011001101

A =

    uint64

 14757395258967641293
```

You can specify unsigned 8-, 16-, 32-, and 64-bit integer types using the suffixes u8, u16, u32, and u64.

```
>> A = 0x2Au32

A =

    uint32

    42
```

You can specify signed 8-, 16-, 32-, and 64-bit integer types using the suffixes `s8`, `s16`, `s32`, and `s64`.

```
>> A = 0x2As32  
  
A =  
  
    int32  
  
    42
```

The leading 1 in this signed binary number indicates that it is a negative value.

```
>> A = 0b11001100110011001100110011001100110011001100110011001100110011001101s64  
  
A =  
  
    int64  
  
   -3689348814741910323
```

Fixed-point types are created using the `fi` constructor.

```
fi(value, signedness, wordlength, fractionlength)
```

```
>> A = fi(magic(3), true, 32, 24)
```

```
A =
```

```
      8      1      6
      3      5      7
      4      9      2
```

```
      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 32
      FractionLength: 24
```

Tom Bryan named it `fi` because J.H. Wilkinson [15] was the first to use `fi` for fixed-point quantization and `fl` for floating-point quantization in 1963 in his book *Rounding Errors in Algebraic Processes* [29]. Tom also liked the name because `fi` is bit-faithful to fixed point in embedded hardware, like the US Marine Corps motto, *Semper fi*, “always faithful” (`fi` is short for the Latin word *fidelis*, “faithful”).

Fixed-point types are defined by their signedness (signed=true or unsigned=false), and their underlying stored integer values, which is what the machine works on,

```
>> A.int

ans =

    3x3 int32 matrix

    134217728    16777216    100663296
     50331648     83886080    117440512
     67108864    150994944     33554432
```

and the value of their least-significant bit (also known as `eps(A)`),

```
>> A.lsb

ans =

    5.960464477539062e-08

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 24
```

which is two to the negative fraction length.

```
>> 2^-24

ans =

    5.960464477539062e-08
```

So the human-readable “real world value” is the underlying stored integer multiplied by the value of the least significant bit.

```
>> double(A.int) * double(A.lsb)

ans =

     8     1     6
     3     5     7
     4     9     2
```

3.2 `cast(x, 'like', T.x)`, `zeros(m,n, 'like', T.z)`

In MATLAB, you can't declare a variable's type, but you can cast to a particular type, like this:

```
>> A = fi(magic(3), true, 32, 24);
>> B = cast(0, 'like', A)

B =

    0

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 24
```

And you can create an array of zeros or ones the same way:

```
>> Z = zeros(2,3, 'like', A)

Z =

    0    0    0
    0    0    0

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 24
```

```
>> N = ones(2,3, 'like', A)

N =

    1    1    1
    1    1    1

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 24
```

Using 'like' with `zeros`, `ones`, and `cast` was introduced in R2013a. You could have cast to builtin types for many years, but that didn't work for fixed-point types.

It's useful to write fixed-point code this way, because now you have a way to separate the data-type specification from the algorithm.

For example, if A changes type and you create B again, then B is created with A 's type. Note that B and A are not dynamically related to one another. You have to call `B = cast(0,'like',A)` again to change B 's type. Re-casting will happen naturally as we will see in Chapter 4.

```
>> A = single([]); % (You don't care about its value)
>> B = cast(0,'like',A)

B =

    single

     0
```

```
>> A = int8([]);
>> B = cast(0,'like',A)

B =

    int8

     0
```

```
>> A = fi([]);
>> B = cast(0,'like',A)

B =

     0

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 16
      FractionLength: 15

>> class(B)

ans =

embedded.fi
```

3.3 Subscripted assignment $A(:)=B$

Another useful trick is how to coerce a variable to retain its type.

In MATLAB you know that if you have a matrix, A , and assign it to another value, then the entire matrix is replaced by the right-hand-side.

```
>> A = magic(3)
```

```
A =
```

```
      8      1      6
      3      5      7
      4      9      2
```

```
>> A = 0
```

```
A =
```

```
      0
```

But if you index into the matrix, then you only replace that element.

```
>> A = magic(3)
```

```
A =
```

```
      8      1      6
      3      5      7
      4      9      2
```

```
>> A(1,1) = 0
```

```
A =
```

```
      0      1      6
      3      5      7
      4      9      2
```

You are probably used to flattening out a matrix using `A(:)`.

```
>> A = magic(3);  
>> A(:)  
  
ans =  
  
     8  
     3  
     4  
     1  
     5  
     9  
     6  
     7  
     2
```

You can also use colon indexing to overwrite all its values.

```
>> A(:) = 0  
  
A =  
  
     0     0     0  
     0     0     0  
     0     0     0
```

Notice the difference between colon indexing, also referred to as subscripted assignment, and direct assignment. With direct assignment, *A* does not maintain its dimensions.

```
>> A = 0  
  
A =  
  
     0
```

3.3.1 Subscripted assignment to retain datatype

You're used to indexing into a matrix to retain its shape. The same trick can be used to retain its type.

For example, create a signed 8-bit integer, A .

```
>> A = int8(2)

A =

    int8

     2
```

Using direct assignment overwrites A and it becomes a double.

```
>> A = 0

A =

     0

>> class(A)

ans =

double
```

You can retain its type using the same subscripted assignment that you used to retain its shape.

```
>> A = int8(2);
>> A(:) = 0

A =

    int8

     0
```

The cast was automatically applied so that A is still an `int8`. You replaced all of its elements with the right-hand-side. It didn't matter that it was a scalar. Indexing into a variable like this works the same for scalars as it does for vectors and matrices.

```
>> A(1) = 0

A =

    int8

     0
```

Subscripted assignment is particularly useful to retain a fixed-point data type.

```
>> A = fi(pi)

A =

    3.1416

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 16
      FractionLength: 13
```

```
>> A(:) = 2

A =

     2

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 16
      FractionLength: 13
```

“Cast-like” and subscripted assignment are two key programming patterns that you will use to convert every-day MATLAB code into code that can be used by any data type.

Chapter 4

Fixed-Point Conversion Process using FIR Filter

The examples for this chapter are in directory

```
exercises/04conversion_with_fir_filter
```

4.1 Workflow

We will go through the following steps to convert a MATLAB algorithm from floating-point to fixed-point and make it generate efficient C and HDL code.

Through experience, we found the following common workflow to be effective. Each step elaborates the design by adding more details about how the algorithm should be realized on embedded hardware. It breaks down the problem into tractable steps.

1. Develop algorithm in MATLAB.
2. Enable fixed-point data types in MATLAB code so that the code can run with different data types (double and single precision floating point, 14-bit fixed point, 18-bit fixed point, etc.).
3. Test MATLAB code with different fixed-point properties (dynamic range, different word lengths, saturate vs. wrap overflow, etc.) and pick the best set of properties for your application.
4. Experiment with different algorithm choices in the MATLAB code (e.g. different numbers of channels and different filter lengths).
5. Compare algorithm choices and fixed-point settings by running floating point and fixed point through the same code with different settings selected.
6. Look at tradeoffs of resources vs. speed using the MATLAB code.
7. If targeting HDL, use MATLAB code as a working design-document to create hardware-optimized Simulink models.

8. Use the MATLAB code to generate bit-faithful fixed-point input and output test vectors for testing the hardware.
9. Iterate until the algorithm meets your specifications.

4.2 Algorithm to Convert

We're going to use a simple finite-impulse-response (FIR) filter as an example of how to convert MATLAB code to fixed-point. The same conversion steps in this example can be used to build up to any size problem.

An FIR filter is a weighted moving average. Given a vector of coefficients $b = [b_0, b_1, b_2, b_3]$, and input at time n , $x(n)$, then the output at time n is:

$$y(n) = b_0x(n) + b_1x(n-1) + b_2x(n-2) + b_3x(n-3).$$

It is called a Finite Impulse Response filter because if the input signal x is an impulse $x = [1, 0, 0, \dots]$, then the response y becomes exactly zero after a finite amount of time $y = [b_0, b_1, b_2, b_3, 0, 0, \dots]$. You can see this in MATLAB with the following commands.

```
b = 1:4;
x = [1 0 0 0 0 0 0 0];
y = filter(b,1,x)

y =

    1    2    3    4    0    0    0    0
```

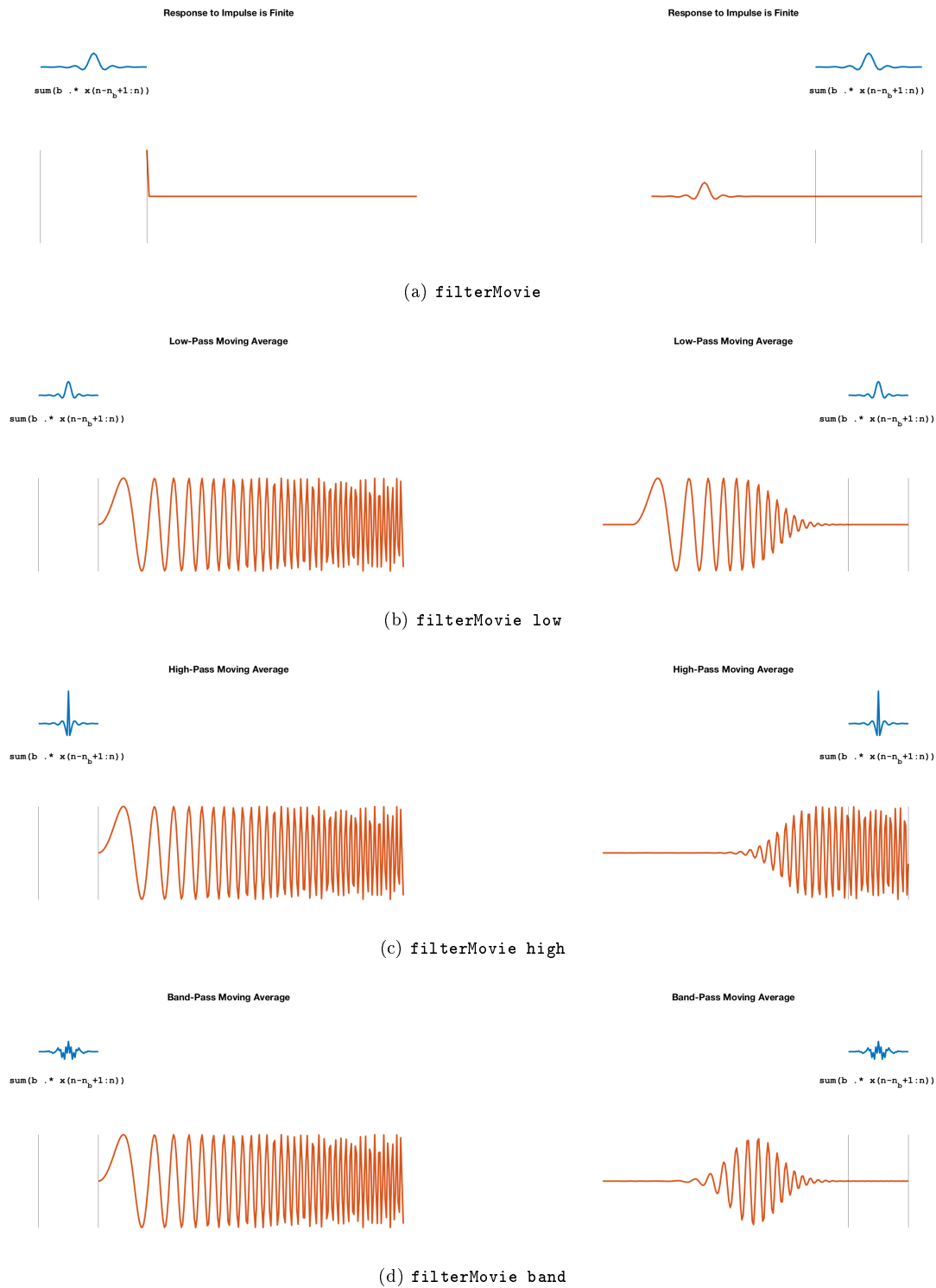


Figure 4.1: FINITE IMPULSE RESPONSE (FIR) FILTER IN ACTION

The response of a moving-average filter to an impulse, and chirp inputs. Changing the coefficients changes which frequencies pass through the filter, and which frequencies are attenuated.

4.2.1 Linear Buffer Implementation

There are many ways to implement an FIR filter. One way is with a linear buffer like in the following function, where b is a row vector of coefficients and z is a vector the same size as b .

```
function [y,z] = fir_filter_linear_buffer(b,x,z)
    y = zeros(size(x));
    for n=1:length(x)
        z = [x(n), z(1:end-1)];
        y(n) = sum(b .* z);
    end
end
```

The linear buffer implementation takes advantage of MATLAB's convenient matrix syntax and is easy to read and understand. However, it introduces a full copy of the state buffer z for every sample of the input.

State vector z is made up of the current and previous samples of x :

```
z = [x(n); x(n-1); ... ; x(n-length(b)+1)]
```

Vector z is used as both an input and an output. When it is an input, it is the initial state of the filter. When it is an output, it is the final state of the filter. Defining the state vector outside the function allows you to continuously stream data through the filter in a real-time system, and to reuse the filter algorithm within the same project.

The state vector is often named z because it's synonymous with the “delays” or “memory elements” z^{-1} associated with the Z -transform. The Z -transform was named in honor of Lotfi Zadeh for his seminal work in this area [19, 20]. Coincidentally, Zadeh was also the father of Fuzzy Logic [31] and the Fuzzy Logic Toolbox in MATLAB [9].

4.2.2 Circular Buffer Implementation

The following is the code that we're going to convert to fixed-point. It implements an FIR filter with a circular buffer.

```
function [y,z,p] = fir_filter(b,x,z,p)
    y = zeros(size(x));
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = 0;
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end
```

The `fir_filter` function stores the previous values of the input x in circular buffer z .

Using a circular buffers is efficient for many hardware implementations because it minimizes copying the data. Variable p is the pointer to the current location of the circular buffer z .

Variables z and p are used in both the input and output to pass the current state of these variables back into the function when it is called repeatedly over time — in particular, when x is input one scalar at a time as the data is streaming through the filter.

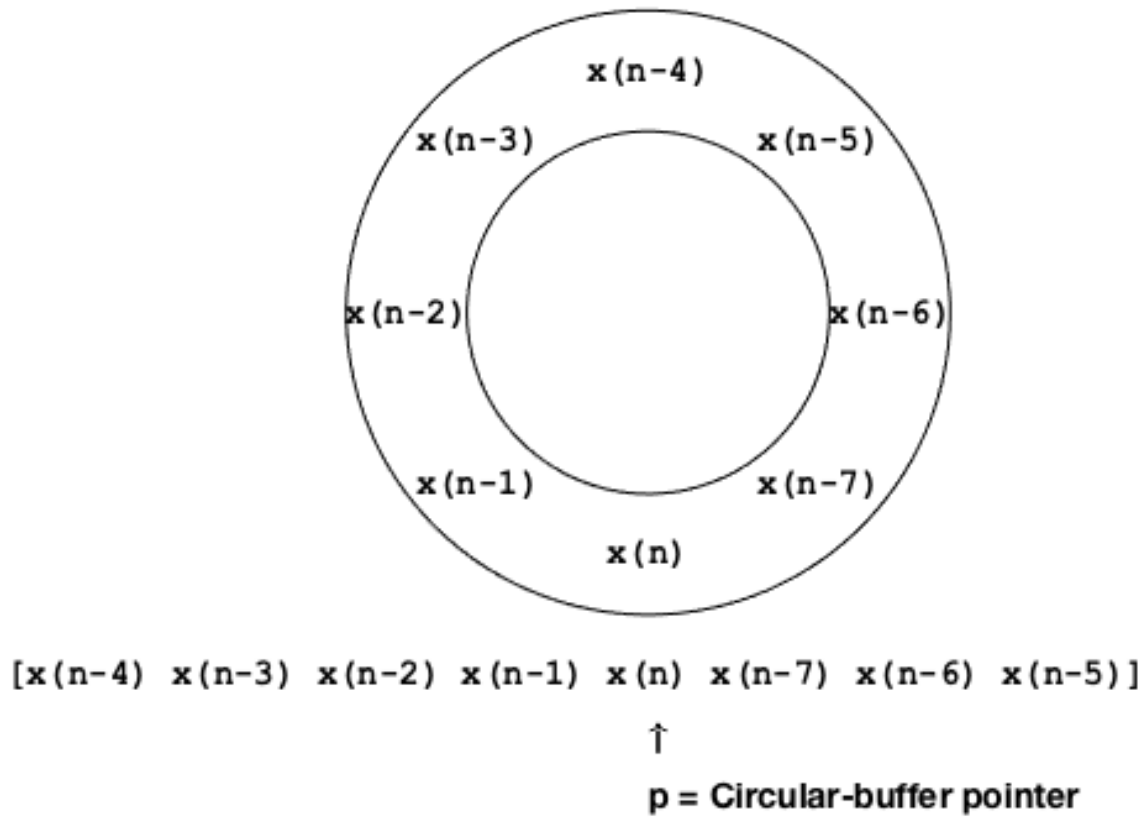


Figure 4.2: CIRCULAR BUFFER

Circular-buffer pointer $p = 5$ in this example. When p increments past the length of the buffer (8 in this case), then it starts again at 1.

4.3 Prepare Code for Instrumentation, Fixed-Point Conversion, and Acceleration

Since the conversion process relies on code-generation technology, the first step is to make sure that code-generation will work on the function. Code generation is supported for most fundamental operations, but not all.

4.4 C++ Analogy: Template Functions

The MATLAB fixed-point conversion process is analogous to C++ template functions. In C++, you can create a template function, like the following, found in file `fir_filter_template.cpp`. This function sums the elements of vector x and returns the sum in output y . This is the example that we will use throughout the conversion process in this section.

The datatypes are defined by template parameters `Tcoefficient`, for the type of coefficient vector `b`, `Tinput` for the type of input `x`, and `Tsum` for the type of the sum stored in the accumulator `acc`. The advantage of template functions is that you can re-use the same algorithm for different data types without changing the algorithm.

```
template <typename Tcoefficient,
          typename Tinput,
          typename Tsum>
Tsum fir_filter(std::vector<Tcoefficient> b,
               Tinput x,
               std::vector<Tinput>& z,
               size_t &p)
{
    const size_t nb = b.size();
    Tsum acc;
    size_t k;
    p++; if (p>=nb){p=0;}
    z[p] = x;
    k = p;
    acc = 0;
    for (size_t j = 1; j <= nb; j++) {
        k++; if (k>=nb){k=0;}
        acc += b[nb - j] * z[k];
    }
    return acc;
}
```

You can call the template function with single-precision floating-point types like the following:

```
// Number of inputs
const size_t N = 10;

// Coefficients
// T.coefficient = single([]);
// b = cast(fir1(7,0.5),'like',T.coefficient)
std::vector<float> b = {-0.0052, -0.0229, 0.0968, 0.4313,
                      0.4313, 0.0968, -0.0229, -0.0052};

// Impulse input
// T.input = single([]);
// x = zeros(1,N,'like',T.input);
// x(1) = 1;
std::vector<float> x(N,0.0);
x[0] = 1.0;

// z = zeros(size(b),'like',T.input)
std::vector<float> z(b.size(),0.0);
std::vector<float> y(N);
size_t p = 0;
for (int n=0; n<N; n++) {
    y[n] = fir_filter<float,float,float>(b,x[n],z,p);
}
std::cout << std::endl << "float type" << std::endl;
for (int n=0; n<N; n++) {
    std::cout << "y["<n<<" ] = " << y[n] << std::endl;
}
```

You can call the template function with short integer input type, and a longer integer type for the accumulated output, like the following. Using an accumulator with higher precision is common in fixed-point algorithms.

```
// Number of inputs
const size_t N = 10;

// Coefficients
// T.coefficient = fi([],1,16,15);
// b = cast(fir1(7,0.5),'like',T.coefficient)
// int(b)
std::vector<short> b = {-169, -750, 3170, 14133,
                      14133, 3170, -750, -169};

// Impulse input
// T.input = fi([],1,16,14);
// x = zeros(1,N,'like',T.input);
// x(1) = 1;
std::vector<short> x(N,0);
x[0] = 16384; // = 1 * 2^14 = int(x(1))

// z = zeros(size(b),'like',T.input)
std::vector<short> z(b.size(),0);

// T.output = fi([],1,16,14);
// y = zeros(1,N,'like',T.output)
std::vector<short> y(N);
size_t p = 0;
for (int n=0; n<N; n++) {
    // Cast sum type to output type by right-shifting
    y[n] = fir_filter<short,short,int>(b,x[n],z,p) >> 14;
}
std::cout << std::endl << "fixed-point type" << std::endl;
for (int n=0; n<N; n++) {
    std::cout << "y["<n<<" ] = " << y[n] << std::endl;
}
```

Compile and run `fir_filter_template.cpp` on Mac or Unix from the command-line. The test case was an impulse input, so the first output elements are the coefficients, and zero afterwards.

```
g++ -std=c++11 fir_filter_template.cpp
./a.out
```

```
float type
y[0] = -0.0052
y[1] = -0.0229
y[2] = 0.0968
y[3] = 0.4313
y[4] = 0.4313
y[5] = 0.0968
y[6] = -0.0229
y[7] = -0.0052
y[8] = 0
y[9] = 0
```

The following are the integers stored in the fixed-point values. They have the same “real-world value” as the floating-point values.

```
fixed-point type
y[0] = -169
y[1] = -750
y[2] = 3170
y[3] = 14133
y[4] = 14133
y[5] = 3170
y[6] = -750
y[7] = -169
y[8] = 0
y[9] = 0
```

4.5 Templated MATLAB Function

The following MATLAB function is the equivalent of the C++ template function in the previous section, and is the end-goal of the exercise in this section. The type of input x and coefficient vector b determined by the caller of this function, the type of output y is determined by creating a vector of zeros with template parameter `T.output`, and the type of the sum in the accumulator `acc` is determined by casting to template parameter `T.sum`. The datatype of `acc` is preserved through the sum `acc(:) = acc + b(j)*z(k)` by using subscripted assignment (colon-indexing).

```
function [y,z,p] = fir_filter(b,x,z,p,T)
    y = zeros(size(x),'like',T.output);
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = cast(0,'like',T.sum);
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc(:) = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end
```

4.6 Conversion Heuristic

The pattern for most MATLAB fixed-point conversion is:

- Create template parameters in MATLAB structs.
- Cast first definitions of variables.
- Use subscripted assignment (colon-indexing) for subsequent assignments into the same variable.

The remaining conversion items are the following, which won't be covered in this workshop.

- Dealing with functions that do not support fixed-point. (Generally, you cast inputs to double and outputs back to fixed-point until you have a function replacement ready. Create fixed-point versions of functions using the methods in this workshop.)
- Changing algorithms depending on the target (e.g. CORDIC vs. lookup table implementations).
- Mixed floating-point and fixed-point code. (Generally, use different types in your template parameter tables, and `propagateFloat` for promoting fixed-point operating on floating-point to floating-point.)
- Optimizing HDL for resources, speed, and latency.

4.7 (1) Separate algorithm from test script

The first step in the conversion is to separate the algorithm code from test script code.

4.7.1 Inlined algorithm and test script

For example, the following MATLAB script has the algorithm inlined in the test script.

algorithm_intermingled.m

```
% TEST INPUT
[b,x,z,p,t] = fir_filter_inputs;
z = zeros(size(b));
p = 1;
y = zeros(size(x));

% ALGORITHM
for n=1:length(x)
    p = incrementCircularPointer(p,length(b));
    z(p) = x(n);
    acc = 0;
    k = p;
    for j = length(b):-1:1
        k = incrementCircularPointer(k,length(b));
        acc = acc + b(j)*z(k);
    end
    y(n) = acc;
end

% VERIFY RESULTS
clf
h1 = subplot(2,1,1);
plot(t,x,t,y)
legend('Chirp Input','Filtered Output')
title('Filter Output')
h2 = subplot(2,1,2);
plot(t,y-filter(b,1,x));
title('Difference between algorithm and builtin MATLAB filter');
linkaxes([h1 h2],'x')
```

4.7.2 Algorithm

The best practice for converting to fixed-point is to separate the algorithm from the test script, like the following MATLAB function.

The algorithm is what we intend to put on an embedded processor and should not contain things like plot and display.

```
function [y,z,p] = fir_filter(b,x,z,p)
    y = zeros(size(x));
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = 0;
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end
```

4.7.3 Test script

Put the creation of test input data, plotting, and verification in test script, `test_fir_filter.m`.

```
% (1) Inputs for test
[b,x,z,p,t,tchirp] = fir_filter_inputs;

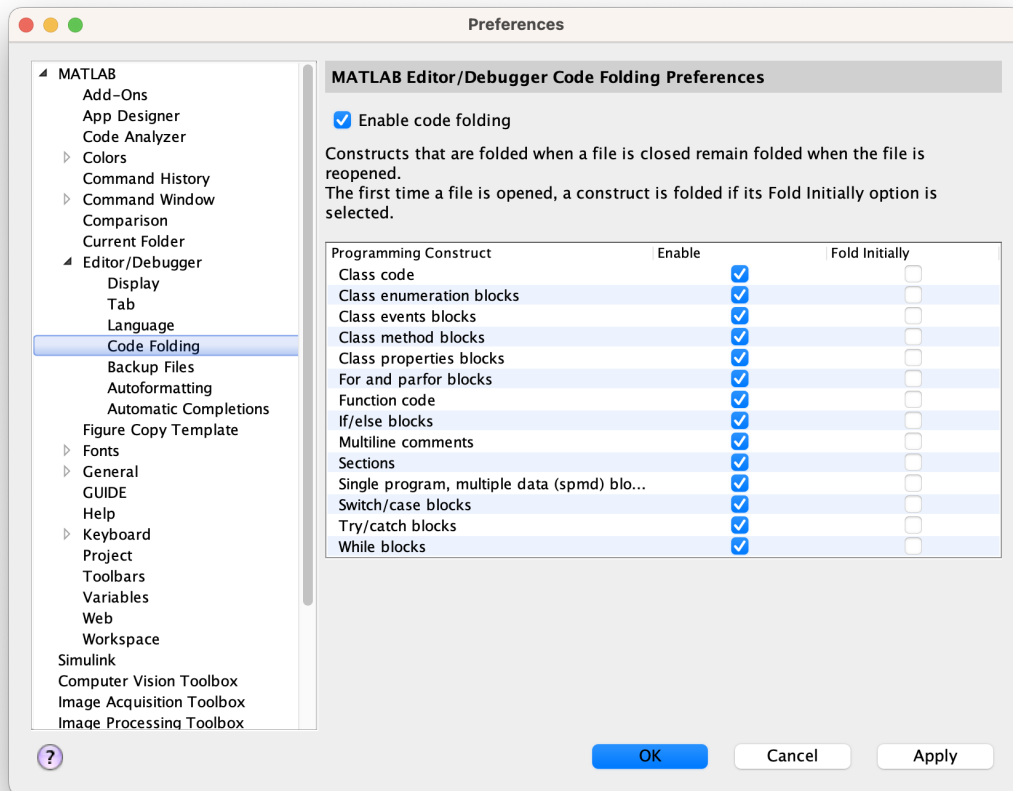
% (1) Builtin filter for comparison
y0 = filter(b,1,x);
fir_filter_plot1(t,tchirp,x,y0);

% (1,2,3) Run
[y,z,p] = fir_filter(b,x,z,p);

% (1) Verify results
fir_filter_plot2(t,tchirp,x,y,y0);
```

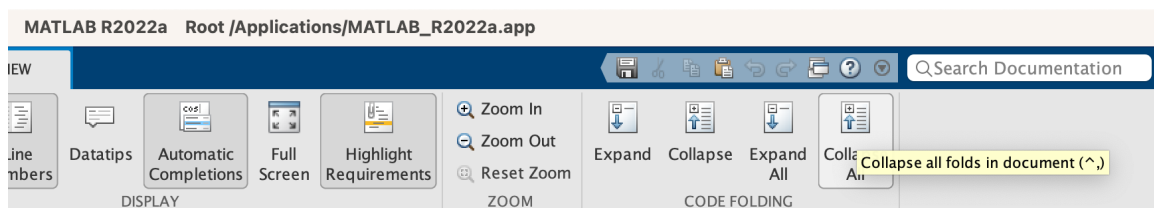
For convenience in the workshop, the test script is divided into numbered sections, with subsequent sections commented out. Uncomment the sections as you get to them.

Enable Code Folding



ENABLE CODE FOLDING

MATLAB Preferences > Editor/Debugger > Code Folding > Enable code folding.



COLLAPSE ALL FOLDS IN DOCUMENT

For readability, hide commented-out sections in the MATLAB Editor by selecting View > Code Folding > Collapse All.

Run Test Script

Run the test script and view the results in Figures 4.3 and 4.4 to verify that the algorithm is working properly.

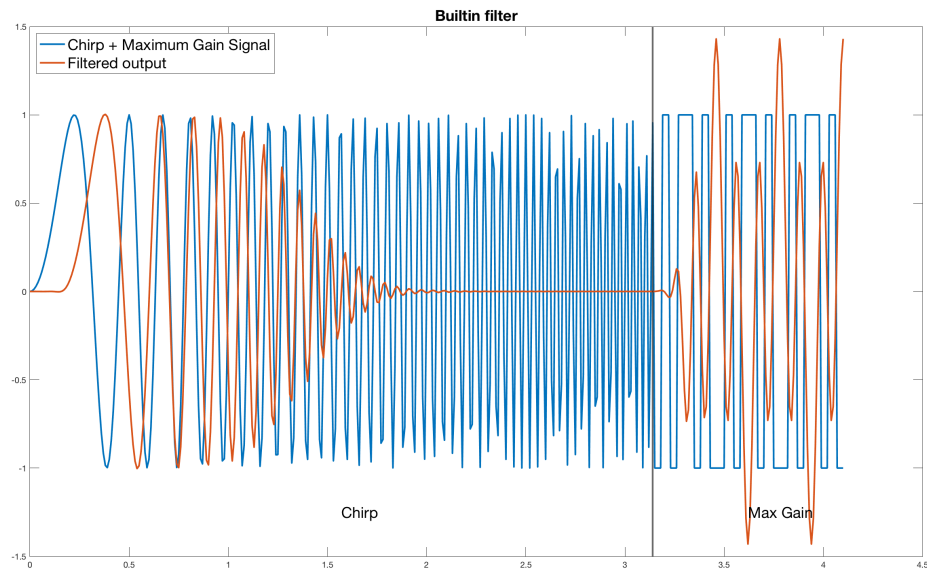


Figure 4.3: BUILTIN FILTER FOR COMPARISON
This is the expected output of the algorithm.

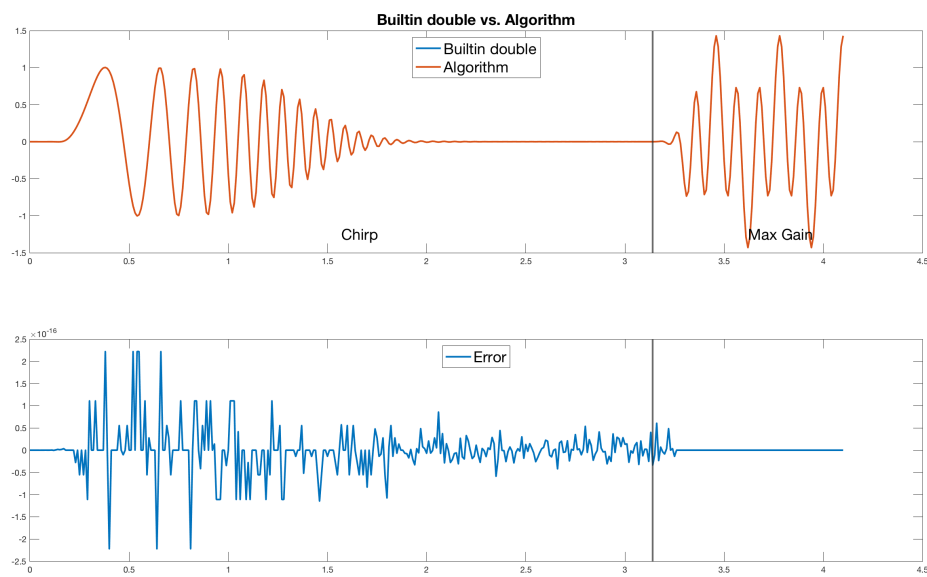


Figure 4.4: BUILTIN FILTER COMPARED TO THE ALGORITHM

Verify that the algorithm running in `double` data types matches the expected value from `builtin filter` command.

4.8 (2) Types Table and Template Parameter

4.8.1 Create a Types Table in Original Double Types

The next step is to create a structure of template parameters in the original types used in the algorithm. The default type in MATLAB is `double`, so that's a good place to start.

```
function T = fir_filter_double_types
    T.input = double([]);
    T.output = double([]);
    T.sum = double([]);
    T.coefficient = double([]);
    T.index = int32([]);
end
```

Since the default type is `double`, you could just do the following. However, it doesn't hurt to use `double([])`, so you can leave it in there to make it more obvious what type is being used.

```
function T = fir_filter_double_types
    T.input = [];
    T.output = [];
    T.sum = [];
    T.coefficient = [];
    T.index = int32([]);
end
```

Only the type of the template parameter is used, so it's handy to initialize them as empty. The only time you want to initialize them to something other than empty, is when the type is complex. In MATLAB whether a type is complex depends on its imaginary part being non-zero. So if you need to create a complex template parameter, do the following.

```
function T = fir_filter_double_types
    T.input = double(1i);
    T.output = double(1i);
    T.sum = double(1i);
    T.coefficient = double(1i);
    T.index = int32([]);
end
```

This example uses real data, so leave the template parameters empty.

4.8.2 Add Template Parameter

Add a template parameter, `T`, to the input of the function. We haven't done anything with template parameter `T` yet. We're doing this step to provide the plumbing for later.

```
function [y,z,p] = fir_filter(b,x,z,p,T)
    y = zeros(size(x));
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = 0;
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end
```

4.8.3 Update Test Script to use the Template Parameter

Update test script `test_fir_filter.m` to use the template parameter, and run again to verify it's working.

```
%% (2,4,5) Define types table
T = fir_filter_double_types;

%% (1,2,3) Run
[y,z,p] = fir_filter(b,x,z,p,T);
```

4.9 (3) Generate code

Our goal is to convert the algorithm for embedded code, so let's start by seeing what the floating-point C code would look like. Generating C code early in the conversion process identifies which functions are not supported by code generation. This is valuable even if the target is eventually FPGA or ASIC.

Update test script `test_fir_filter.m` to generate a MATLAB Executable (mex) file using the template parameter. Compiling the algorithm usually makes it run faster, and also helps debug limitations of generated code. For example, MATLAB allows variables to change size and type as a function runs, but C and HDL do not. It's good to debug `codegen` limitations on double types, before introducing any other potential issues with fixed-point types.

The `-args` option defines the types, sizes, and complexity of the inputs. The value of the input arguments does not matter at compile time, so it's convenient to create them as zeros of the type of the input: `zeros(n,1,'like',T.x)`. The template parameter T is used to define the type of the return argument, y , and can be used for defining other types, as we will see in later chapters of this workshop.

Cast the inputs like their counterparts in types-table T , add codegen commands to the test script to generate C code, and re-run.

```
%% (3) Build MATLAB Executable (mex) in target types
b = cast(b,'like',T.coefficient);
x = cast(x,'like',T.input);
z = zeros(size(b),'like',T.input);
p = cast(1,'like',T.index);
codegen fir_filter -args {b,x,z,p,T}

%% (1,2,3) Run
[y,z,p] = fir_filter_mex(b,x,z,p,T);

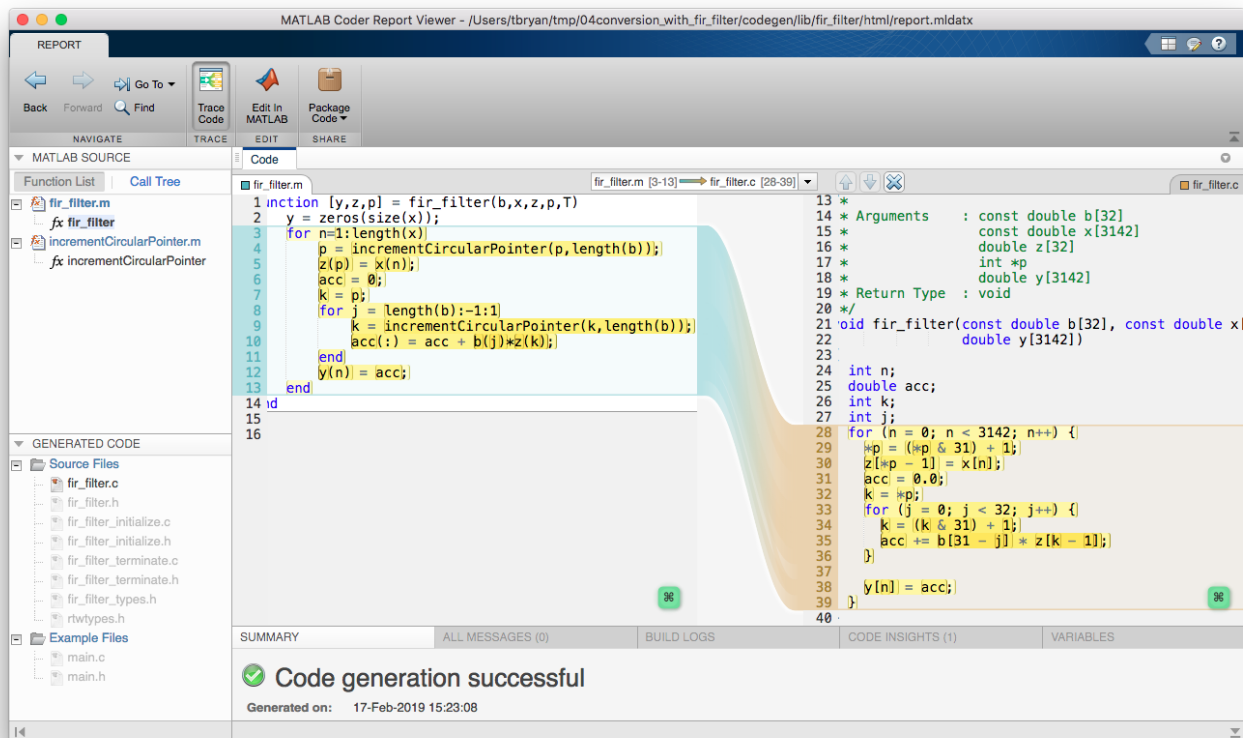
%% (3) C Code Generation
% Embedded devices never have the input over all time that is often
% simulated in MATLAB.
% Scalar input x elides the input/output for-loop.
% Vector input x is buffered, or frame input.

codegen fir_filter -args {b,x,z,p,T} -config:lib -launchreport
```

Since you don't declare types in MATLAB, you have to tell the compiler what type and size the input is by giving it a prototype argument, `-args {x}`. The `-config:lib` command indicates that we're generating C source code, and the `-launchreport` option will create a compilation report.

Run the test bench again to see the generated code.

Look at the generated C code and compare to the original MATLAB by selecting "Trace Code" in the MATLAB Coder Report Viewer. The "Trace Code" feature is only available with *Embedded Coder* product.



Loop counters j and k , were automatically declared integer type because you can't have floating-point indices in C, so you don't need to do anything extra to convert indices to fixed-point. We do math on circular buffer pointer p , so we needed to declare it as an integer type so MATLAB didn't think it should be double.

The inputs b , x , z , and output y are declared `double`, which is double-precision floating-point.

Variables x and z are used as both input and output, so they are passed by reference in the C code.

If x is passed as a scalar to the `codegen` command, then the outer loop will be omitted in the generated code.

The types-table parameter T is only used to declare data types, so it is omitted in the generated code. *Using types tables does not add any overhead to the generated code.*

```
void fir_filter(const double b[32], const double x[3142],
               double z[32], int *p,
               double y[3142])
{
    int n;
    double acc;
    int k;
    int j;
    for (n = 0; n < 3142; n++) {
        *p = (*p & 31) + 1;
        z[*p - 1] = x[n];
        acc = 0.0;
        k = *p;
        for (j = 0; j < 32; j++) {
            k = (k & 31) + 1;
            acc += b[31 - j] * z[k - 1];
        }

        y[n] = acc;
    }
}
```

4.10 (4) Single-Precision Floating-Point Types

4.10.1 Create a single-precision floating-point template parameter table

Even though the goal is to produce fixed-point code, create a single-precision floating-point (32-bit float) template parameter table. This will help expose potential code generation issues with changing types, and we can look at the generated code to see if all types (other than indices) have been successfully converted from double to single.

```
function T = fir_filter_single_types
    T.input = single([]);
    T.output = single([]);
    T.sum = single([]);
    T.coefficient = single([]);
    T.index = int32([]);
end
```

4.10.2 Change test script to run with single types

Change the test script to run with single types, and re-run.

```
%% (2,4,5) Define types table
T = fir_filter_<u>single</u>-types;
```

You will get the following error that indicates a problem with changing types. Even though MATLAB can change types on the fly, C and HDL are strongly typed, and do not allow this.

```
??? This assignment writes a 'single' value into a 'double'
type. Code generation does not support changing types through
assignment. Check preceding assignments or input type
specifications for type mismatches.
```

```
Error in ==> fir_filter Line: 10 Column: 13
```

```
Code generation failed: View Error Report
```

```
Error using codegen
```

```
Error in test_fir_filter (line 24)
```

```
codegen fir_filter -args b,x,z,p,T
```

Click on “View Error Report” and hover over the expressions. You can see that *y* is initialized as a double, but MATLAB rules return single in a mixed double+single operation.

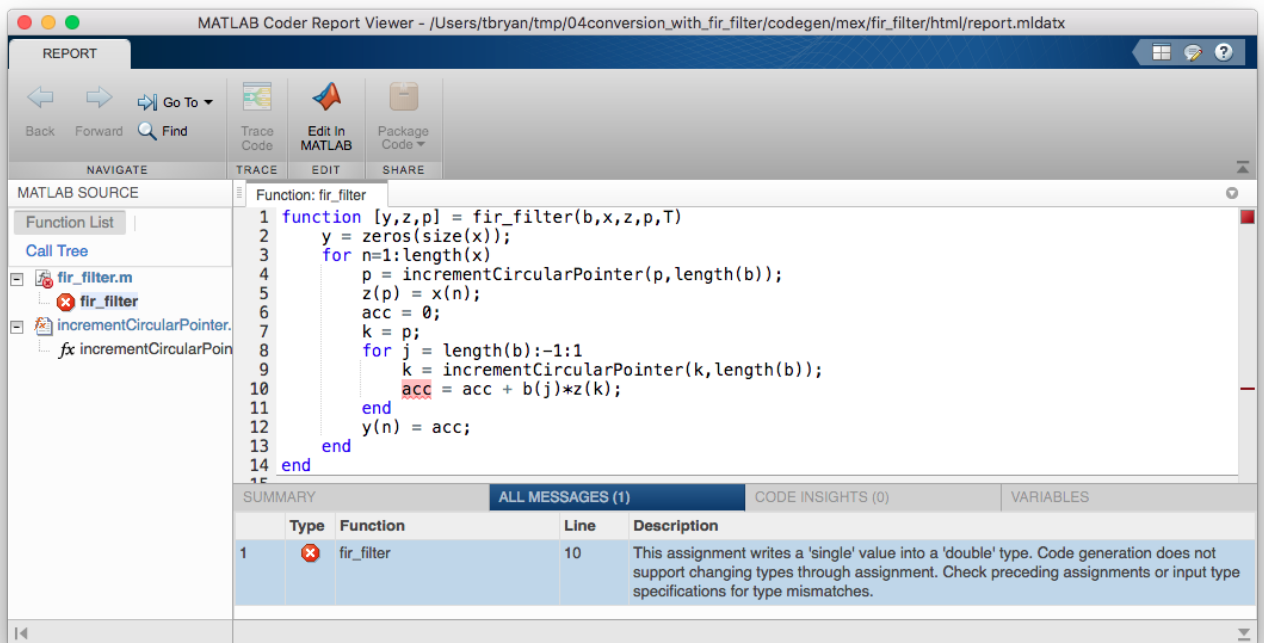


Figure 4.5: TYPE MISMATCH ERROR.

In MATLAB, adding double to single returns single. Variable `acc` was determined to be double, so it's changing `acc`'s type from double to single here. Fix by changing the line to `acc(:) = ...`

4.10.3 Assign in `acc(:)=...`

Fix the assignment error by using subscripted assignment on `acc`, which preserves its type, as follows.

```
function [y,z,p] = fir_filter(b,x,z,p,T)
    y = zeros(size(x));
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = 0;
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc(:) = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end
```

Run again.

4.10.4 Output and accumulator are still double

Re-run the test file and you can see that the output `y` and accumulator `acc` are double. Our goal was to fully specify the output type, so this needs to be fixed.

```
>> class(y)

ans =

double
```

```
void fir_filter(const float b[32], const float x[3142],
               float z[32], int *p,
               double y[3142])
{
    int n;
    double acc;
    int k;
    int j;
    for (n = 0; n < 3142; n++) {
        *p = (*p & 31) + 1;
        z[*p - 1] = x[n];
        acc = 0.0;
```

```

    k = *p;
    for (j = 0; j < 32; j++) {
        k = (k & 31) + 1;
        acc = (float)acc + b[31 - j] * z[k - 1];
    }

    y[n] = acc;
}
}

```

4.10.5 Specify output and accumulator type

To specify the output type, cast y to template variable type $T.output$, and cast accumulator acc to template variable type $T.sum$ as follows.

```

function [y,z,p] = fir_filter(b,x,z,p,T)
    y = zeros(size(x),'like',T.output);
    for n=1:length(x)
        p = incrementCircularPointer(p,length(b));
        z(p) = x(n);
        acc = cast(0,'like',T.sum);
        k = p;
        for j = length(b):-1:1
            k = incrementCircularPointer(k,length(b));
            acc(:) = acc + b(j)*z(k);
        end
        y(n) = acc;
    end
end

```

If our end-goal was all `single` types, then, in this example, we could have just cast y and acc to x 's type, as follows.

```

    y = zeros(size(x),'like',x);

    acc = cast(0,'like',x);

```

However, in fixed-point, we will need to allow y and acc 's types to grow integer bits, and so casting to x 's type could result in overflows.

4.11 (5) Instrument Code and Visualize Types with Fixed-Point

4.11.1 Create Fixed-Point Types Table

Create `fir_filter_dsp_types.m` to specify fixed-point data types specialized for C code for DSP. The `dataType` parameter enables you to change between fixed-point and scaled-double types, which are discussed in the next section. Let the `fi` object assume default types.

```
function T = fir_filter_dsp_types(dataType)
    if nargin < 1
        dataType = 'Fixed';
    end
    T.coefficient = fi([], 'DataType', dataType);
    T.input       = fi([], 'DataType', dataType);
    T.output      = fi([], 'DataType', dataType);
    T.sum         = fi([], 'DataType', dataType);
    T.index = int32([]);
end
```

4.11.2 Use Fixed-Point Types in Test Script

Change the test script so it uses the fixed-point types.

```
%% (2,4,5) Define types table
T = fir_filter_dsp_types;
```

4.11.3 Instrument with Scaled Double Types

Uncomment the following sections in the test script to build an instrumented mex file, and to visualize and propose data types.

“Scaled Double” types do all their computations in double-precision floating-point, and also know the fixed-point settings. This enables them to detect when a computation would have gone out of range of the fixed-point type, indicating overflow, and when a value would be too small to be represented in the fixed-point type, indicating underflow. Underflow occurs when a non-zero value becomes zero as the result of casting.

```
%% (5) Build instrumented mex
T2 = fir_filter_dsp_types('ScaledDouble');
b2 = cast(b,'like',T2.coefficient);
x2 = cast(x,'like',T2.input);
z2 = zeros(size(b),'like',T2.input);
p2 = cast(1,'like',T2.index);
buildInstrumentedMex fir_filter ...
    -args b2,x2,z2,p2,T2 -histogram -o instrumented_filter
```

The `-histogram` parameter in the above is necessary to log histograms of the instrumented values.

```
%% (5) Run instrumented mex to find underflows and overflows
[y2,z2,p2] = instrumented_filter(b2,x2,z2,p2,T2);
```

```
%% (5) Visualize data types
vis = VisualizeDatatypes('instrumented_filter',3, ...
    '', '-proposeFL');
```

Examine the data-type visualization plots in Figure 4.6 to see the dynamic range of the values that were instrumented while running the instrumented mex file. The instrumented values accumulate with each run until you either re-compile with `buildInstrumentedMex` or call `clearInstrumentationResults` on the instrumented MEX file.

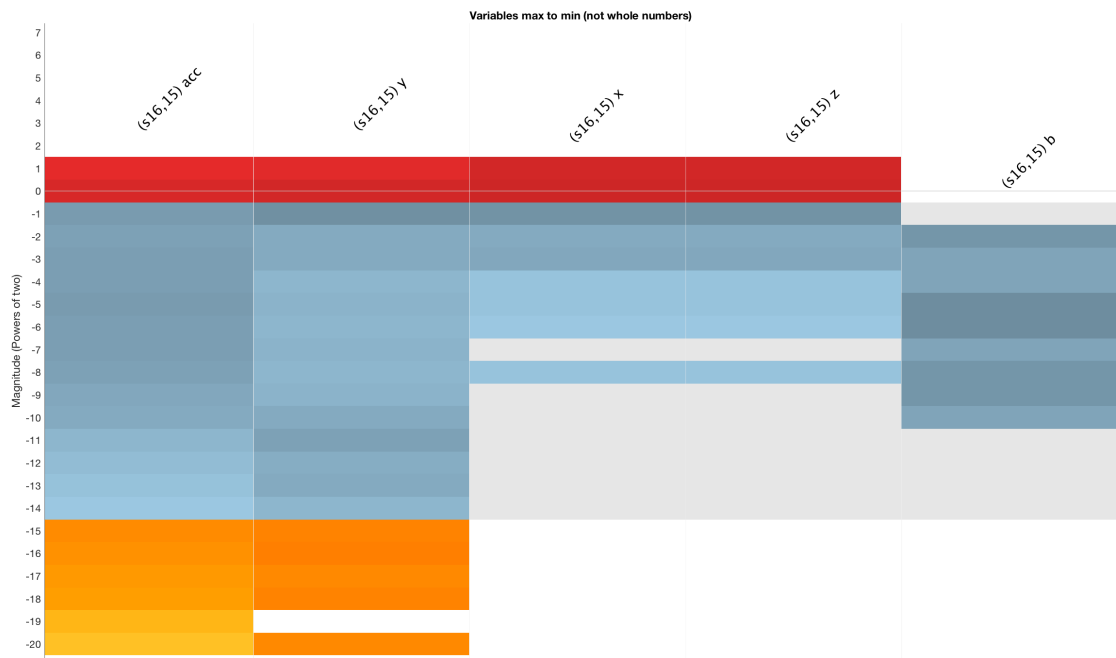


Figure 4.6: VISUALIZATION OF INSTRUMENTED VALUES

You can see that using default types results in **overflow** in the accumulator, output, input, and states, and that the coefficients have more head room. Also, the accumulator and output **underflow**. Logged values in range are displayed in **blue**. Darker blue means a higher concentration of values in that bin. Dynamic range of the variable are the band of gray.

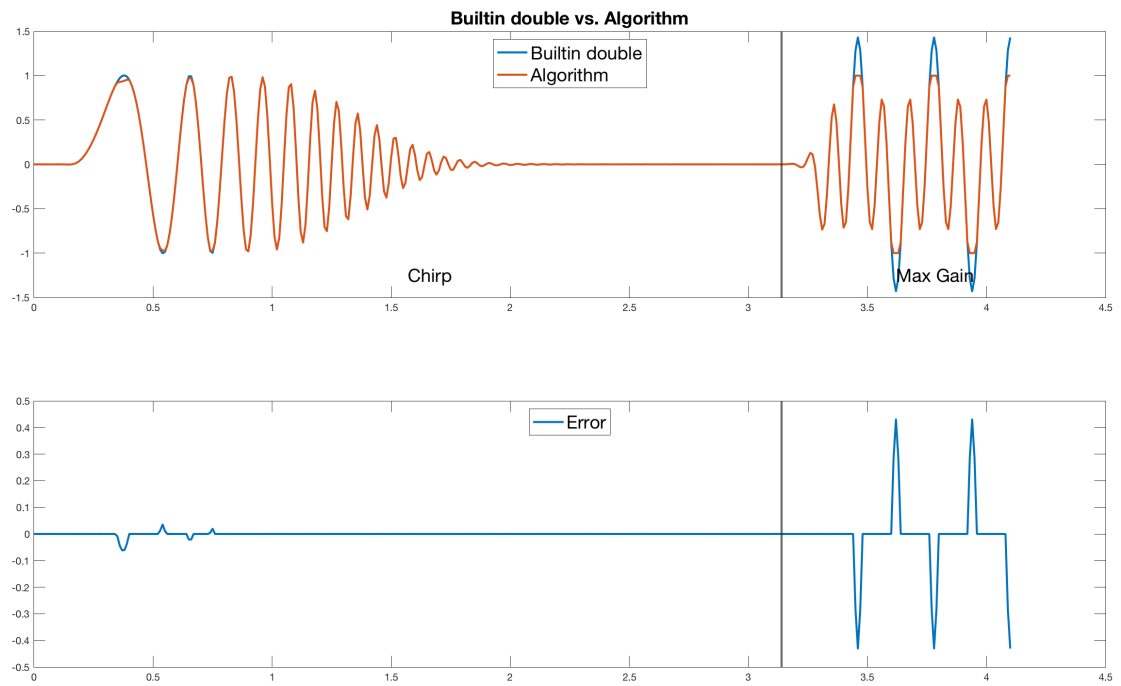


Figure 4.7: FIXED-POINT OUTPUT WITH OVERFLOWS
 You can see the large errors due to overflows in the Error plot.

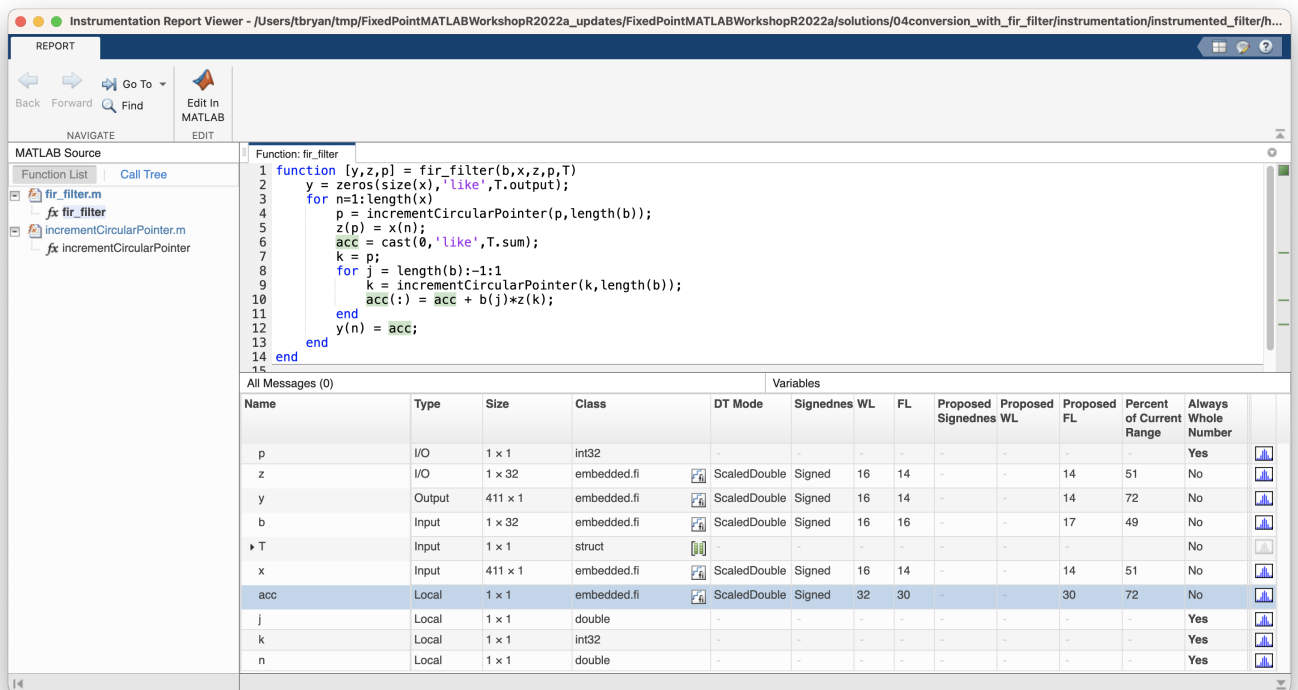


Figure 4.8: INSTRUMENTATION REPORT VIEWER

This report shows overflows highlighted in red. Click on one of the histogram icons to see a flattened version of the Datatype Visualization plot on one variable or expression.

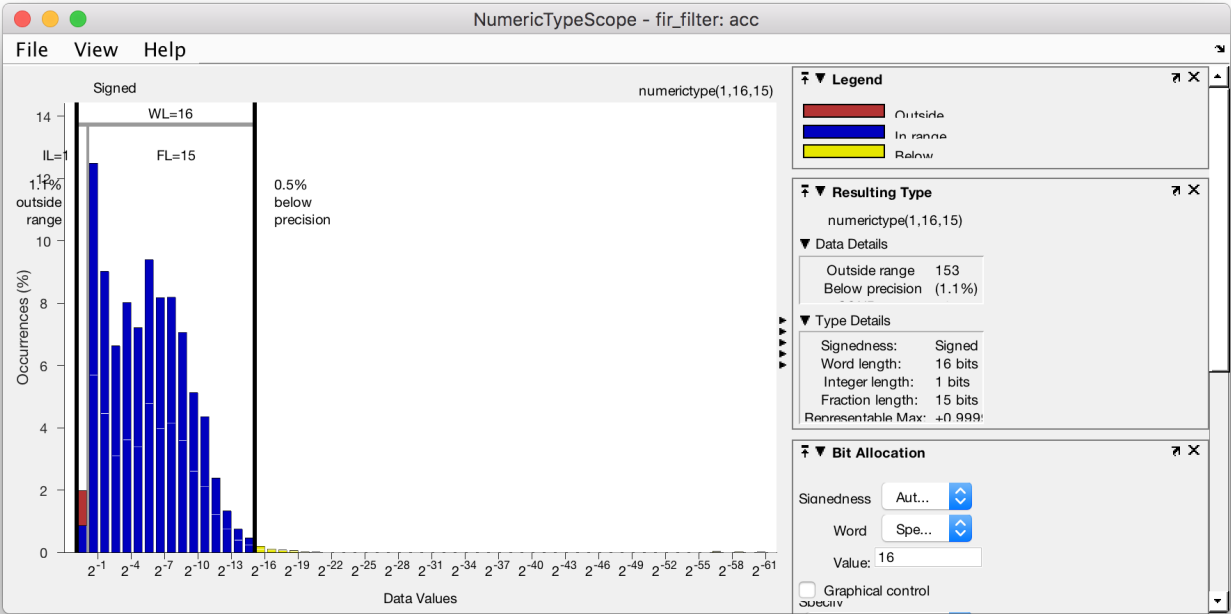


Figure 4.9: NUMERIC TYPE SCOPE
Histogram plot of variable `acc`

4.12 Choose Fixed-Point Types

Two of the main ways of choosing fixed-point types are the following. We will go through both of these methods.

1. Choose types based on theory
 - + You can choose types only knowing statistics of the data (such as the range of the input, and required precision of the output) and inner-workings of the algorithm (like the impulse- and frequency-response of a filter). Often you do know these things because the range and precision of the inputs are governed by things like the analog-to-digital converter (ADC) and the limitations of the target hardware, and the precision of the output is governed by the requirements of the system, such as dB of attenuation in the stop-band of a filter. Also, you can know the inner-workings of the algorithm, especially if it's a linear system, like an FIR filter, FFT, or CORDIC. For more information, see the following references: The theory of fixed-point quantization was discovered by Widrow in 1956 [27], and Widrow and Kollar wrote a comprehensive book called *Quantization Noise* in 2008 [28].
 - Δ You have to know the theory of the system, like the frequency response and impulse response. This is easy when it's a feed-forward linear system, like an FIR filter, but can be difficult for non-linear systems or systems with feedback that can become unstable with quantization.
2. Choose types based on logged values from simulation data
 - + You don't have to know anything about the system to choose types.
 - Δ The simulation data must cover the full range of possible input values and also excite the maximum and minimum values in the output. Changing to a different set of inputs and coefficients changes the scaling.

4.12.1 Choose types based on theory

In the following, assume that the input, output, and coefficients are 16-bit based on DSP hardware requirements. Adjust these values to suit your particular implementation.

The steps are in the following MATLAB Live Script

Choose_fir_filter_types_based_on_theory.mlx

Compute type of input

Assume you are given a requirement that the input x is in the following range

$$-1 \leq x \leq 1.$$

If you know that the input is strictly less than +1 (which it often is), then you can save an additional bit of precision and use “fractional” types. A fractional type is one where the fraction length is one less than the word length, for example `fi([],1,16,15)` (Texas Instruments calls this q15 format). In this example, we will not make this optimization.

Given a range of values, you can let the `fi` object auto-scale:

```
>> x = fi([-1,1],1,16)

x =
    -1         1

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 16
    FractionLength: 14
```

You can query the range of the data type, and see that $-2 \leq x < 2$, which is more than we need, but it allows +1 to be represented exactly.

```
>> range(x)
ans =
    -2.0000         1.9999

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 16
    FractionLength: 14
```

If we choose a purely “fractional” type (one less fraction bit than the word length), then you can see that the range just misses $+1$. The range is exactly $-1 \leq x \leq 1 - 2^{-15}$. If we condition the input to be strictly less than $+1$, then we can use this type. To keep this example simple, we will not use this optimization.

```
>> format long
>> fi([-1,1],1,16,15)
ans =
    -1.0000000000000000    0.999969482421875

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 16
      FractionLength: 15
```

Compute type of output

The maximum gain of an FIR for any input is the sum of the absolute values of the filter coefficients, which is also known as the 1-norm of the vector of coefficients.

For these particular coefficients, the maximum gain is

```
>> b0 = fir_filter_inputs;

>> max_gain = norm(b0,1) % == sum(abs(b))

max_gain =
    1.4305
```

Compute the output type by letting the `fi` object auto-scale on the maximum gain by specifying 16-bit word length and not specifying the fraction length. The gain of the filter will be applied to the input values, so the upper bound of the output will be the gain multiplied by the maximum input value. In this example, x is between -1 and 1 , so $\max(|x|) \leq 1$, but we do the multiplication anyway so this algorithm is generalized for any input.

```
>> output_upper_bound = max_gain * max(abs(x(:)))
>> y = fi(output_upper_bound, 1, 16)

y =
    1.4305

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Signed
           WordLength: 16
      FractionLength: 14
```

The range of the output is $-2 \leq y \leq 2 - 2^{-14}$, which covers the maximum gain of 1.4305.

```
>> range(y)
ans =
    -2.0000     1.9999

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Signed
           WordLength: 16
      FractionLength: 14
```

Compute type of accumulator

Builtin integer types in C and many DSPs are 8, 16, 32 bits, and sometimes 64 bits. Multiplying 16-bit input by 16-bit coefficient will produce a 32-bit product. It would be convenient to also keep the accumulator in 32-bits. Compute the output type by letting the `fi` object auto-scale on the maximum gain by specifying 32-bit word length not specifying the fraction length.

```
>> acc = fi(max_gain, 1, 32)

acc =
    1.4305

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 30
```

The range of the accumulator is $-2 \leq \text{acc} \leq 2 - 2^{-30}$, which covers the maximum gain of 1.4305.

```
>> format long
>> range(acc)
ans =
-2.0000000000000000    1.999999999068677

    DataTypeMode: Fixed-point: binary point scaling
    Signedness: Signed
    WordLength: 32
    FractionLength: 30
```

Compute type of coefficients

The `fi` object will auto-scale and pick the maximum precision (i.e. maximum fractional bits) if you don't specify fraction length.

```
>> b = fi(b0, 1, 16)

b =
  Columns 1 through 5
    -0.0006    -0.0018    -0.0026    -0.0016     0.0024
  ...
  Columns 31 through 32
    -0.0018    -0.0006

      DataTypeMode: Fixed-point: binary point scaling
      Signedness: Signed
      WordLength: 16
      FractionLength: 17
```

Fraction length 17 longer than word length 16 is okay because it's not a physical thing — it's just the scaling of the integer representation:

$$\text{"Real World Value"} = \text{"Stored Integer"} \times 2^{-17}.$$

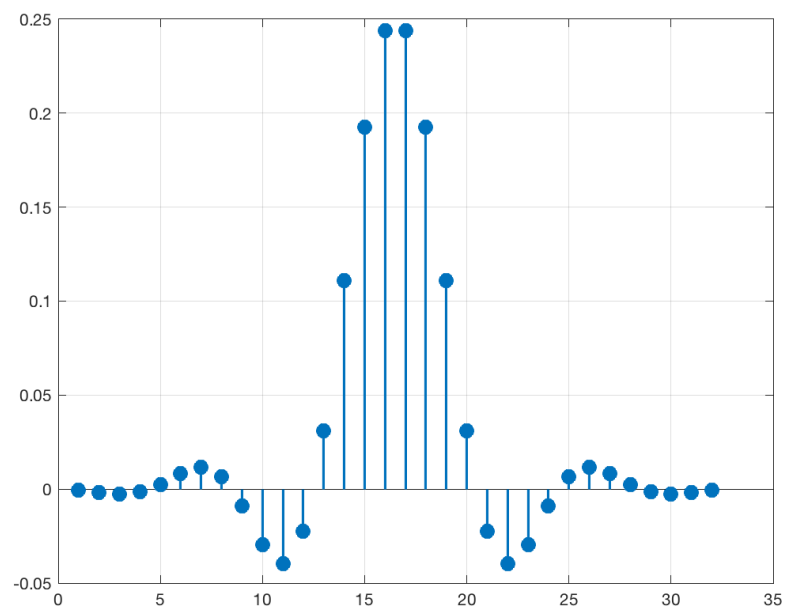


Figure 4.10: FIR FILTER COEFFICIENTS
The coefficients for this filter are all less than 0.25 in magnitude (`stem(b)`)

Update Types Table

Using the types computed in the previous sections, update the types table with the following settings.

```
function T = fir_filter_dsp_types(dataType)
    if nargin < 1
        dataType = 'Fixed';
    end
    T.coefficient = fi([],1,16,17,'DataType',dataType);
    T.input       = fi([],1,16,14,'DataType',dataType);
    T.output      = fi([],1,16,14,'DataType',dataType);
    T.sum         = fi([],1,32,30,'DataType',dataType);
    T.index = int32([]);
end
```

Re-run with new types

At this point, the converted algorithm hasn't changed. We have only modified the types in the types table.

Re-run the test script with the new types. You can see from the following plots that there are no overflows and small difference between builtin floating-point filter and the fixed-point filter.

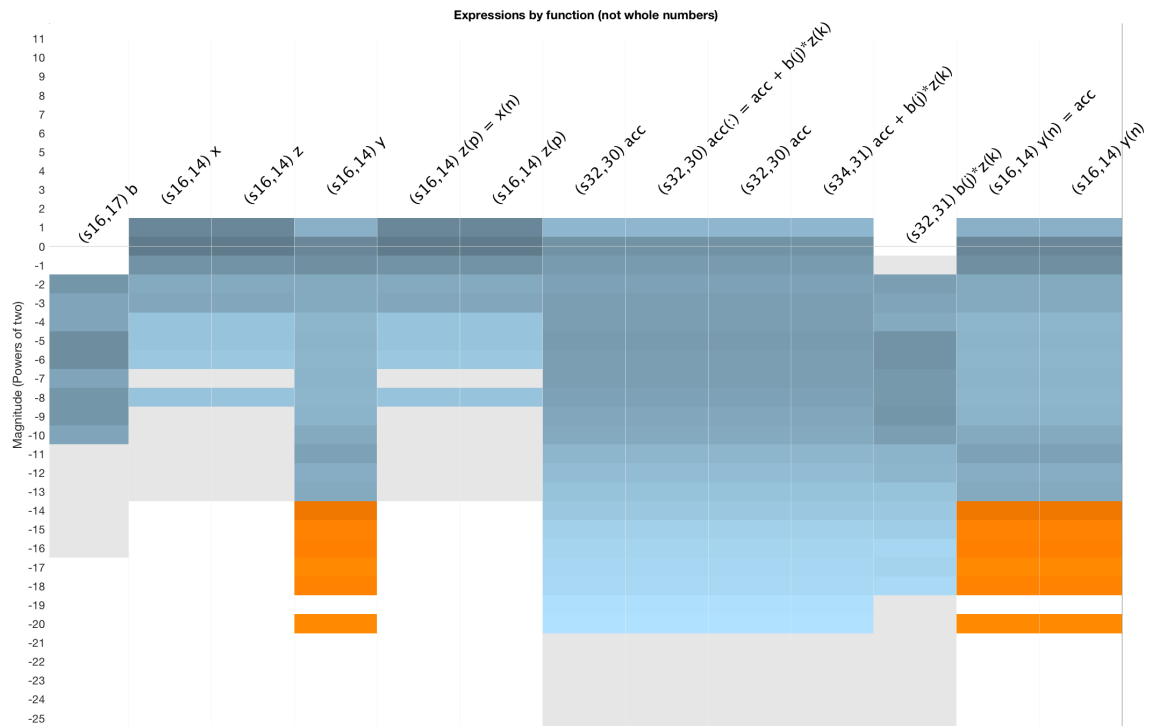


Figure 4.11: DATATYPE VISUALIZER

Inspect the datatype visualization plots to see that expressions attain their maximum range without overflowing, and the only underflows are when the accumulator is cast down to the output type.

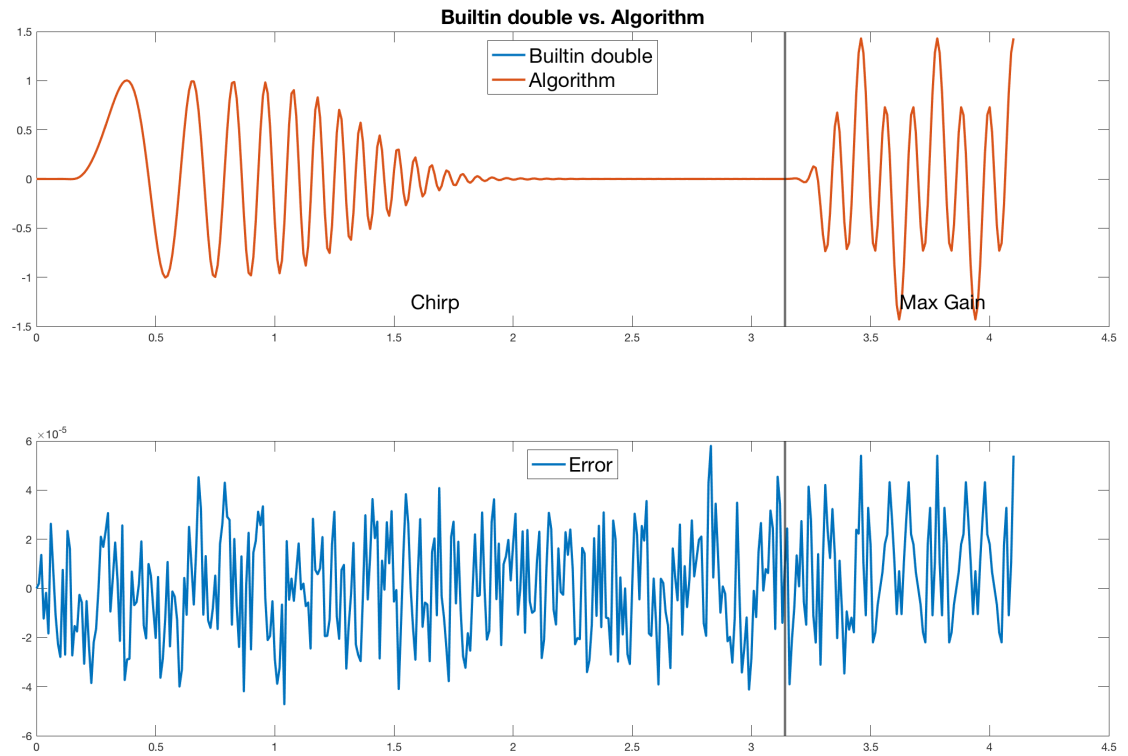


Figure 4.12: BUILTIN FILTER COMPARED TO FIXED-POINT FILTER

The error is on the order of 6×10^{-5}

The screenshot shows the Instrumentation Report Viewer interface. The top panel displays the MATLAB source code for a function named `fir_filter`. The code is as follows:

```

1 function [y,z,p] = fir_filter(b,x,z,p,T)
2   y = zeros(size(x),'like',T.output);
3   for n=1:length(x)
4     p = incrementCircularPointer(p,length(b));
5     z(p) = x(n);
6     acc = cast(0,'like',T.sum);
7     k = p;
8     for j = length(b):-1:1
9       k = incrementCircularPointer(k,length(b));
10      acc(:) = acc + b(j)*z(k);
11    end
12    y(n) = acc;
13  end
14 end
15

```

The bottom panel displays a table of variables with their types, sizes, and simulation ranges. The table is titled "VARIABLES" and has the following columns: Name, Type, Size, Class, DT Mode, Signedness, WL, FL, Proposed Signedness, Proposed WL, Proposed FL, Percent of Current Range, Always Whole Number, Sim Min, and Sim Max.

Name	Type	Size	Class	DT Mode	Signedness	WL	FL	Proposed Signedness	Proposed WL	Proposed FL	Percent of Current Range	Always Whole Number	Sim Min	Sim Max
z	I/O	1 x 32	embedded.fi	ScaledDouble	Signed	16	14	-	-	14	51	No	-1	1
p	I/O	1 x 1	int32	-	-	-	-	-	-	-	-	Yes	1	32
y	Output	411 x 1	embedded.fi	ScaledDouble	Signed	16	14	-	-	14	72	No	-1.4304879666746044	1.4304879666746044
b	Input	1 x 32	embedded.fi	ScaledDouble	Signed	16	17	-	-	17	98	No	-0.03981645226642165	0.2437302038864849
x	Input	411 x 1	embedded.fi	ScaledDouble	Signed	16	14	-	-	14	51	No	-1	1
T	Input	1 x 1	struct	-	-	-	-	-	-	-	-	No	-	-
n	Local	1 x 1	double	-	-	-	-	-	-	-	-	Yes	1	411
acc	Local	1 x 1	embedded.fi	ScaledDouble	Signed	32	30	-	-	30	72	No	-1.4304879666746044	1.4304879666746044
k	Local	1 x 1	int32	-	-	-	-	-	-	-	-	Yes	1	32
j	Local	1 x 1	double	-	-	-	-	-	-	-	-	Yes	1	32

Figure 4.13: INSTRUMENTATION REPORT VIEWER

The report shows no overflows and the proposed types are the same as the actual types.

4.12.2 Choose types based on logged values from simulation data

To choose types based on logged values from simulation data:

1. Turn on logging with `buildInstrumentedMex` in the test script.
2. Run simulated data through the generated MATLAB executable (MEX) files.
3. Inspect the logged values from the plots and report generated by `VisualizeDatatypes` in the test script.
4. Choose types based on the proposed types in the *Instrumentation Report Viewer*.

Types proposed from simulation data only reflect the quality of test inputs. Unless you can guarantee that the inputs represent all possible cases, then they are like the Pirate’s Code — “they’re more like ‘guidelines’ than actual rules.”

For feed-forward linear systems, like an FIR filter, the limits are easily tested, so choosing types based on logged values from simulation data is a viable approach.

The steps are in the following MATLAB Live Script

`Choose_fir_filter_types_based_on_simulation.mlx`

Full Range Test Inputs

Good test inputs for any algorithm should test both that the algorithm produces expected results, and also push the output to the limits.

Assuming that the input is in the range $-1 \leq x \leq 1$, you can force an FIR filter to achieve its maximum gain by using an input of +1 and -1 that matches the sign of its coefficients.

You can see the maximum gain 1.4305 achieved in the right side of the plot in Figure 4.14.

```
b = fir_filter_inputs;
x = sign(fliplr(b));
y = filter(b,1,x);
stem([b;x;y]','filled');
legend('Coefficients','Max Gain Input','Filtered Output')
```

```
>> max_gain = norm(b,1)

max_gain =
    1.4305
```

The “flip left-right” (`fliplr`) in the code above is because the n th input $x(n)$ is multiplied by the first coefficient $b(1)$, the previous input $x(n-1)$ by the second coefficient $b(2)$, and so forth, as seen in the difference equation.

$$y(n) = b(1)x(n) + b(2)x(n-1) + b(3)x(n-2) \cdots + b(\text{end})x(n - \text{length}(b) + 1).$$

So the MATLAB code for the output $y(n)$ at time n is

```
y(n) = sum(b .* x(n:-1:n-length(b)+1))
```

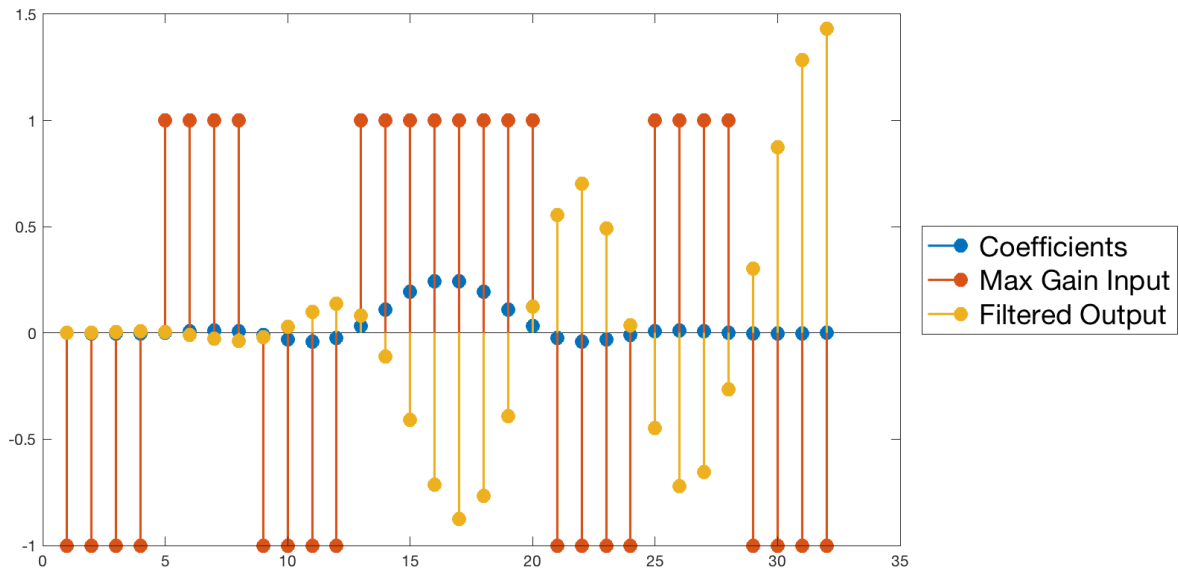


Figure 4.14: MAXIMUM GAIN OUTPUT

Input of -1 and $+1$ matching coefficient signs achieves maximum gain of `norm(b,1)` on the right side of the plot.)

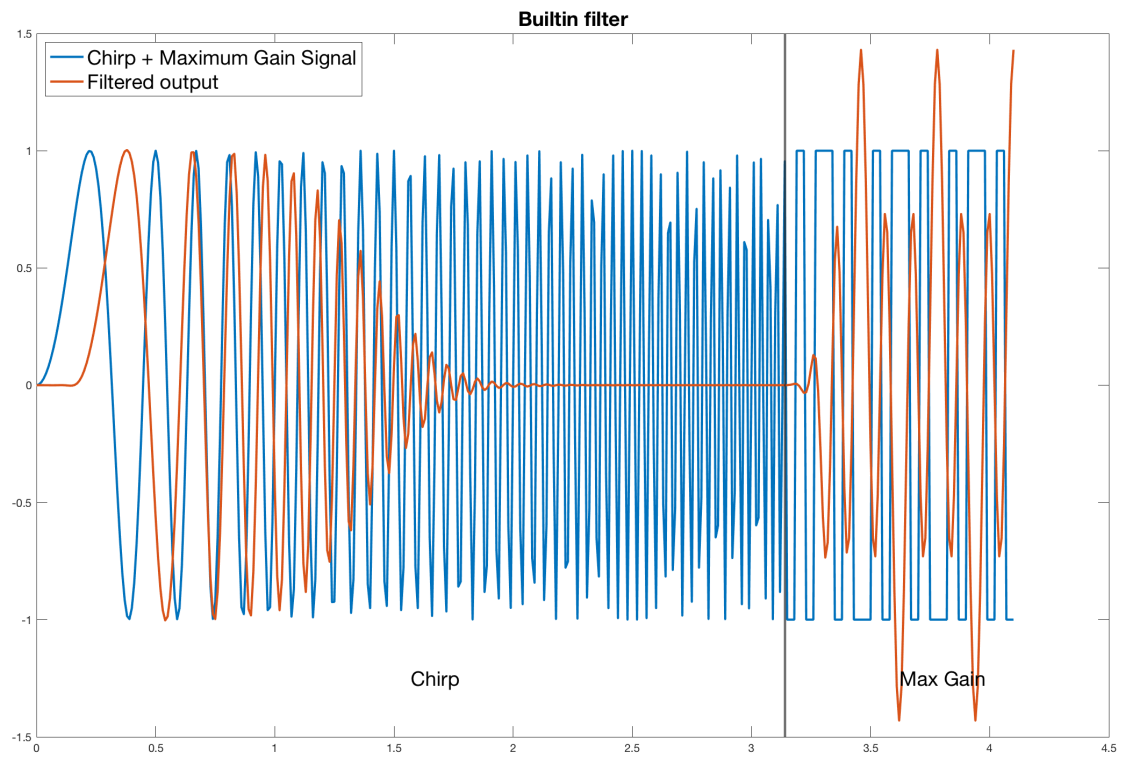


Figure 4.15: TEST INPUTS

The test file uses inputs of both a chirp signal to verify the low-pass action of the filter, and a maximum-gain input to test the full range of outputs

Proposed types from Instrumentation Report

Turn on “Propose Fraction Length” in the visualization report by using the `-proposeFL` option.

```
%% (5) Visualize data types
vis = VisualizeDatatypes('instrumented_filter',3, ...
    ' ', '-proposeFL');
```

The screenshot displays the Instrumentation Report Viewer interface. The top panel shows the MATLAB source code for a function named `fir_filter`. The code implements a fixed-point FIR filter with a circular pointer. The bottom panel shows a table of data types for the function's variables.

Function: fir_filter

```

1 function [y,z,p] = fir_filter(b,x,z,p,T)
2     y = zeros(1,length(x));
3     for n=1:length(x)
4         p = incrementCircularPointer(p,length(b));
5         z(p) = x(n);
6         acc = cast(0,'like',T,'sum');
7         k = p;
8         for j = length(b):-1:1
9             k = incrementCircularPointer(k,length(b));
10            acc(:) = acc + b(j)*z(k);
11        end
12        y(n) = acc;
13    end
14 end
15

```

Data Types Table:

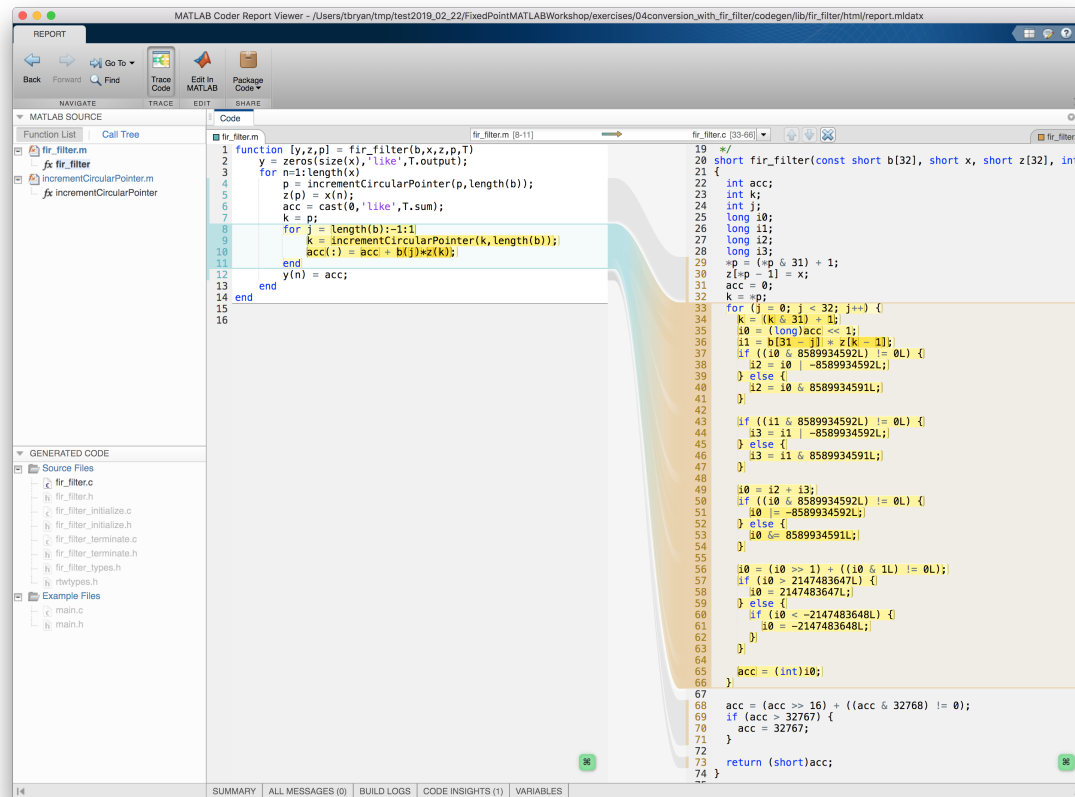
Name	Type	Size	Class	DT Mode	Variables				Always Whole Number		
					Signedness	WL	FL	Proposed Signedness		Proposed WL	Proposed FL
p	I/O	1 x 1	int32	-	-	-	-	-	-	Yes	
z	I/O	1 x 32	embedded.fi	ScaledDouble	Signed	16	14	-	14	51	No
y	Output	411 x 1	embedded.fi	ScaledDouble	Signed	16	14	-	14	72	No
b	Input	1 x 32	embedded.fi	ScaledDouble	Signed	16	16	-	17	49	No
* T	Input	1 x 1	struct	-	-	-	-	-	-	-	No
x	Input	411 x 1	embedded.fi	ScaledDouble	Signed	16	14	-	14	51	No
acc	Local	1 x 1	embedded.fi	ScaledDouble	Signed	32	30	-	30	72	No
j	Local	1 x 1	double	-	-	-	-	-	-	-	Yes
k	Local	1 x 1	int32	-	-	-	-	-	-	-	Yes
n	Local	1 x 1	double	-	-	-	-	-	-	-	Yes

Figure 4.16: INSTRUMENTATION REPORT VIEWER

Look at the proposed data types in the Instrumentation Report Viewer and update the fixed-point types table based on the proposed types.

4.13 More efficient fimath

Look at the generated C code and observe that it's got a lot of rounding and saturation code that you don't want.



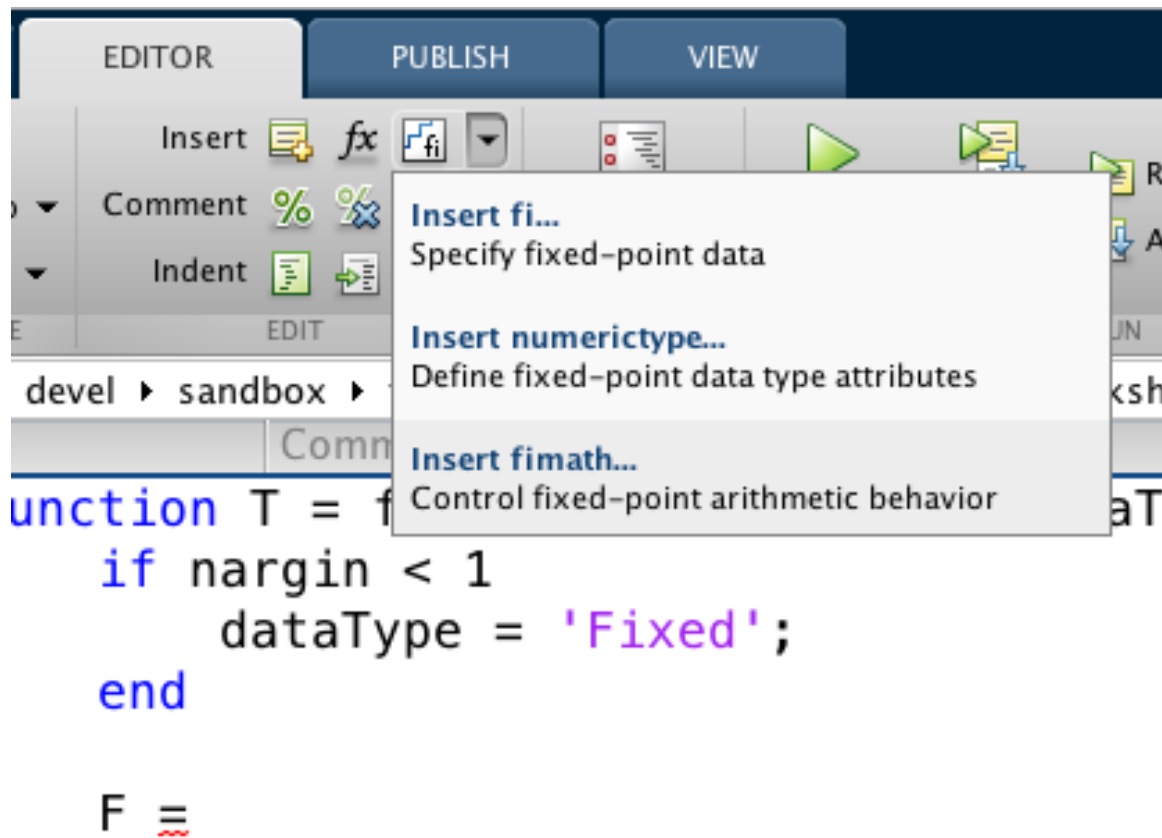
GENERATED C FROM FIRST TYPES

To fix that, add `fimath` settings that are more efficient for embedded C.

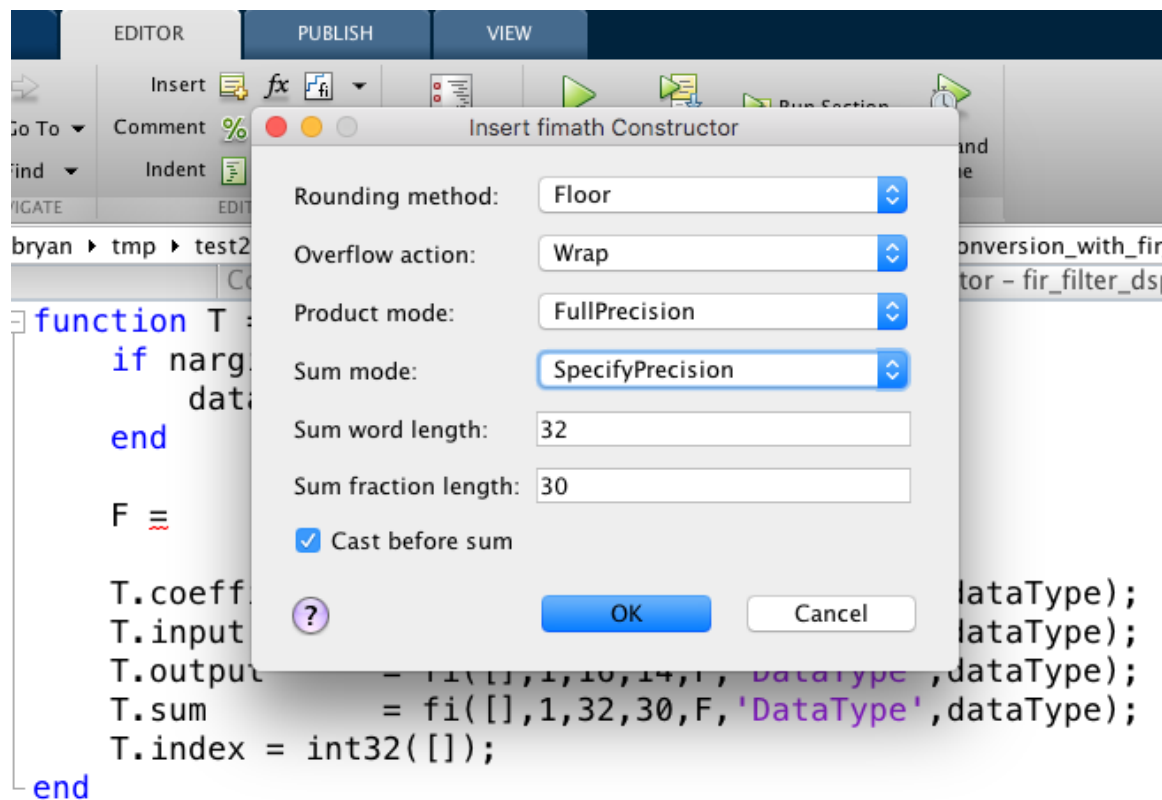
Edit file `fir_filter_dsp_types.m` and type

```
F =
```

Then from the MATLAB Editor menu, select Insert `fimath`, and change the settings to floor rounding, wrap overflow, and sum mode that matches the sum type in `T.sum`.



INSERT `FIMATH`



FIMATH TYPES

DSP friendly math is floor, wrap, and specify the sum type to match `T.sum`.

Floor rounding and wrap overflow naturally produce more efficient code because bits are just dropped for those modes. However, bias is added with floor rounding and you should be careful that you have chosen types that won't overflow because wrap overflow can be devastating to an algorithm.

Biased rounding is most pronounced in recursive algorithms. For example, when the Vancouver Stock Exchange used floor rounding in a recursive algorithm to compute its index in 1982, and the index dropped 480 points solely due to this rounding choice [17].

Even though rounding to floor is biased in an FIR filter, it is less consequential because it is not a recursive algorithm so the errors don't feed back and build up over time. Compare the outputs of the filter in Figure 4.17 with nearest rounding to Figure 4.18 with floor rounding.

The 32-bit sum word length is to match the natural integer type for the sum.

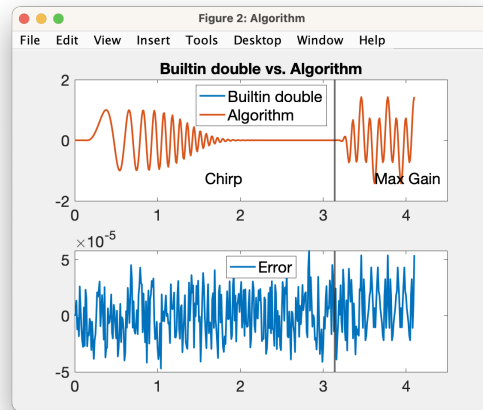


Figure 4.17: FIXED-POINT WITH NEAREST ROUNDING
The errors are unbiased.

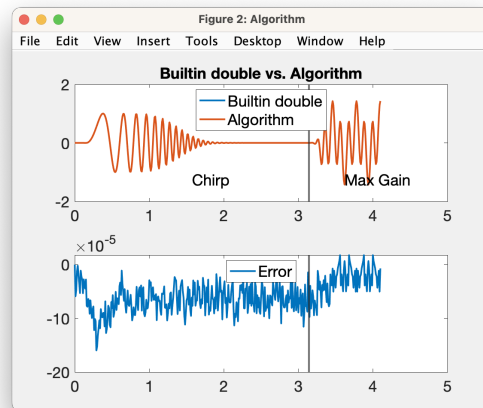


Figure 4.18: FIXED-POINT WITH FLOOR ROUNDING
The errors are biased towards floor, but they do not grow over time like they would with biased rounding in a recursive filter.

When you select “Okay”, the `fimath` constructor will be filled out in the file. You can also type these settings manually, but the Insert `fimath` feature of the editor is handy.

```
function T = fir_filter_dsp_types(dataType)
    if nargin < 1
        dataType = 'Fixed';
    end

    F = fimath('RoundingMethod', 'Floor', ...
        'OverflowAction', 'Wrap', ...
        'ProductMode', 'FullPrecision', ...
        'SumMode', 'SpecifyPrecision', ...
        'SumWordLength', 32, ...
        'SumFractionLength', 30, ...
        'CastBeforeSum', true);
    % Don't attach fimath to the coefficient type.
    % They are constant, so let the round to nearest
    % for more accuracy.
    T.coefficient = fi([],1,16,17,'DataType',dataType);
    T.input        = fi([],1,16,14,E,'DataType',dataType);
    T.output       = fi([],1,16,14,E,'DataType',dataType);
    T.sum          = fi([],1,32,30,E,'DataType',dataType);
    T.index = int32([]);
end
```

Re-run the test script again and look at the generated C code in the MATLAB Coder report [4.19](#). You can see that the additional rounding and overflow code is gone, but there’s an additional right-shift when summing into the accumulator because the product and sum aren’t aligned.

```
short fir_filter(const short b[32], short x, short z[32], int *p)
{
    int acc;
    int k;
    int j;
    *p = (*p & 31) + 1;
    z[*p - 1] = x;
    acc = 0;
    k = *p;
    for (j = 0; j < 32; j++) {
        k = (k & 31) + 1;
        acc += (b[31 - j] * z[k - 1]) >> 1;
    }

    return (short)(acc >> 16);
}
```

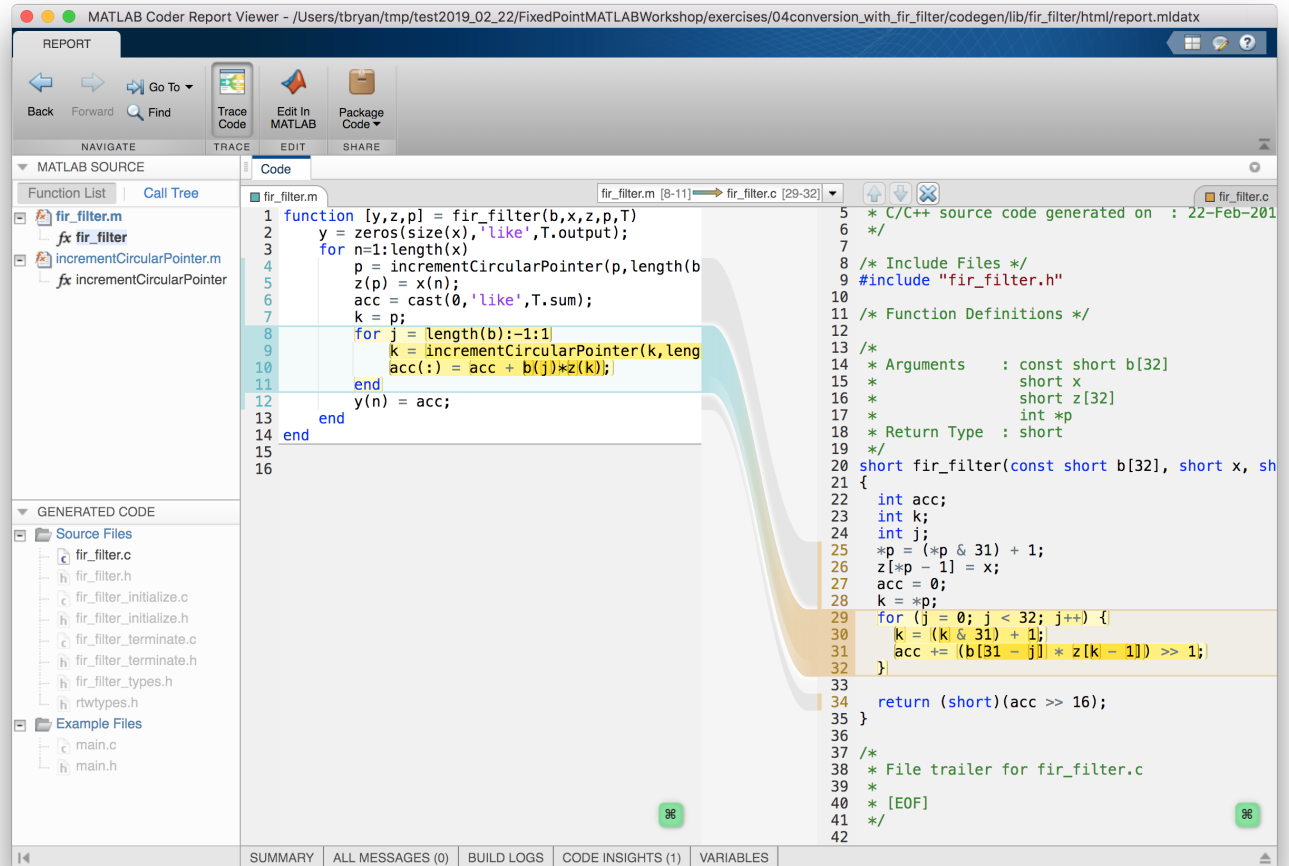


Figure 4.19: C CODE WITH DSP FRIENDLY TYPES
It's much better, but still has an extra shift in the accumulator.

4.13.1 Align product and sum

Change the coefficient type to have 16 fractional bits so that the multiplication of the input by the coefficient will naturally align with the sum type $(16, 16) \times (16, 14) \rightarrow (32, 30)$

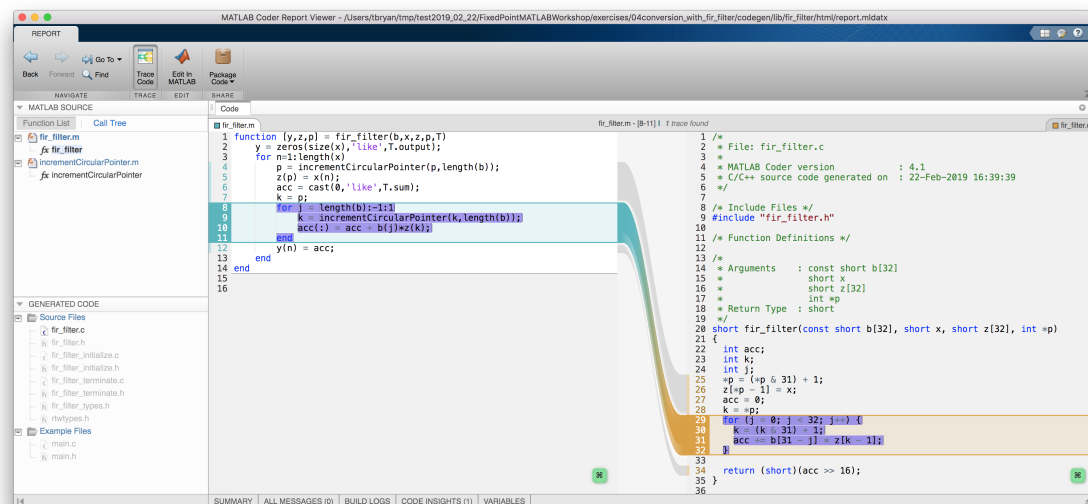
```

T.coefficient = fi([],1,16,16,F,'DataType',dataType);
T.input       = fi([],1,16,14,F,'DataType',dataType);
T.sum         = fi([],1,32,30,F,'DataType',dataType);

end

```

Re-run the test script again and look at the generated code



FINAL C CODE

The generated code is on par with code hand-written by an expert. The math is natural, with no loss of precision until the final cast to the output.

Notice that the math is natural, with no loss of precision until the final cast to the output.

```
short fir_filter(const short b[32], short x, short z[32], int *p)
{
    int acc;
    int k;
    int j;
    *p = (*p & 31) + 1;
    z[*p - 1] = x;
    acc = 0;
    k = *p;
    for (j = 0; j < 32; j++) {
        k = (k & 31) + 1;
        acc += b[31 - j] * z[k - 1];
    }

    return (short)(acc >> 16);
}
```

4.14 Verify Quantized Filter Design

Use the *Filter Designer* app to verify that the quantized filter still meets the specification of the original floating-point filter.

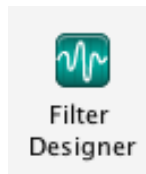
In addition to designing filters, the *Filter Designer* app lets you quickly see the effects of choosing different fixed-point types.

You can also use the *Filter Designer* app from MATLAB command-line

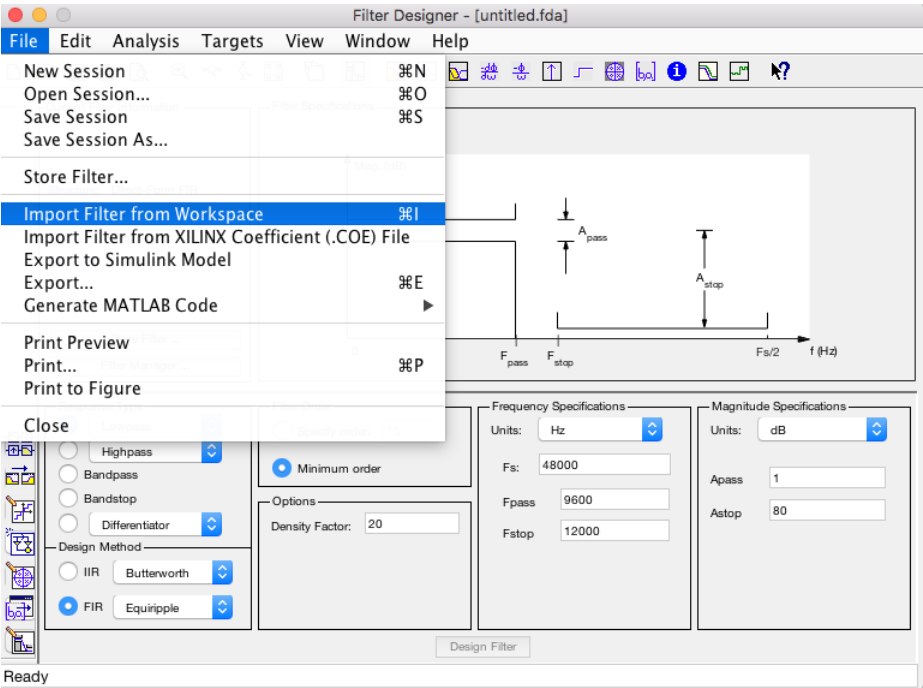
```
>> filterDesigner
```

Or launch from

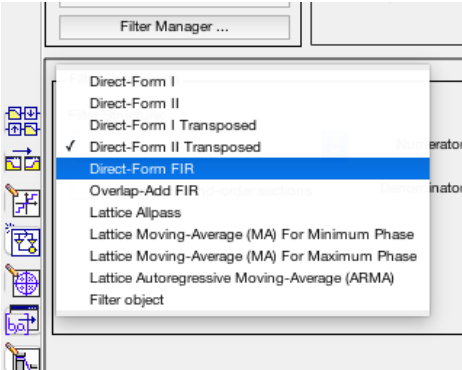
Apps > Signal Processing and Communications > Filter Designer



Follow the steps in the following figures to load the filter and set fixed-point types to match the types in `fir_filter_dsp_types`.



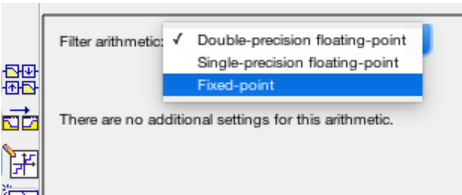
(a) Import filter



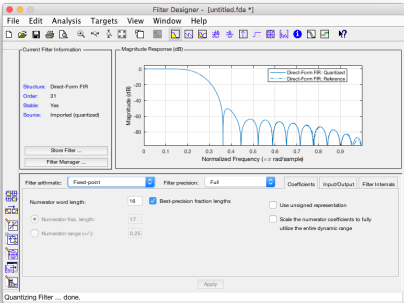
(b) FIR



(c) Coefficients b0



(d) Fixed point



(e) Coefficient type

Figure 4.20: FILTER DESIGNER
Import filter coefficients from workspace and select fixed-point.

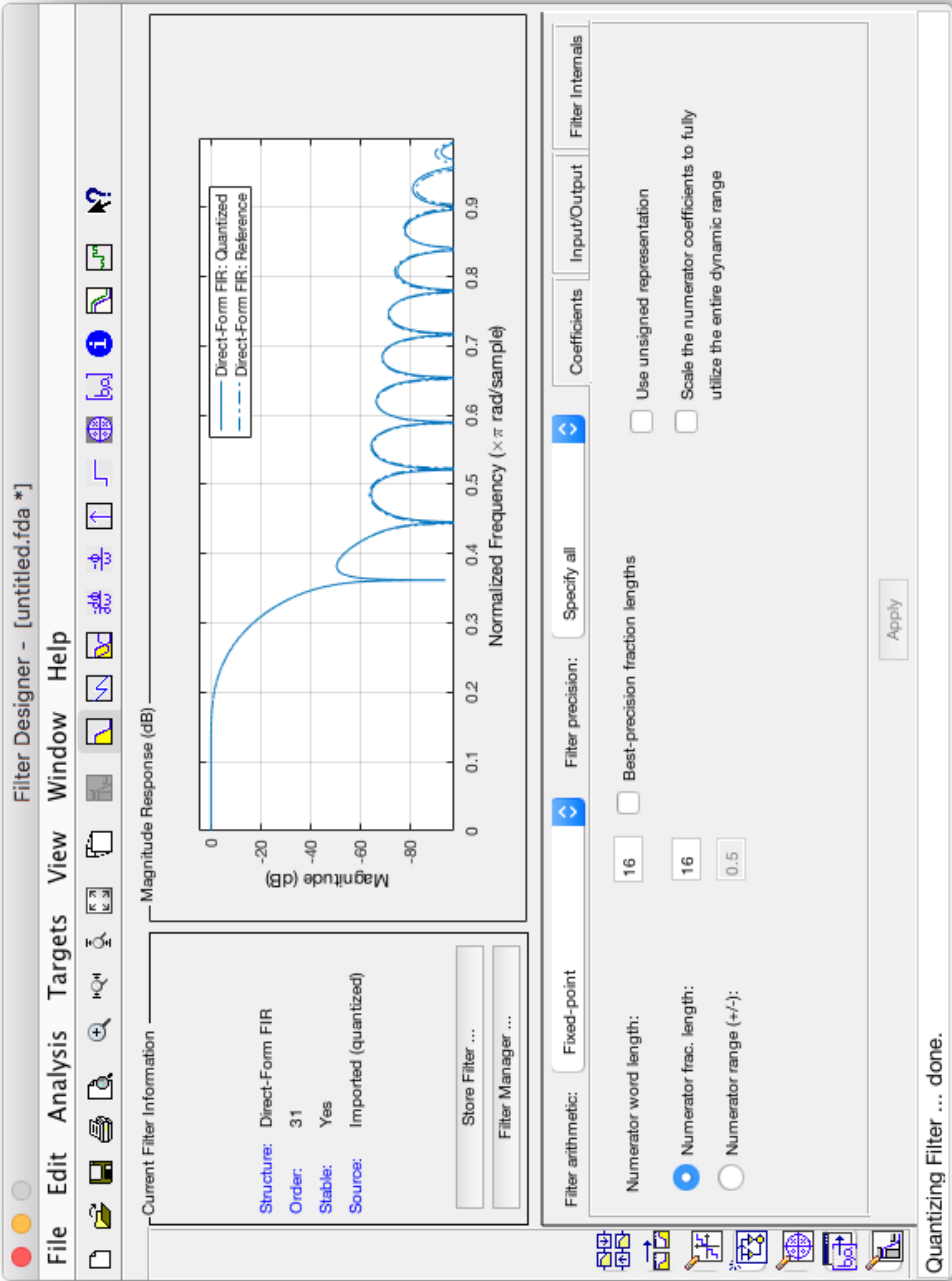


Figure 4.21: COEFFICIENT TYPES
Choose fixed-point types for coefficients, input, and output.

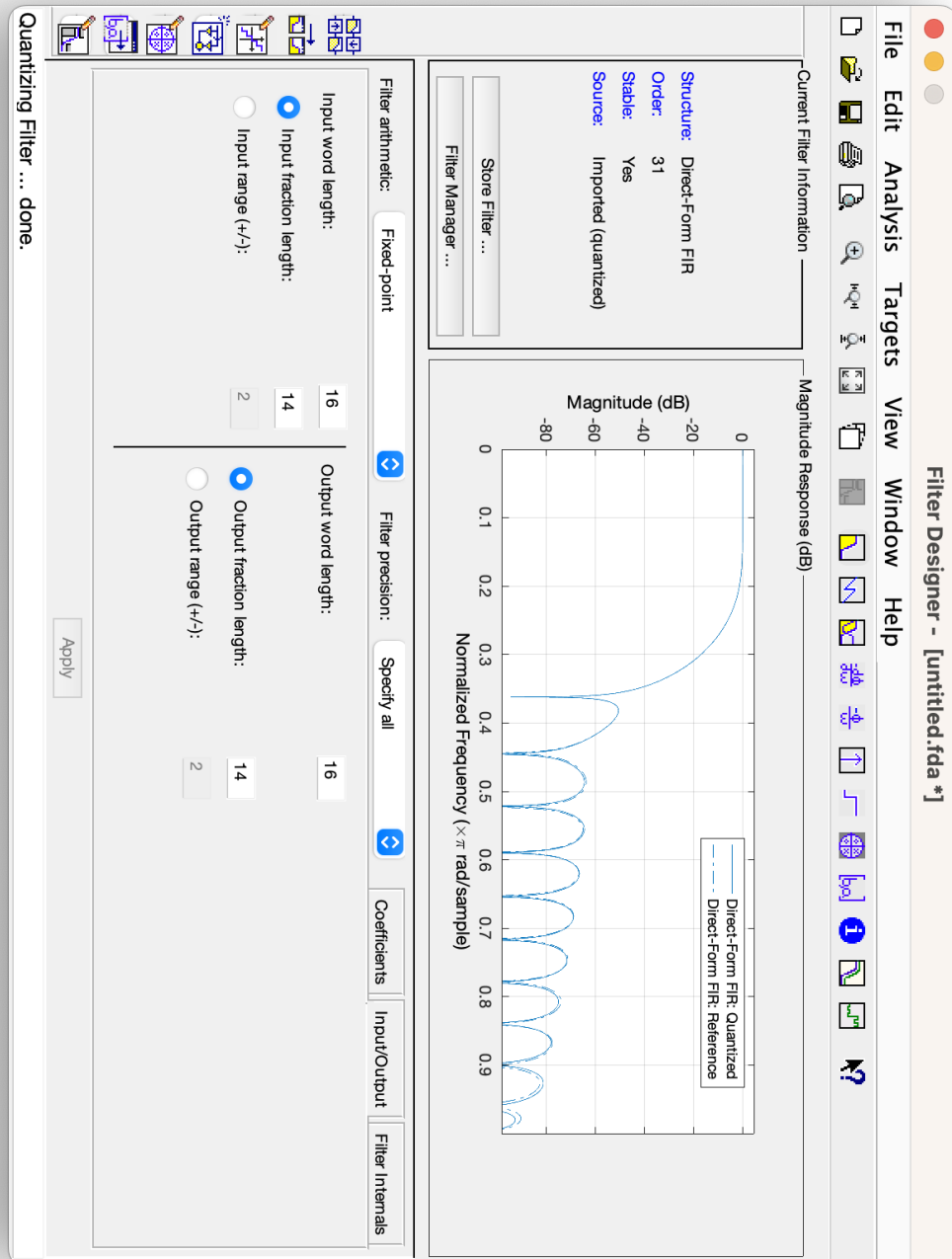


Figure 4.22: INPUT/OUTPUT TYPES
Choose fixed-point types for coefficients, input, and output.

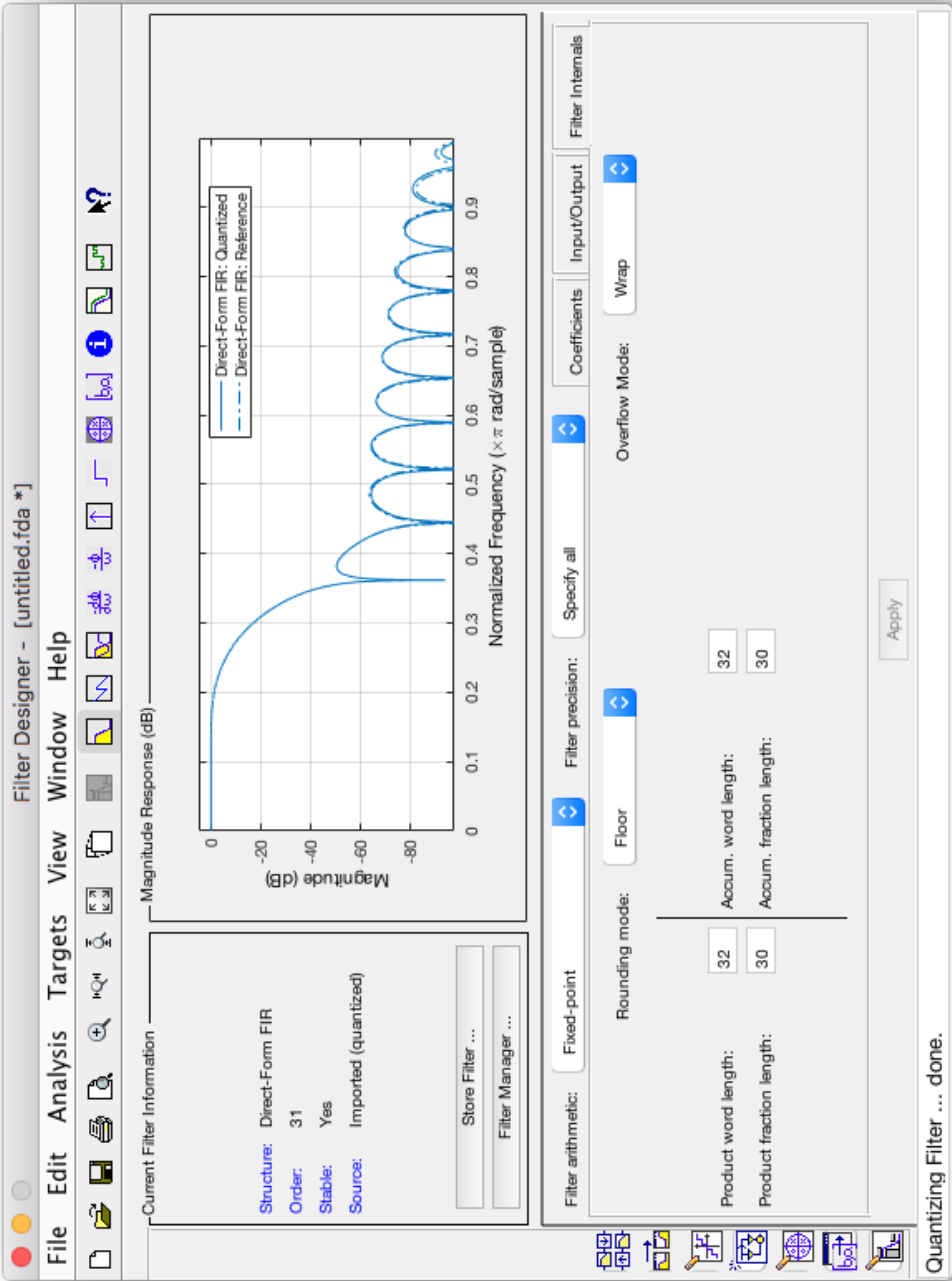


Figure 4.23: PRODUCT/SUM TYPES
Choose fixed-point types for products and sums.

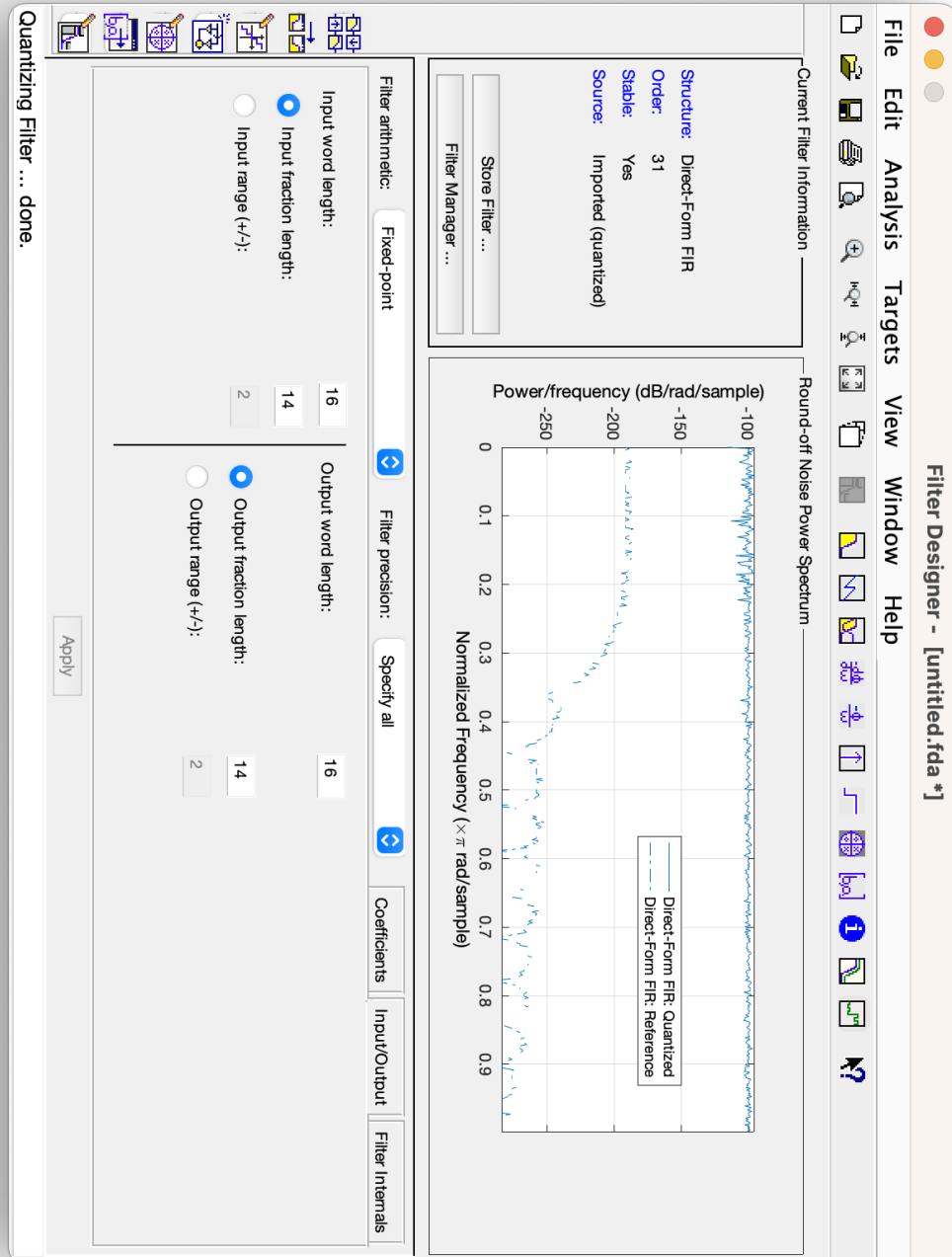


Figure 4.24: NOISE POWER SPECTRUM
Look at the noise power spectrum.

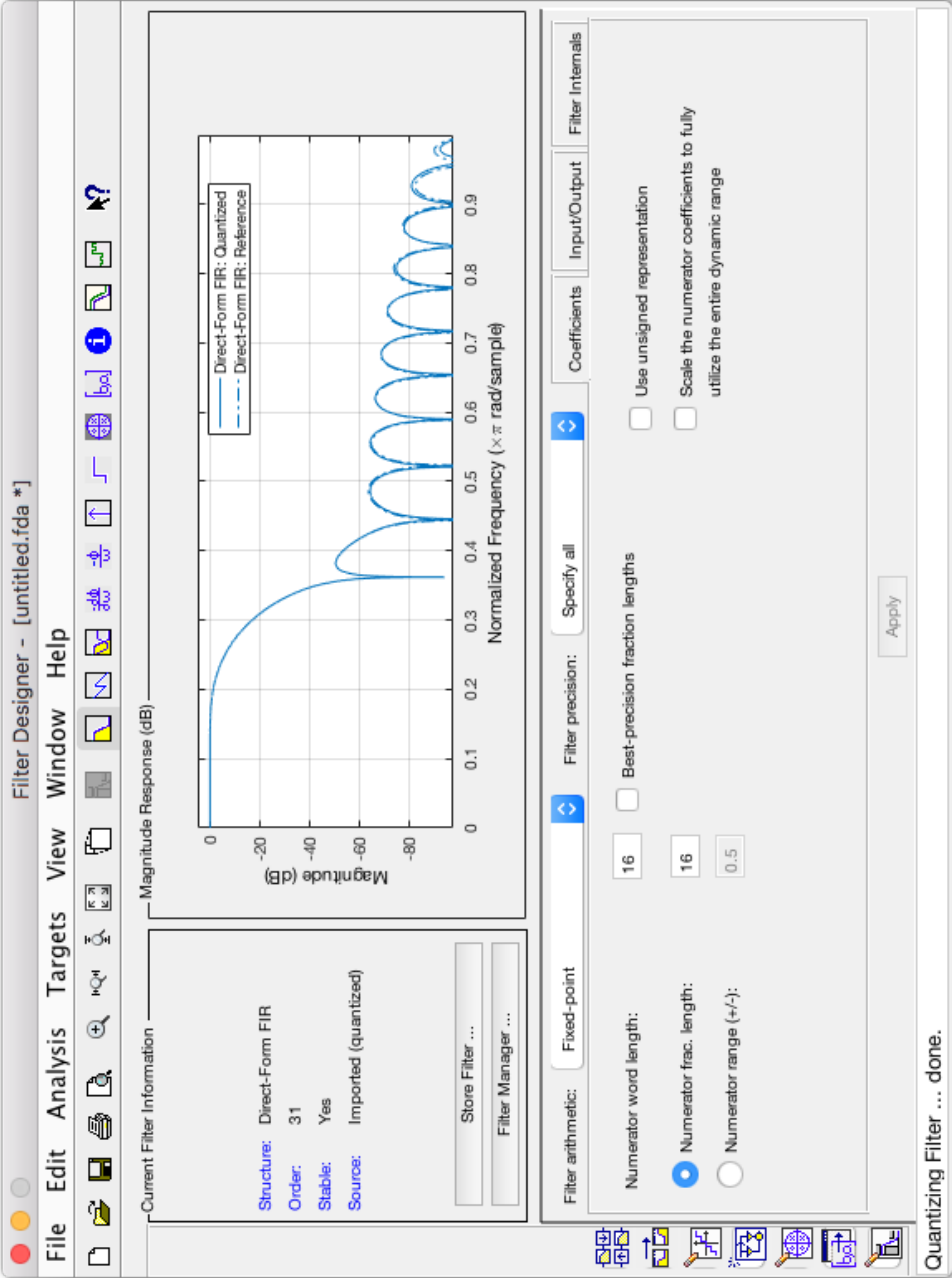


Figure 4.25: FINAL FILTER

View the different filter responses with the fixed-point settings from `fir_filter_dsp_types.m` and verify that the fixed-point filter meets your requirements as compared to the floating-point filter.

Chapter 5

Convert CORDIC Algorithm to Fixed Point

The examples for this chapter are in directory

`exercises/05cordicmagnitudephase`

Given Cartesian coordinates x and y , the problem is to compute polar coordinates, magnitude r and phase θ , of a complex signal $z = x + iy = re^{i\theta}$.

Using only shifts and adds, the COordinate Rotation DIgital Computer (CORDIC) [25, 26] algorithm can be used to apply an orthogonal Givens Rotation

$$G(\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}$$

to a point (x, y) , producing $G(\theta) \cdot (x, y) = (\sqrt{x^2 + y^2}, 0) = (r, 0)$. To say $G(\theta)$ is orthogonal means that $G(\theta)^T G(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. In addition to computing the magnitude r , you can obtain the phase θ by counting the directions of the CORDIC iterations it took to rotate y to zero.

givens_example.m

```
% Givens rotation applied to point (x,y) to transform
% to (hypot(x,y), 0)
x = 10;
y = 20;
theta = atan2(y,x);

% Givens rotation
G = [cos(theta)  sin(theta)
     -sin(theta)  cos(theta)]

G * [x
     y]

ans =

    22.3607
     0.0000
% hypot is a better way of computing sqrt(x^2+y^2)
hypot(x,y)

ans =

    22.3607

% G is orthogonal
G' * G

ans =

     1     0
     0     1
```

5.1 Derivation of the CORDIC Algorithm

You can factor out the cosine term of a Givens rotation to produce a series of rotations that require only shifts and adds, and iterate by choosing angles $\theta_n = \text{atan}(2^{-n})$ for $n = 0, 1, 2, 3, \dots$ so that $\tan(\theta_n) = 2^{-n}$ and $\cos(\theta_n) = 1 / (\sqrt{1 + 2^{-2n}})$. A clockwise rotation corresponds to $+\theta_n$, and a counterclockwise rotation corresponds to $-\theta_n$. A multiplication by 2^{-n} is equivalent to a right-shift by n bits.

$$\begin{aligned} G(\pm\theta_n) &= \begin{bmatrix} \cos(\theta_n) & \sin(\pm\theta_n) \\ -\sin(\pm\theta_n) & \cos(\theta_n) \end{bmatrix} \\ &= \cos(\theta_n) \begin{bmatrix} 1 & \frac{\sin(\pm\theta_n)}{\cos(\theta_n)} \\ -\frac{\sin(\pm\theta_n)}{\cos(\theta_n)} & 1 \end{bmatrix} \\ &= \cos(\theta_n) \begin{bmatrix} 1 & \tan(\pm\theta_n) \\ -\tan(\pm\theta_n) & 1 \end{bmatrix} \\ &= \frac{1}{\sqrt{1 + 2^{-2n}}} \cdot \begin{bmatrix} 1 & \pm 2^{-n} \\ \mp 2^{-n} & 1 \end{bmatrix}. \end{aligned}$$

A Givens rotation $G(\theta)$ can be expressed exactly as an infinite product of the series of rotations when the angle is in the right half-plane ($-\pi/2 \leq \theta \leq \pi/2$). If the angle is in the left half-plane, then it can be reflected into the right half-plane by adding π and negating x and y .

$$G(\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} = \prod_{n=0}^{\infty} \frac{1}{\sqrt{1 + 2^{-2n}}} \cdot \begin{bmatrix} 1 & \pm 2^{-n} \\ \mp 2^{-n} & 1 \end{bmatrix}.$$

The product $\prod_{n=0}^{\infty} \frac{1}{\sqrt{1 + 2^{-2n}}}$ quickly converges to a number close to $K = 0.607252935008881$, and so the expression can be replaced by this constant, which is known as the inverse CORDIC growth constant. The angle θ is the sum of the series of angles $\theta = \sum_0^{\infty} \pm\theta_n$, where the $\pm$ signs are determined as described in the next section.

In practice, we limit the number of iterations to one less than the fixed-point word length, which is when the value will be shifted off to zero, so the expression becomes the following.

$$G(\theta) \approx K_N \cdot \prod_{n=0}^N \begin{bmatrix} 1 & \pm 2^{-n} \\ \mp 2^{-n} & 1 \end{bmatrix}.$$

5.2 The CORDIC Algorithm

Reflect the initial point $(x_0, y_0) = (x, y)$ into the right half-plane, then iterate to drive y to zero.

At the n th iteration, if $y_n < 0$, then rotate counterclockwise through angle $-\theta_n$. Keep a running sum of the angles that you iterated over. Using the identity $\tan(-\theta_n) = -\tan(\theta_n)$, the iteration is

$$\begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix} = \begin{bmatrix} 1 & -2^{-n} \\ 2^{-n} & 1 \end{bmatrix} \cdot \begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_n - 2^{-n}y_n \\ y_n + 2^{-n}x_n \end{bmatrix}$$

$$\theta = \theta - \theta_n$$

Otherwise rotate clockwise through angle θ_n .

$$\begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix} = \begin{bmatrix} 1 & 2^{-n} \\ -2^{-n} & 1 \end{bmatrix} \cdot \begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_n + 2^{-n}y_n \\ y_n - 2^{-n}x_n \end{bmatrix}$$

$$\theta = \theta + \theta_n$$

The product of the gain factors $\sqrt{1 + 2^{-2n}}$ quickly converges to around 1.6468. In MATLAB

```
n = 0:20;
prod(sqrt(1 + 2.^(-2*n)))

ans =

    1.6468
```

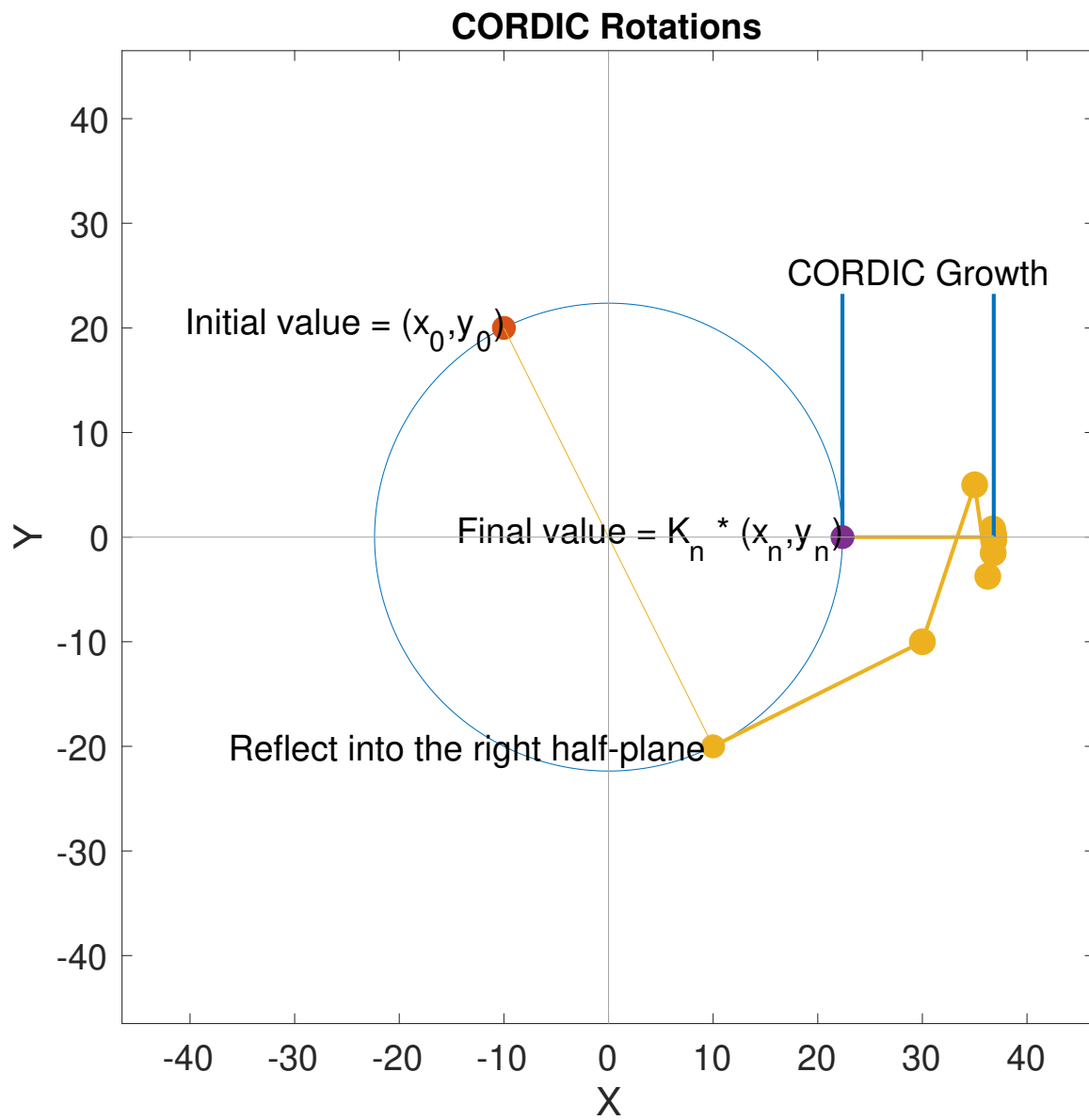
The overall number of additional integer bits to add to prevent overflow is

```
ceil(log2(1.6468 * sqrt(2)))

ans =

    2
```

The $\sqrt{2}$ factor is the for the maximum growth in the hypotenuse of (x, y) .



CORDIC ROTATIONS

`visualize_cordic(-10,20)`. The initial value is $(x, y) = (-10, 20)$.

5.3 CORDIC Kernel

The CORDIC algorithm was invented in 1959 by Jack Volder [25]. The CORDIC algorithm is desirable in embedded applications because it only uses shifts and adds.

The CORDIC vectoring algorithm rotates x and y through N iterations is given in the following MATLAB code. Each successive rotation cuts y by half of x , reversing direction if it overshoots the x axis, and x grows by half of y , until y is near zero. The angles are summed in `theta`, accounting for clockwise and counterclockwise rotations.

The angle for each rotation is stored in constant vector `a = atan(2.^ -(0:N-1)')`. Since `a` is constant, it will appear as a constant table in the generated code and will not call the `atan()` function in the generated code. In this implementation, the angle is in radians. If you wish to work in normalized units, then you can scale `a` by a normalization factor such as π or 2π . The algorithm is given in the following code:

```
function [x, y, theta] = cordic_vectoring_kernel(x, y, theta, N, a)
    if x < 0
        % Reflect into the right half-plane
        x(:) = -x;
        y(:) = -y;
        angle_correction = cast(pi, 'like', theta);
    else
        angle_correction = cast(0, 'like', theta);
    end
    for n = 0:N-1
        x_shifted = bitsra(x, n);
        y_shifted = bitsra(y, n);
        if y < 0
            % Counterclockwise rotation
            x(:) = x - y_shifted;
            y(:) = y + x_shifted;
            theta(:) = theta - a(n+1);
        else
            % Clockwise rotation
            x(:) = x + y_shifted;
            y(:) = y - x_shifted;
            theta(:) = theta + a(n+1);
        end
    end
    theta(:) = theta + angle_correction;
end
```

The growth in x is a constant that quickly converges to around 1.6468. For given N , it is exactly

$$A_n = \text{prod}(\text{sqrt}(1+2.^{-2*(0:N-1)}))$$

At the end of the CORDIC rotations, x is normalized by multiplying by the constant

$$K_n = 1/A_n$$

You can skip the normalization by the inverse of the gain constant K_n if you factor it into the “real-world value” interpretation of the output in your application.

5.4 CORDIC Datatypes

Since the growth constant $A_n \approx 1.6468$ is less than 2, you know that you will only ever need at most one bit of growth, and you can choose datatypes for the CORDIC algorithm based only on the type z of the input. This allows the computations to be done in-place. Also, the maximum number of iterations N (and hence the optimal accuracy) will be the word length of the input, because y will shift out to 0 after that.

The following is a good choice for the types in the CORDIC algorithm for DSP applications where you want to do the computation in-place in the same word length as the input, and therefore matching builtin C integer types. In HDL applications, you may want to let the word length grow one bit to $w + 1$ and increase the fraction length by one, which will give you one more bit of precision.

```
function [T,N,a] = cordic_magnitude_angle_types(z)
    if isfi(z)
        datatype = z.DataType;
        w = z.WordLength;
        N = w - 1;
        f = z.FractionLength;
        F_xy = fimath('RoundingMethod', 'Floor', ...
            'OverflowAction', 'Wrap', ...
            'SumMode', 'SpecifyPrecision', ...
            'SumWordLength', w, ...
            'SumFractionLength', w-3);
        F_theta = fimath('RoundingMethod', 'Floor', ...
            'OverflowAction', 'Wrap', ...
            'SumMode', 'SpecifyPrecision', ...
            'SumWordLength', w, ...
            'SumFractionLength', w-4);
        T.a = fi([], 0, w, w-1, 'DataType', datatype);
        T.x = fi([], 1, w, w-3, F_xy, 'DataType', datatype);
        T.y = fi([], 1, w, w-3, F_xy, 'DataType', datatype);
        T.theta = fi([], 1, w, w-4, F_theta, 'DataType', datatype);
        % Create the angle table with no fimath and
        % default nearest rounding.
        a = fi(cordic_angular_increments(N), T.theta.numericType);
    else
```

```

    N = 52;
    T.a = cast([], 'like', z);
    T.x = cast([], 'like', z);
    T.y = cast([], 'like', z);
    T.theta = cast([], 'like', z);
    a = cast(cordic_angular_increments(N), 'like', z);
end
end

```

5.5 CORDIC Cartesion-to-Polar function

The final CORDIC algorithm computes magnitude and phase from a vector of complex inputs. The initial wrapper function operates over vectors and arrays, and the kernel vectoring algorithm operates over scalars. If this function is called with scalar input, then code generation will elide the for loop, and you will be left with only the scalar operation.

```

function [magnitudeZ, angleZ] = cordic_cart2pol(Z)
    [T,N,a] = cordic_magnitude_angle_types(real(Z(1)));
    Kn = inverse_cordic_growth_constant(N);
    magnitudeZ = zeros(size(Z), 'like', T.x);
    angleZ = zeros(size(Z), 'like', T.theta);
    for i = 1:numel(Z)
        x = cast(real(Z(i)), 'like', T.x);
        y = cast(imag(Z(i)), 'like', T.y);
        theta = zeros(1, 'like', T.theta);
        [magnitudeZ(i), ~, angleZ(i)] = ...
            cordic_vectoring_kernel(x, y, theta, N, a);
        magnitudeZ(i) = magnitudeZ(i) * Kn;
    end
    magnitudeZ = removefimath(magnitudeZ);
    angleZ = removefimath(angleZ);
end

```

```

function a = cordic_angular_increments(N)
    a = atan(2.^-(0:N-1)');
end

```

```

function Kn = inverse_cordic_growth_constant(N)
% Kn = INVERSE_CORDIC_GROWTH_CONSTANT(N) returns the inverse of the
% CORDIC growth factor after N iterations. The inverse cordic growth
% constant Kn quickly converges to around 0.60725.
    Kn = 1/cordic_growth_constant(N);
end

```

```
function An = cordic_growth_constant(N)
% CORDIC_GROWTH_CONSTANT(N) returns the CORDIC growth factor after N
% iterations. The cordic growth constant quickly converges to around
% 1.6468.
    An = prod(sqrt(1+2.^(-2*(0:double(N)-1))));
end
```

5.6 CORDIC Test Function

Edit and run the test function

test_cordic_cart2pol

Edit `cordic_cart2pol` function and experiment with different data types and `fimath` settings.
Re-run the test function to see the results.

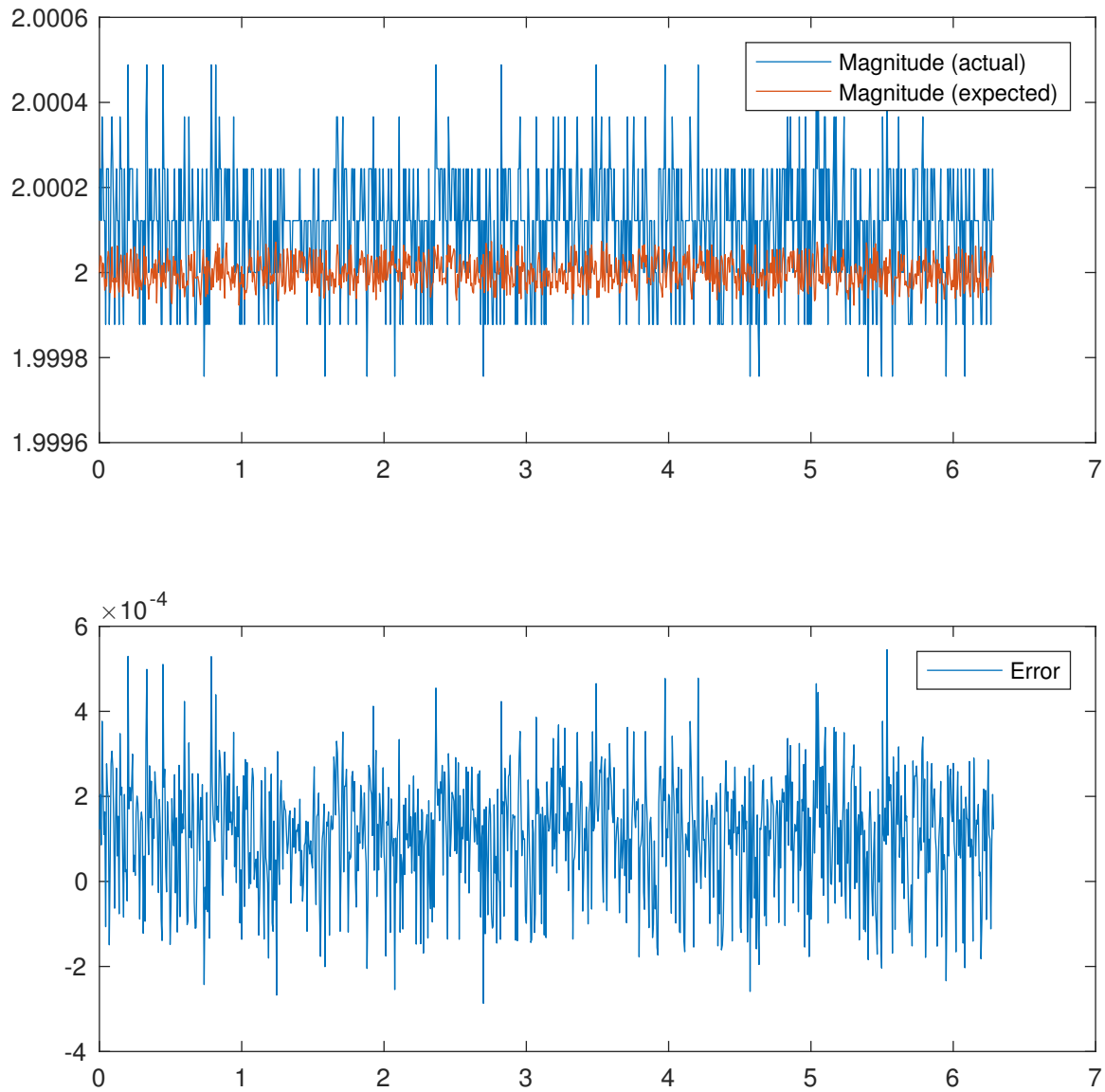


Figure 5.1: CORDIC MAGNITUDE OUTPUT
Look at the error plots to see if the actual values are close to the expected values.

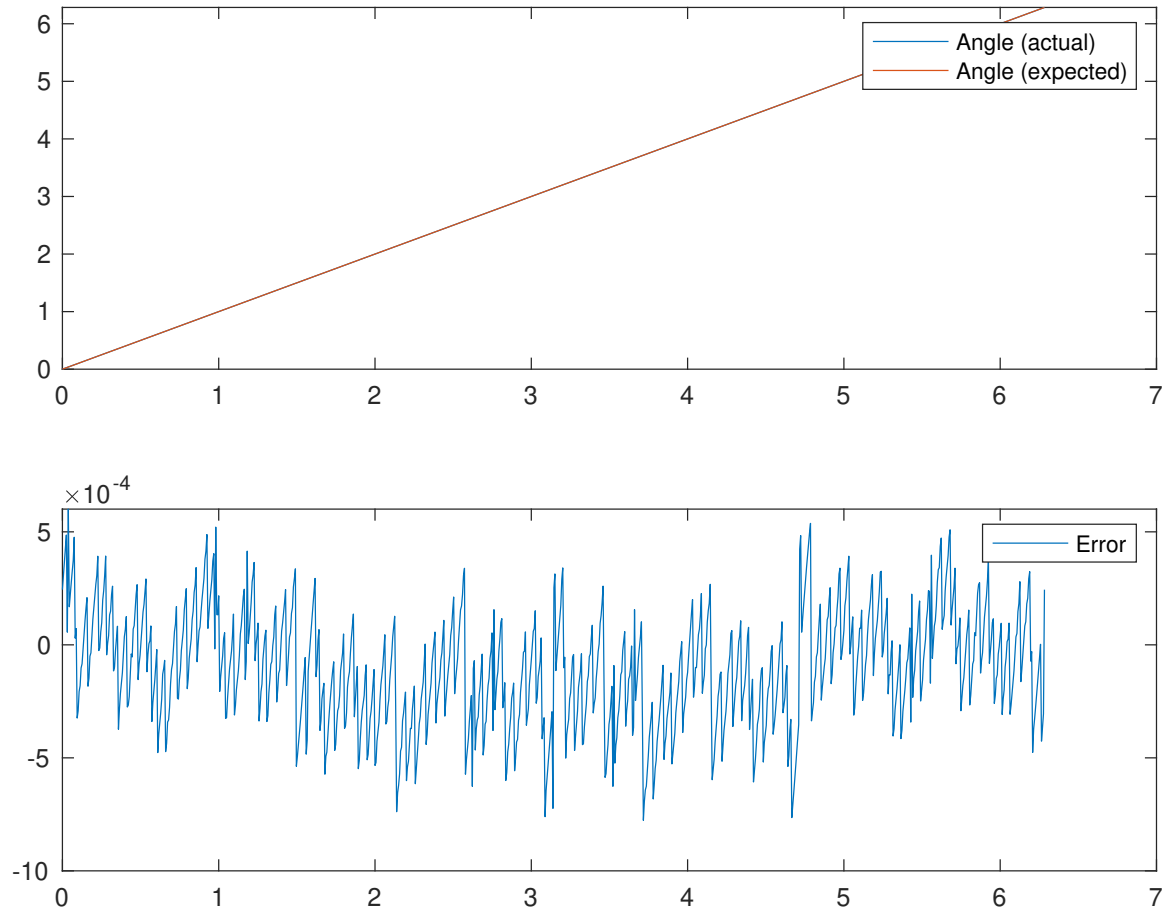


Figure 5.2: CORDIC ANGLE OUTPUT

Look at the error plots to see if the actual values are close to the expected values.

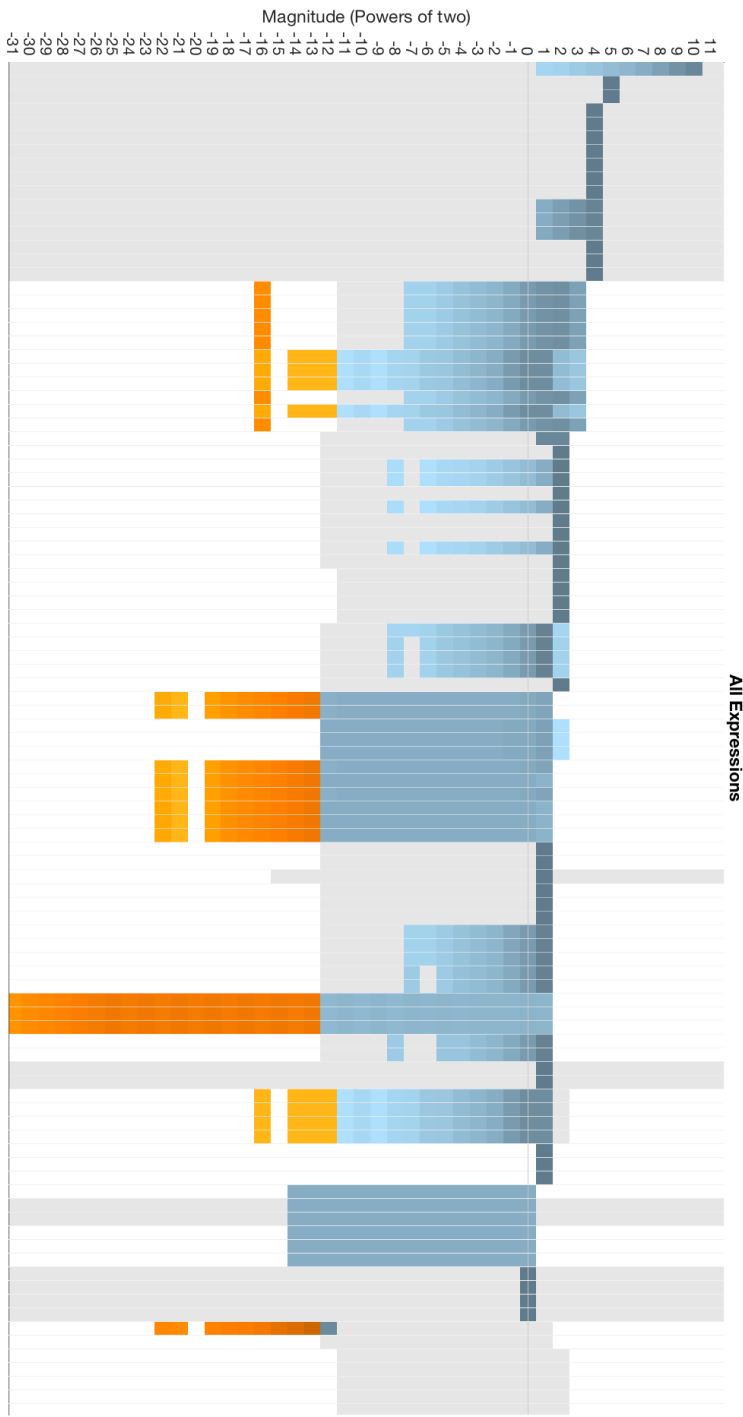


Figure 5.3: ALL EXPRESSIONS
Look at the datatype visualization for all the expressions and you can see that the types are tight to their upper bounds, and that there are no overflows.

5.6.1 Generate C from the MATLAB function

You can generate C code with default settings with the following command.

```
codegen cordic_cart2pol -args {Z(1)} -config:lib -launchreport
```

The following is the generated C code.

```

#include "cordic_cart2pol.h"
void cordic_cart2pol(const cint16_T Z, short *magnitudeZ, short *angleZ)
{
    static const short a[15] = {3217, 1899, 1003, 509, 256, 128, 64, 32,
                                16, 8, 4, 2, 1, 0, 0};

    int n;
    short b0;
    short theta;
    short x;
    short y;
    x = Z.re;
    y = Z.im;
    theta = 0;
    if (Z.re < 0) {
        /* Reflect into the right half-plane */
        x = (short)-Z.re;
        y = (short)-Z.im;
        b0 = 12867;
    } else {
        b0 = 0;
    }
    for (n = 0; n < 15; n++) {
        short x_shifted;
        short y_shifted;
        x_shifted = (short)(x >> n);
        y_shifted = (short)(y >> n);
        if (y < 0) {
            /* Counterclockwise rotation */
            x -= y_shifted;
            y += x_shifted;
            theta -= a[n];
        } else {
            /* Clockwise rotation */
            x += y_shifted;
            y -= x_shifted;
            theta += a[n];
        }
    }
    *angleZ = (short)(theta + b0);
    *magnitudeZ = (short)((x * 19898) >> 15);
}

```

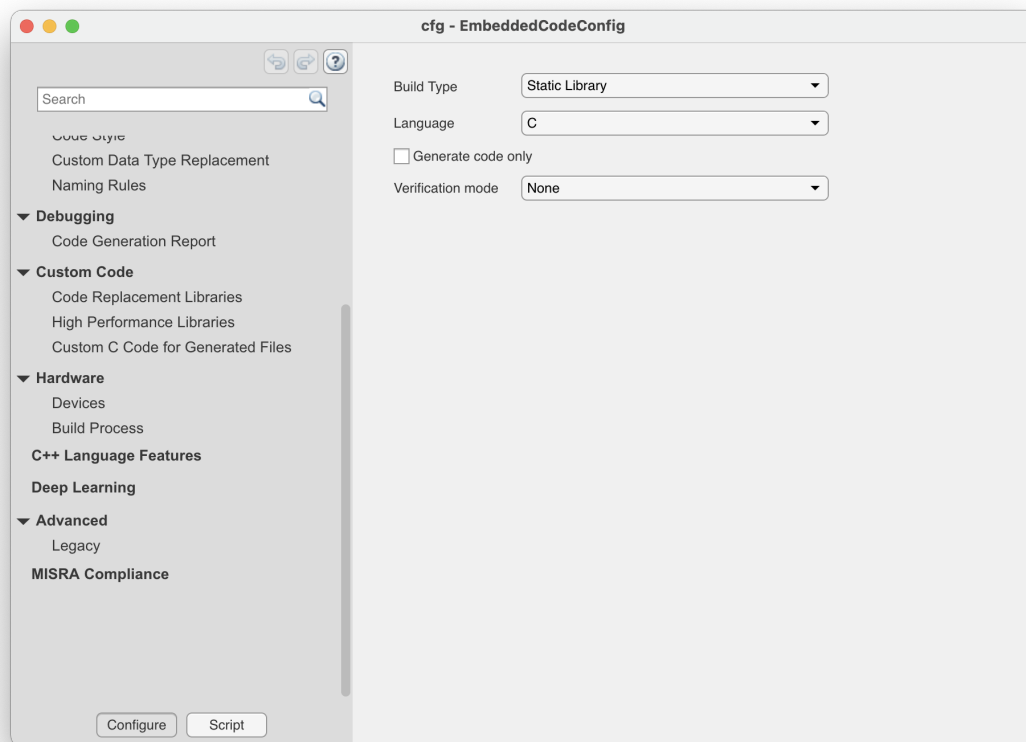
Code Generation Settings for C

Add the following lines to the test file to generate C++ code with specific settings.

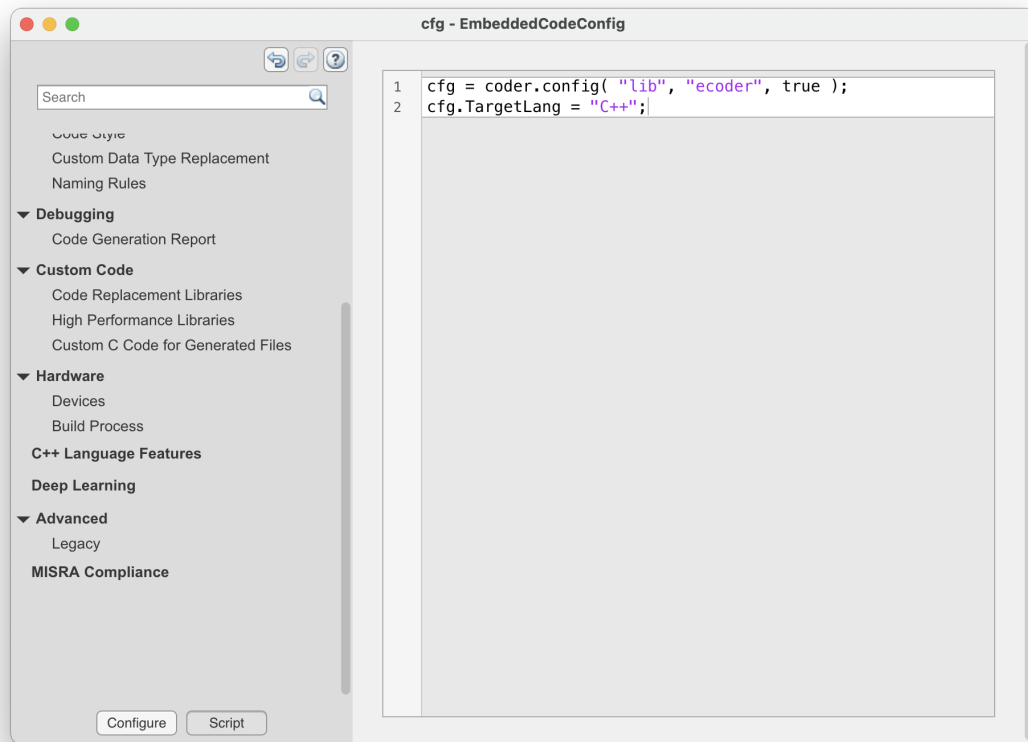
```
%% Generate C++ code with specific settings
cfg = coder.config( "lib", "ecoder", true );
cfg.PurelyIntegerCode = true;
cfg.TargetLang = "C++";
codegen cordic_cart2pol -args {Z(1)} -config cfg -launchreport
```

To see all settings configurations, create a `coder.config` object and open it.

```
cfg = coder.config( "lib" );
open cfg
```



CODER CONFIGURATION APP
`cfg = coder.config( "lib"); open cfg`



CODER CONFIGURATION SCRIPT
Select "Script" to copy and paste your changes.

5.6.2 Generate HDL from the MATLAB function

You can generate HDL code with default settings with the following command.

```
codegen cordic_cart2pol -args {Z(1)} -config:hdl -launchreport
```

The following is the generated HDL code. Even though the generated HDL code is high quality, there is a loop. The loop means that the code has to finish executing before the next clock tick, which means that the clock can't go faster than this code can go. An ASIC or FPGA can only go as fast as the longest path between registers (known in Simulink as Delay blocks). It is possible to further elaborate the MATLAB code to add registers, parallelization, and timing, but it is easier and more natural to elaborate the design in Simulink for efficient HDL code generation.

Section 5.7 shows how to integrate the CORDIC kernel in MATLAB with Simulink to implement a hardware-optimized CORDIC algorithm that shares the resources for the kernel.

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;
USE work.cordic_cart2pol_pkg.ALL;

ENTITY cordic_cart2pol IS
    PORT( Z_re      : IN      std_logic_vector(15 DOWNTO 0);  -- sfix16_En13
          Z_im      : IN      std_logic_vector(15 DOWNTO 0);  -- sfix16_En13
          magnitudeZ : OUT     std_logic_vector(15 DOWNTO 0);  -- sfix16_En13
          angleZ     : OUT     std_logic_vector(15 DOWNTO 0)  -- sfix16_En12
    );
END cordic_cart2pol;

ARCHITECTURE rtl OF cordic_cart2pol IS

    -- Constants
    CONSTANT a : vector_of_signed16(0 TO 14) :=
        (to_signed(16#0C91#, 16), to_signed(16#076B#, 16), to_signed(16#03EB#, 16), to_signed(16#0100#, 16), to_signed(16#0080#, 16), to_signed(16#0040#, 16), to_signed(16#0010#, 16), to_signed(16#0008#, 16), to_signed(16#0004#, 16), to_signed(16#0001#, 16), to_signed(16#0000#, 16), to_signed(16#0000#, 16)); -- s

    -- Signals
    SIGNAL Z_re_signed      : signed(15 DOWNTO 0);  -- sfix16_En13
    SIGNAL Z_im_signed      : signed(15 DOWNTO 0);  -- sfix16_En13
    SIGNAL magnitudeZ_tmp   : signed(15 DOWNTO 0);  -- sfix16_En13
    SIGNAL angleZ_tmp       : signed(15 DOWNTO 0);  -- sfix16_En12

BEGIN
    Z_re_signed <= signed(Z_re);
```

```

Z_im_signed <= signed(Z_im);

cordic_cart2pol_1_output : PROCESS (Z_im_signed, Z_re_signed)
    VARIABLE tmp : signed(15 DOWNT0 0);
    VARIABLE tmp_0 : signed(15 DOWNT0 0);
    VARIABLE tmp_1 : signed(15 DOWNT0 0);
    VARIABLE tmp_2 : signed(15 DOWNT0 0);
    VARIABLE c : signed(15 DOWNT0 0);
    VARIABLE c_0 : signed(15 DOWNT0 0);
    VARIABLE tmp_3 : signed(15 DOWNT0 0);
    VARIABLE tmp_4 : signed(15 DOWNT0 0);
    VARIABLE c_1 : signed(15 DOWNT0 0);
    VARIABLE c_2 : signed(15 DOWNT0 0);
    VARIABLE cast : signed(16 DOWNT0 0);
    VARIABLE cast_0 : signed(16 DOWNT0 0);
    VARIABLE cast_1 : signed(16 DOWNT0 0);
    VARIABLE cast_2 : signed(16 DOWNT0 0);
    VARIABLE mul_temp : signed(31 DOWNT0 0);
    VARIABLE cast_3 : vector_of_unsigned8(0 TO 14);
    VARIABLE cast_4 : vector_of_unsigned8(0 TO 14);
    VARIABLE cast_5 : vector_of_unsigned8(0 TO 14);
    VARIABLE cast_6 : vector_of_unsigned8(0 TO 14);
BEGIN
    c := to_signed(16#0000#, 16);
    c_0 := to_signed(16#0000#, 16);
    c_1 := to_signed(16#0000#, 16);
    c_2 := to_signed(16#0000#, 16);
    cast_0 := to_signed(16#00000#, 17);
    cast_2 := to_signed(16#00000#, 17);
    cast := to_signed(16#00000#, 17);
    cast_1 := to_signed(16#00000#, 17);
    --HDL code generation from MATLAB function: cordic_cart2pol
    -- Counterclockwise rotation
    -- Clockwise rotation
    -- Reflect into the right half-plane
    tmp_1 := Z_im_signed;
    IF Z_re_signed < to_signed(16#0000#, 16) THEN
        tmp := to_signed(16#3243#, 16);
        cast := resize(Z_re_signed, 17);
        cast_0 := - (cast);
        tmp_0 := cast_0(15 DOWNT0 0);
        cast_1 := resize(Z_im_signed, 17);
        cast_2 := - (cast_1);
        tmp_1 := cast_2(15 DOWNT0 0);
    ELSE

```

```

    tmp := to_signed(16#0000#, 16);
    tmp_0 := Z_re_signed;
END IF;
tmp_4 := tmp_1;
tmp_3 := tmp_0;
tmp_2 := to_signed(16#0000#, 16);

FOR n IN 0 TO 14 LOOP
    cast_3(n) := unsigned(to_signed(n, 32)(7 DOWNT0 0));
    c_2 := SHIFT_RIGHT(tmp_4, to_integer(cast_3(n)));
    cast_4(n) := unsigned(to_signed(n, 32)(7 DOWNT0 0));
    c_1 := SHIFT_RIGHT(tmp_3, to_integer(cast_4(n)));
    IF tmp_4 < to_signed(16#0000#, 16) THEN
        tmp_3 := tmp_3 - c_2;
        tmp_4 := tmp_4 + c_1;
    ELSE
        tmp_3 := tmp_3 + c_2;
        tmp_4 := tmp_4 - c_1;
    END IF;
    cast_5(n) := unsigned(to_signed(n, 32)(7 DOWNT0 0));
    c_0 := SHIFT_RIGHT(tmp_1, to_integer(cast_5(n)));
    cast_6(n) := unsigned(to_signed(n, 32)(7 DOWNT0 0));
    c := SHIFT_RIGHT(tmp_0, to_integer(cast_6(n)));
    IF tmp_1 < to_signed(16#0000#, 16) THEN
        tmp_0 := tmp_0 - c_0;
        tmp_1 := tmp_1 + c;
        tmp_2 := tmp_2 - a(n);
    ELSE
        tmp_0 := tmp_0 + c_0;
        tmp_1 := tmp_1 - c;
        tmp_2 := tmp_2 + a(n);
    END IF;
END LOOP;

mul_temp := tmp_3 * to_signed(16#4DBA#, 16);
magnitudeZ_tmp <= mul_temp(30 DOWNT0 15);
angleZ_tmp <= tmp_2 + tmp;
END PROCESS cordic_cart2pol_1_output;

magnitudeZ <= std_logic_vector(magnitudeZ_tmp);

angleZ <= std_logic_vector(angleZ_tmp);

END rtl;
```

5.7 Combining MATLAB and Simulink for Efficient HDL

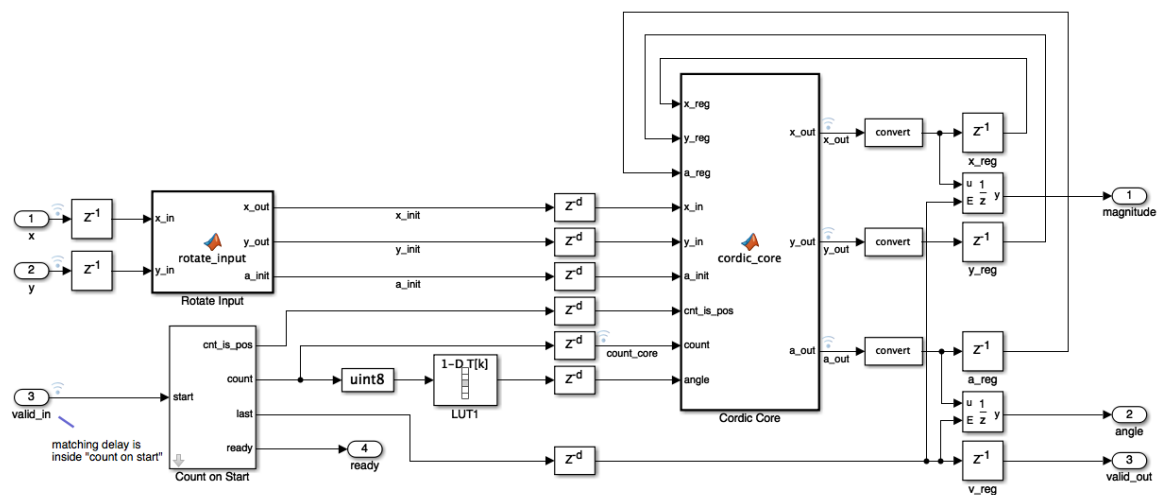
The following model integrates MATLAB code in the CORDIC kernel with Simulink to produce an HDL-optimized resource-shared design. The result of each CORDIC iteration is stored in a register (a Simulink delay block) and fed back into the CORDIC kernel for the next iteration. The resources for the kernel are shared between all the iterations. Breaking up each iteration with registers means that only one iteration needs to complete before the next clock tick, and so the clock rate can be very fast.

```
solutions/05cordicmagnitudephase/simulink_cordic_cart2pol_model.slx
```

Run

```
test_simulink_cordic_cart2pol
```

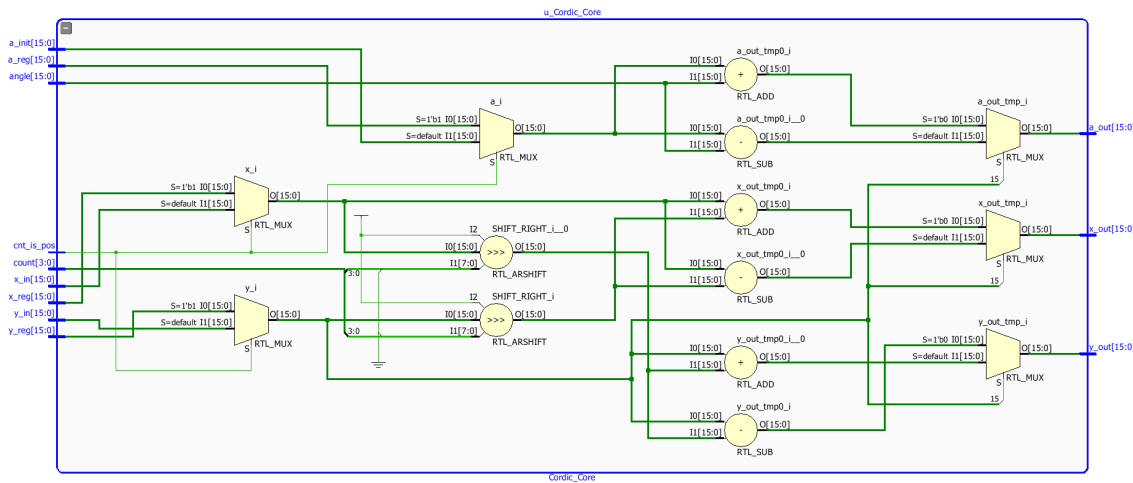
to open and test the model.



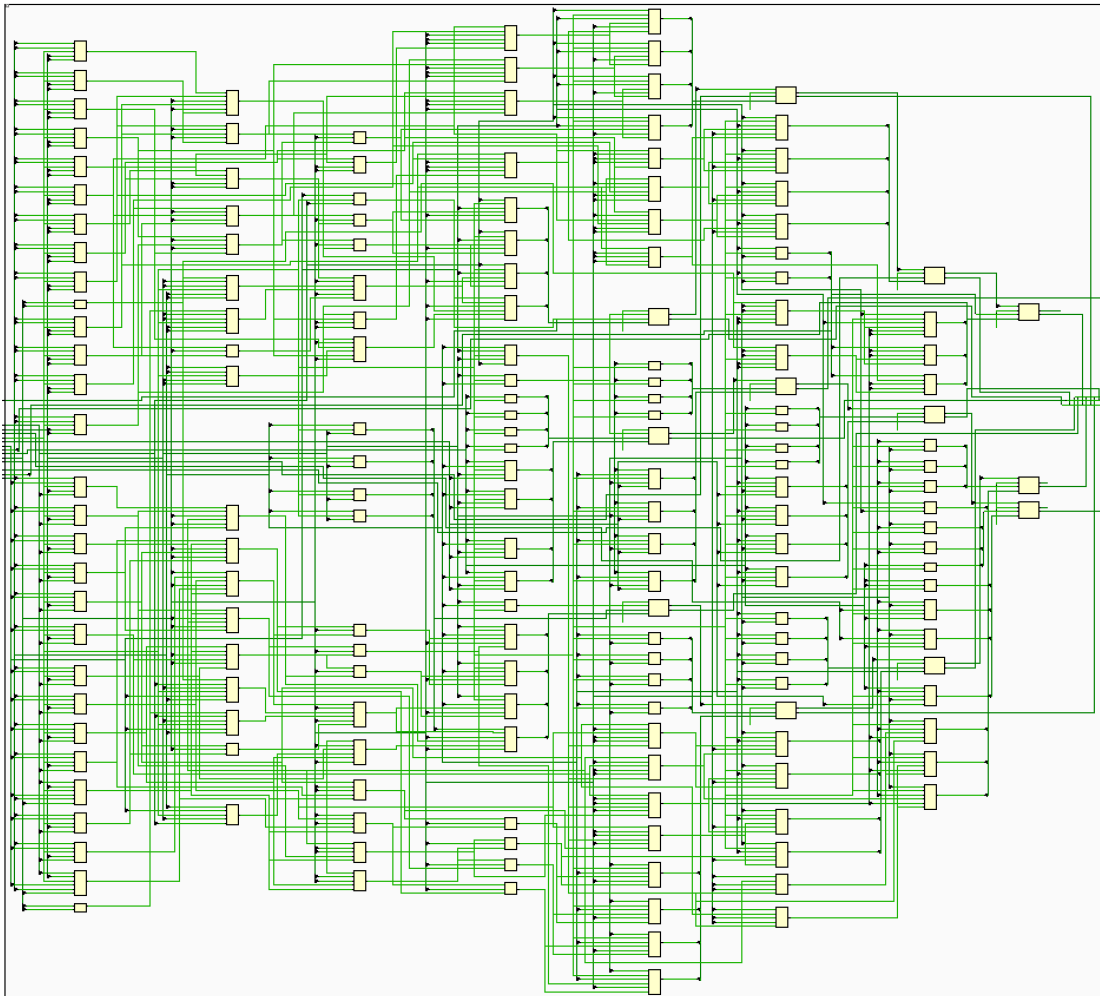
HDL code generation produced the following schematics. You can see that there are only shifts and adds. Simulink coupled with MATLAB code in MATLAB Function blocks produces good HDL because you can model dataflow, timing, and parallelism.

Some details about this model:

- The model is optimized to minimize area. The CORDIC core is shared between all iterations. If you want to maximize throughput, you can duplicate and pipeline the CORDIC core, one for each iteration.
- The CORDIC gain is not factored out in the model. The magnitude was used as a relative value in the channelized receiver developed in Chapter 6 Converting DFT Polyphase Filter Bank to Fixed Point. As long as all channels have the same gain, then their magnitudes compare. Not factoring out the CORDIC gain allows you to save resources (See also [32], [11]). However, we do need to factor out the CORDIC gain in Chapter 7 Linear Systems of Equations to preserve the attributes of the QR decomposition. The CORDIC gain factor will be addressed in that chapter.



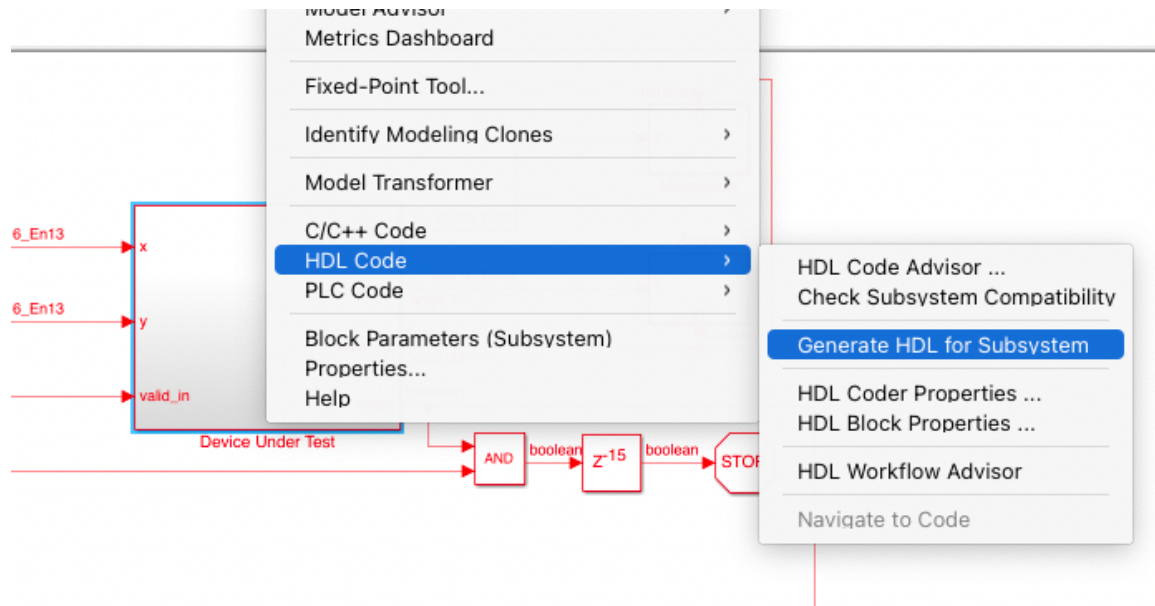
VIVADO RTL SCHEMATIC



VIVADO SYNTHESIS SCHEMATIC

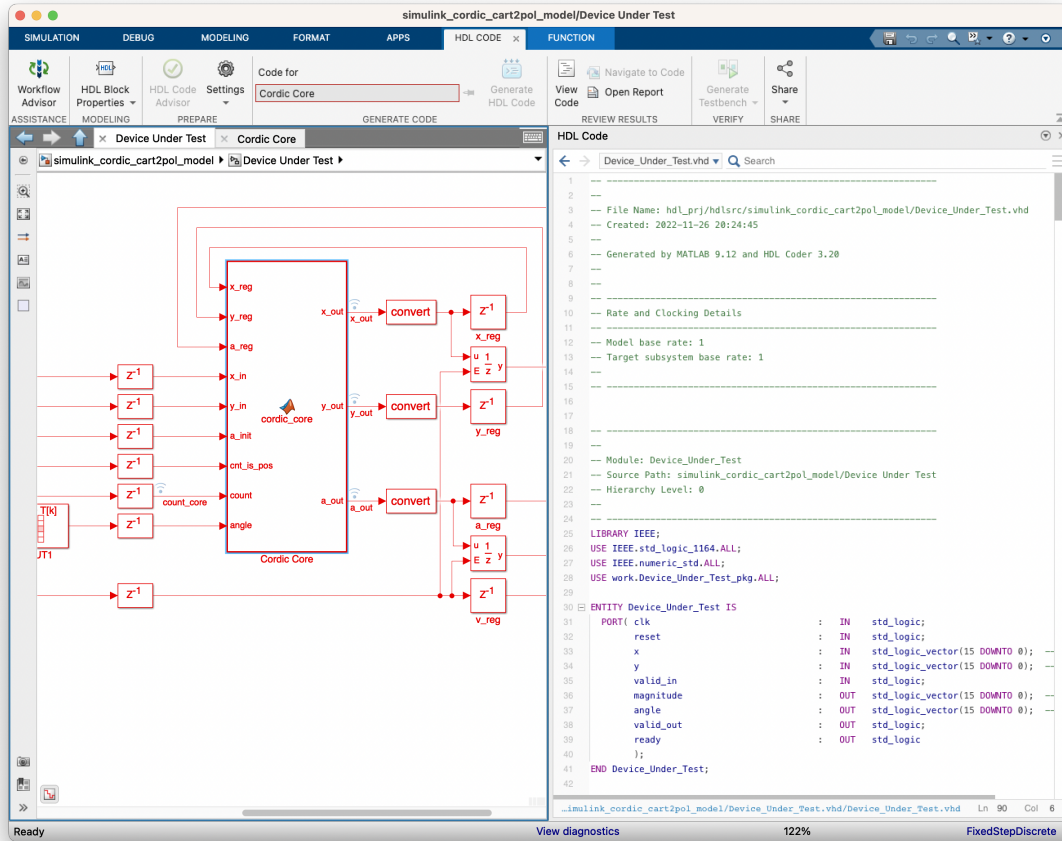
5.7.1 Generate HDL from Simulink Model

To generate HDL from the model, right-click on the “Device Under Test” subsystem block and select one of the HDL Coder options.



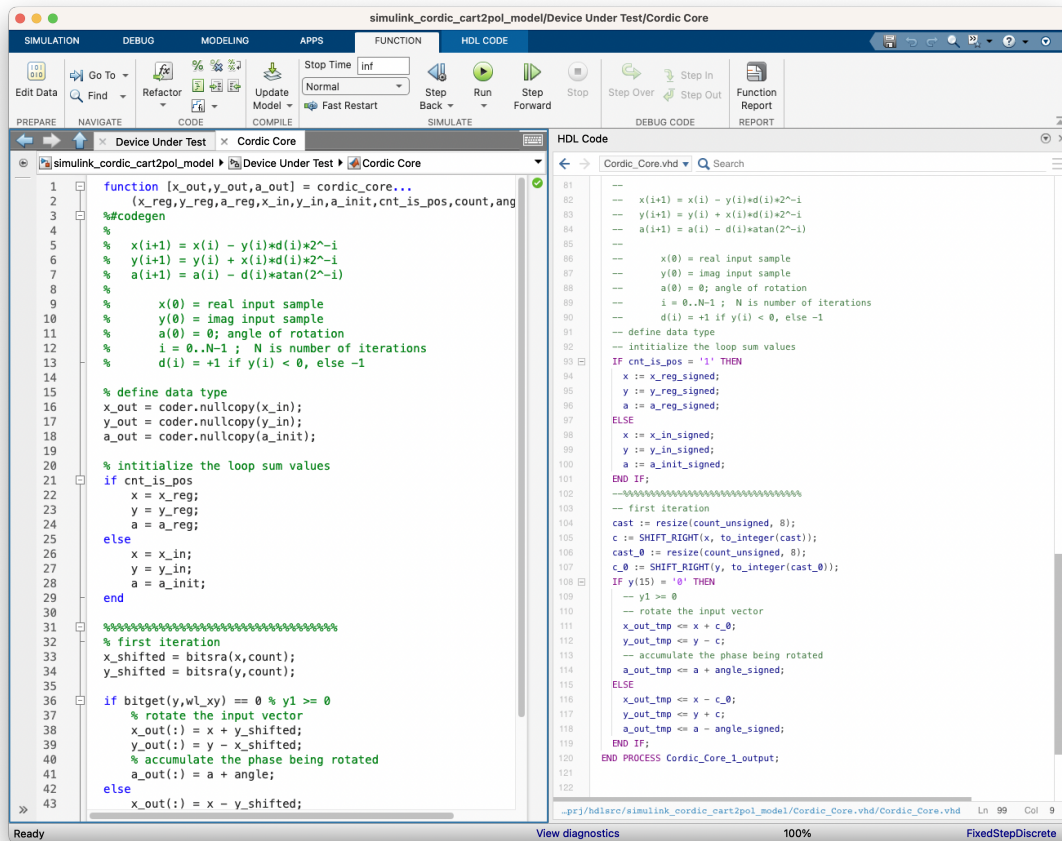
GENERATE HDL CODE FROM SIMULINK MODEL

Right-click on the “Device Under Test” subsystem block and select one of the HDL Coder options.



NAVIGATE TO CODE FROM BLOCK

After HDL code generation completes, navigate to the VHDL files by clicking on Cordic Core subsystem.



CORDIC CORE SUBSYSTEM

VHDL file corresponding to the MATLAB code in the Cordic Core subsystem.

After HDL code generation completes, look at the following file to see the core of the CORDIC algorithm.

hdl_prj/hdlsrc/simulink_cordic_cart2pol_model/Cordic_Core.vhd

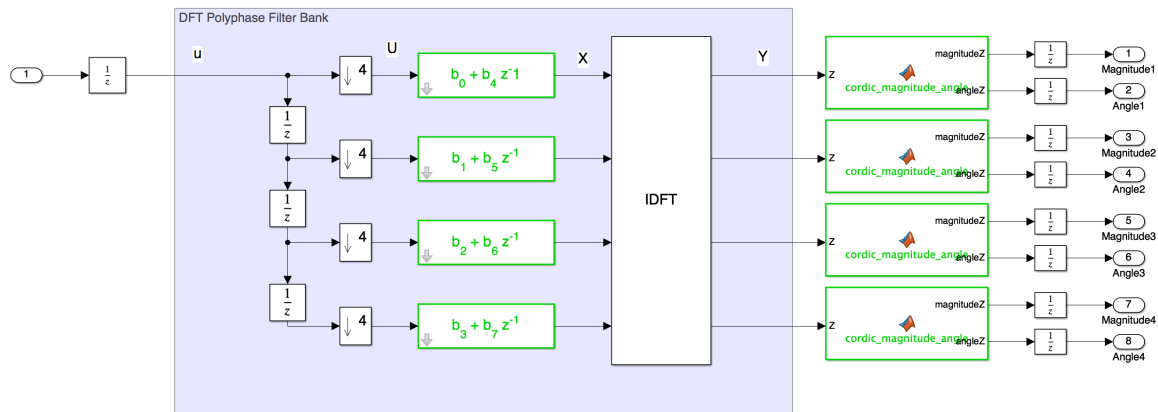
```
cast := resize(count_unsigned, 8);
c := SHIFT_RIGHT(x, to_integer(cast));
cast_0 := resize(count_unsigned, 8);
c_0 := SHIFT_RIGHT(y, to_integer(cast_0));
IF y(15) = '0' THEN
    -- y1 >= 0
    -- rotate the input vector
    x_out_tmp <= x + c_0;
    y_out_tmp <= y - c;
    -- accumulate the phase being rotated
    a_out_tmp <= a + angle_signed;
ELSE
    x_out_tmp <= x - c_0;
    y_out_tmp <= y + c;
    a_out_tmp <= a - angle_signed;
END IF;
```

Chapter 6

Convert DFT Polyphase Filter Bank to Fixed Point

The examples for this chapter are in directory

`exercises/06dftpolyphasefilterbank`



6.1 MATLAB implementation

The following MATLAB function implements the channelizer using the pieces we put together in the preceding chapters. It is made up of an FIR filter bank, an inverse discrete Fourier transform (IDFT), and a bank of CORDIC magnituded-to-phase functions (= cartesian to polar).

```
function [magnitudeY, angleY] = dft_polyphase_filter(B,U,T)
    X = fir_filter_bank(B,U,T.fir_filter);
    Y = inverse_fft(X,T.fft,1);
    [magnitudeY, angleY] = cordic_cart2pol(Y);
end
```

6.2 Test file

Test the channelizer by running test function `test_dft_polyphase_channelizer`.

```
%% Generate inputs
M = 32; % Number of channels in the filter bank.
N = 8; % Number of taps in each FIR filter.
n_out = 4000; % Number of outputs at each output of the filter bank
datatypeselector = 'hdl'; % 'double' 'single' 'hdl' 'dsp'
fidatatype = 'Fixed';
prototypevalue = 1i;
[T, B, B, U, b, u, t, fs] = generate_inputs(M, ...
                                             N, ...
                                             n_out, ...
                                             datatypeselector, ...
                                             fidatatype, ...
                                             prototypevalue);

%% Build and run in target type
build_dft_polyphase_filter(B, U, T);
[absY, angleY] = dft_polyphase_filter_mex(B, U, T);

%% Build and run instrumented in scaled double types
fidatatype = 'ScaledDouble';
[T2, B2, B2, U2] = generate_inputs(M, ...
                                   N, ...
                                   n_out, ...
                                   datatypeselector, ...
                                   fidatatype, ...
                                   prototypevalue);
build_instrumented_dft_polyphase_filter(B2, U2, T2);
%%
[absY2, angleY2] = dft_polyphase_filter_instrumented(B2, U2, T2);
```

```
%% Plot
plot_one_channel_at_a_time(M, fs, absY, 1)
%%
t2 = plot_dft_polyphase_filter_outputs(M, fs, absY, angleY, 1);
%%
plot_dft_polyphase_filter_errors(M, fs, absY, angleY, absY2, angleY2, 2);

%% Visualize datatypes
% Function to focus on. Empty for all functions
% fun = 'fir_filter_circular_buffer';
fun = '';
vis = VisualizeDatatypes( 'dft_polyphase_filter_instrumented', 3, fun);
```

6.3 Types tables

Edit the following files to experiment with different data types.

```
fft_double_types.m
fft_dsp_types.m
fft_hdl_types.m
fft_single_types.m

fir_filter_double_types.m
fir_filter_dsp_types.m
fir_filter_hdl_types.m
fir_filter_single_types.m
```

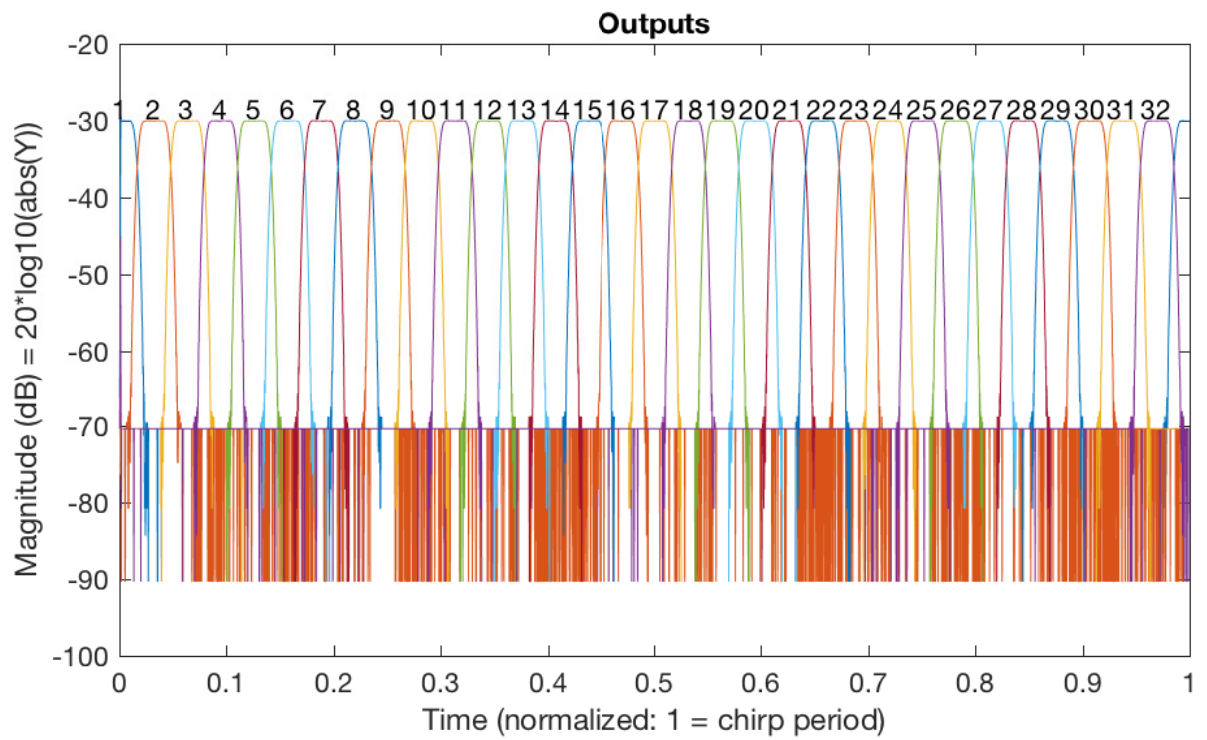


Figure 6.1: OUTPUT OF FILTER BANK WITH COMPLEX CHIRP INPUT

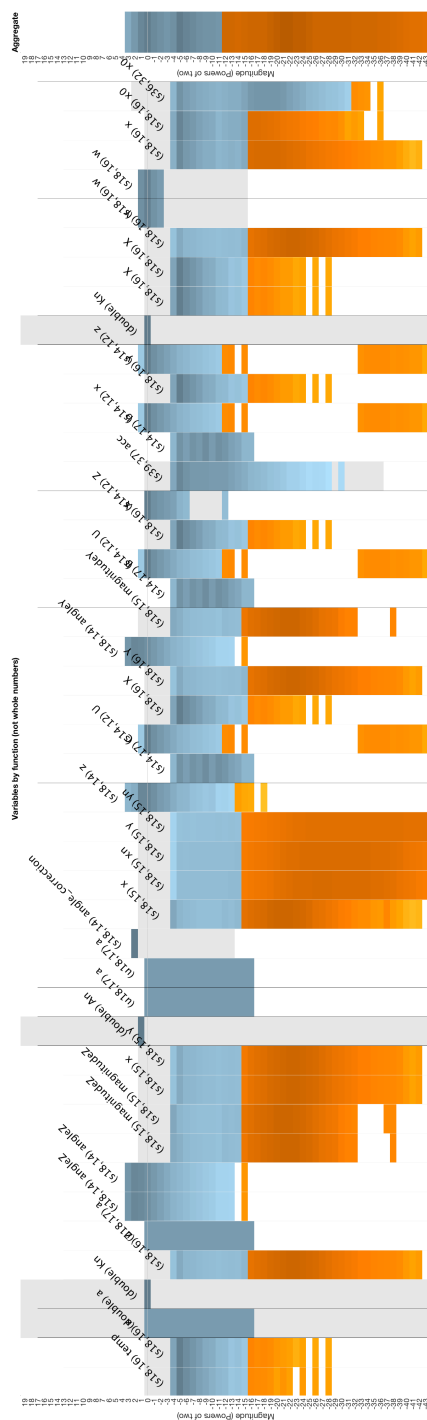


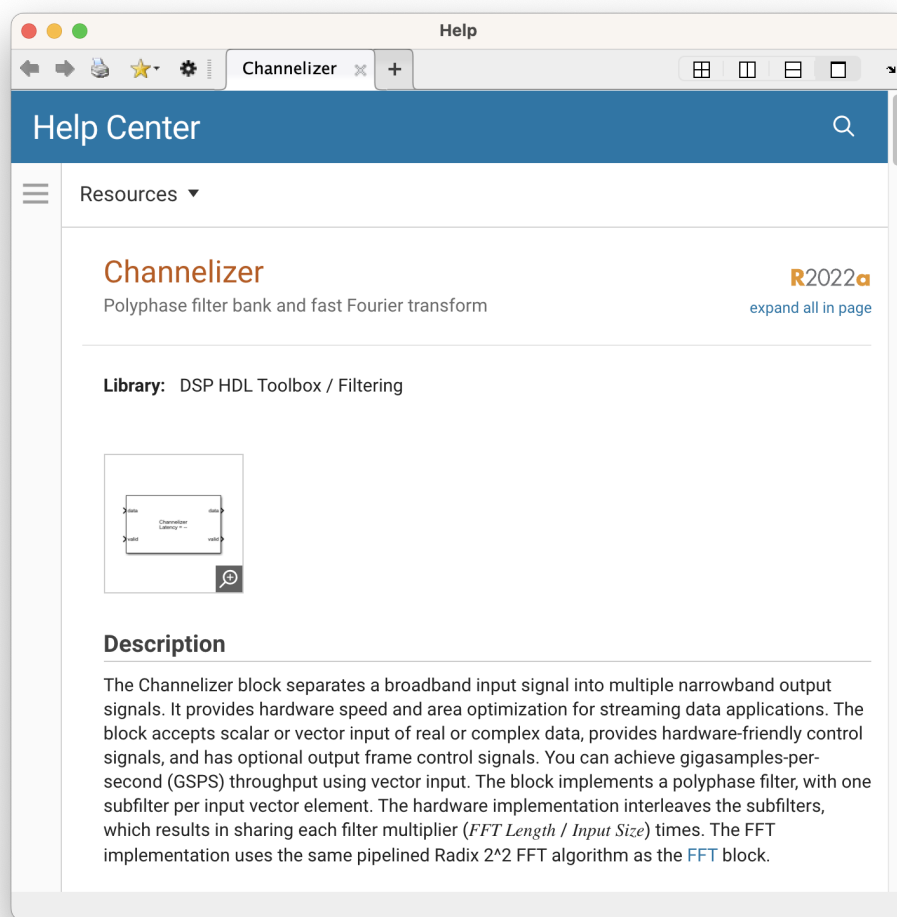
Figure 6.2: DATATYPE VISUALIZATION

6.4 DSP HDL Toolbox Channelizer

DSP HDL Toolbox has an HDL-optimized Channelizer block that does the filtering and DFT portions that we developed from scratch above. The example in the above sections is to show how you would create your own intellectual property if MathWorks did not have what you need.

To read the documentation, run the following in the MATLAB Command Window.

```
web(fullfile(docroot, 'dsphdl/ref/channelizer.html'))
```



Chapter 7

Linear Systems of Equations

Almost anything you can do with A^{-1} can be done without it.

George E. Forsythe and Cleve B. Moler,
Computer Solution of Linear Algebraic Systems, 1967

In the vast majority of practical computational problems, it is unnecessary and inadvisable to actually compute A^{-1} .

George E. Forsythe, Michael A. Malcolm, and Cleve B. Moler,
Computer Methods for Mathematical Computations, 1977, p. 31

The best example of a problem in which the matrix inverse should not be computed is the linear equations problem $Ax = b$.

Nicholas J. Higham,
Accuracy and Stability of Numerical Algorithms, 2002, Chapter 14

You can't always get what you want
But if you try sometimes you'll find
You get what you need

Keith Richards and Mick Jagger,
The Rolling Stones, 1969

The examples for this chapter are in directory

`exercises/07linearequations`

In this chapter, we will investigate fixed-point MATLAB code to solve linear systems of equations. We will also show examples of integrating the fixed-point MATLAB code with Simulink for efficient HDL code generation.

7.1 Notation

We will use the following standard linear-algebra notation.

- Variable A is an m -by- n real or complex matrix with $m \geq n$ (square, or tall-and-skinny). Some disciplines use other variable names, and present the data matrix as short and fat—these formulations are just the transposed version of what we present here.
- A' is the transpose of A if A is real, and the complex-conjugate transpose if A is complex.
- Variable X is an unknown matrix, to be solved for. Often, X is a vector. When X has more than one column, then it is equivalent to solving for individual vectors with matrix $X = [x_1, x_2 \dots, x_p]$.
- Variable B is a known matrix. Often, B is a vector. When B has more than one column, then it is equivalent to solving a system with multiple right-hand sides $B = [b_1, b_2 \dots, b_p]$.
- Scalar λ is a Tikhonov regularization parameter, often called a diagonal-loading parameter.
- Matrix I_n is an n -by- n identity matrix.
- Matrix $0_{n,p}$ is an n -by- p matrix of zeros.
- m is the number of rows in matrix A
- n is the number of columns in matrix A
- p is the number of columns in X and B .

7.2 Linear Equations

We will investigate the following linear equations, commonly found in radar [18], and massive multi-input multi-output (MIMO) beamforming [12]. We will never use matrix inverse. We will avoid computing $A'A$ if possible because it squares the condition number which makes it more sensitive to round-off errors, and it either halves the dynamic range or doubles the fixed-point word length (sometimes the system gives you $A'A$, so you can't avoid it).

$$(A'A)X = B, \text{ used in beamforming} \quad (7.1)$$

$$AX = B, \text{ classical least-squares} \quad (7.2)$$

$$\begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = \begin{bmatrix} 0_{n,p} \\ B \end{bmatrix}, \text{ least-squares with Tikhonov regularization} \quad (7.3)$$

$$\begin{bmatrix} \lambda I_n \\ A \end{bmatrix}' \begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = (\lambda^2 I_n + A'A)X = B, \text{ beamforming with regularization} \quad (7.4)$$

7.3 Datatypes for Linear Systems of Equations

The required number of integer bits to prevent overflow in the solution of $AX = B$ are derived from the following bounds on the growth of $R = Q'A$ and $C = Q'B$ [3], where A is an m -by- n matrix. The required number of integer bits is added to the number of bits of precision of the input, plus one for the sign bit, plus one bit for intermediate CORDIC gain of approximately 1.6468 [24]. The bounds for X are derived in [4, 5].

The elements of $R = Q'A$ are bounded in magnitude by

$$\max(|R(:)|) \leq \sqrt{m} \max(|A(:)|).$$

The elements in $C = Q'B$ are bounded in magnitude by

$$\max(|C(:)|) \leq \sqrt{m} \max(|B(:)|).$$

For the solution of $AX = B$, the elements of X are bounded in magnitude by

$$\max(|X(:)|) \leq \frac{\sqrt{m} \max(|B(:)|)}{\min(\text{svd}(A))}.$$

For the solution of $(A'A)X = B$, the elements of X are bounded in magnitude by

$$\max(|X(:)|) \leq \frac{n \cdot \max(|B(:)|)}{\min(\text{svd}(A))^2}.$$

For the solution of $\begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = \begin{bmatrix} 0_{n,p} \\ B \end{bmatrix}$, the elements of X are bounded in magnitude by

$$\max(|X(:)|) \leq \frac{\sqrt{m} \max(|B(:)|)}{\sqrt{\min(\text{svd}(A))^2 + \lambda^2}}.$$

For the solution of $\begin{bmatrix} \lambda I_n \\ A \end{bmatrix}' \begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = (\lambda^2 I_n + A'A)X = B$ the elements of X are bounded in magnitude by

$$\max(|X(:)|) \leq \frac{n \cdot \max(|B(:)|)}{\min(\text{svd}(A))^2 + \lambda^2}.$$

Computing the singular value decomposition to derive the above bounds on X is more computationally expensive than the entire matrix solve, so functions `fixed.complexSingularValueLowerBound` and `fixed.realSingularValueLowerBound` are used to estimate a bound on $\min(\text{svd}(A))$.

For a complete list of functions to compute types for linear systems of equations, see section “Analytically Determine Fixed-Point Data Types for Linear System Solvers and Matrix Factorizations” on reference page [Fixed-Point Matrix Operations in MATLAB](#).

7.4 QR Decomposition using CORDIC

We will use the QR decomposition to solve each of the linear equations. We use CORDIC to transform complex diagonal elements to real, and to zero out the sub-diagonal elements of the matrix.

The derivation in this section iteratively transforms A to $R = Q'A$, where R is an upper-triangular matrix and, $\text{norm}(Q) = 1$. The fixed-point algorithm we use only produces orthogonal Q (i.e. $Q'Q = I$) when A is nonsingular. The same transformations are applied to B to transform B to $C = Q'B$. Equations 7.1 and 7.6 do not require B to be transformed, so ignore the transformation of B in the derivation for those equations.

Open and step through the following examples.

```
QRZeroSubDiagonalExample.mlx
QRExample.mlx
```

7.4.1 Graphical Representation of CORDIC QR Algorithm

For each row of A in the equation $AX = B$, apply Givens rotations simultaneously to the rows of A and B so that A is transformed to upper-triangular $R = Q'A$ and B is transformed to $C = Q'B$.

In the following sections,

- Q_θ is the Givens transformation that rotates the first element to real.
- Q_ϕ is the Givens transformation that rotates the sub-diagonal element to zero.

Initial values for QR

For the least-squares problem $Ax = b$, the initial values are

$$R_0 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad c_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Initial values for Tikhonov regularization

To compute the least-squares solution to

$$\begin{bmatrix} \lambda I \\ A \end{bmatrix} x = \begin{bmatrix} 0 \\ b \end{bmatrix}$$

initialize R_0 and c_0 to

$$R_0 = \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & \lambda \end{bmatrix} \quad c_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Cell 1 for $n = 3$

A new row at time i enters from the sensor array.

$$\begin{array}{ccc} Q_\theta \left[\begin{array}{ccc|c} a_{i1} & a_{i2} & a_{i3} & b_i \end{array} \right] & \rightarrow & \left[\begin{array}{ccc|c} |a_{i1}| & a_{i2} & a_{i3} & b_i \end{array} \right] \\ & \swarrow & \\ Q_\phi \left[\begin{array}{ccc|c} |r_{11}| & r_{12} & r_{13} & c_1 \\ & \bullet & \bullet & \bullet \\ & & \bullet & \bullet \\ \hline |a_{i1}| & a_{i2} & a_{i3} & b_i \end{array} \right] & \rightarrow & \left[\begin{array}{ccc|c} |r_{11}| & r_{12} & r_{13} & c_1 \\ & \bullet & \bullet & \bullet \\ & & \bullet & \bullet \\ \hline 0 & a_{i2} & a_{i3} & b_i \end{array} \right] \end{array}$$

The processed row is passed to the next cell.

Cell 2 for $n = 3$

The processed row from Cell 1 enters.

$$\begin{array}{ccc} Q_\theta \left[\begin{array}{ccc|c} 0 & a_{(i-1)2} & a_{(i-1)3} & b_{i-1} \end{array} \right] & \rightarrow & \left[\begin{array}{ccc|c} 0 & |a_{(i-1)2}| & a_{(i-1)3} & b_{i-1} \end{array} \right] \\ & \swarrow & \\ Q_\phi \left[\begin{array}{ccc|c} \bullet & \bullet & \bullet & \bullet \\ & |r_{22}| & r_{23} & c_2 \\ & & \bullet & \bullet \\ \hline 0 & |a_{(i-1)2}| & a_{(i-1)3} & b_{i-1} \end{array} \right] & \rightarrow & \left[\begin{array}{ccc|c} \bullet & \bullet & \bullet & \bullet \\ & |r_{22}| & r_{23} & c_2 \\ & & \bullet & \bullet \\ \hline 0 & 0 & a_{(i-1)3} & b_{i-1} \end{array} \right] \end{array}$$

The processed row is passed to the next cell.

Cell 3 for $n = 3$

The processed row from Cell 2 enters.

$$\begin{array}{ccc}
 Q_\theta \quad [\begin{array}{ccc|c} 0 & 0 & a_{(i-2)3} & b_{i-2} \end{array}] & \rightarrow & [\begin{array}{ccc|c} 0 & 0 & |a_{(i-2)3}| & b_{i-2} \end{array}] \\
 & \swarrow & \\
 Q_\phi \quad \left[\begin{array}{ccc|c} \bullet & \bullet & \bullet & \bullet \\ & \bullet & \bullet & \bullet \\ \hline & & |r_{33}| & c_3 \\ \hline 0 & 0 & |a_{(i-2)3}| & b_{i-2} \end{array} \right] & \rightarrow & \left[\begin{array}{ccc|c} \bullet & \bullet & \bullet & \bullet \\ & \bullet & \bullet & \bullet \\ \hline & & |r_{33}| & c_3 \\ \hline 0 & 0 & 0 & b_{i-2} \end{array} \right]
 \end{array}$$

The processed upper triangular R and c are ready to be output.

7.4.2 Rotate first element to real

If matrix A is complex, then the complex pivot elements must be transformed to real values in order to use CORDIC to rotate the pivots to zero. For each pivot, this requires an additional CORDIC rotation.

Given $1 \times n$ complex vector $y = A(k, :)$ and $1 \times p$ complex vector $v = B(k, :)$, this algorithm computes the equivalent of the following Givens transformations using the CORDIC algorithm to zero out the imaginary part of $y(1)$.

$$\begin{bmatrix} y'(1) & \dots & y'(n) & | & v'(1) & \dots & v'(p) \end{bmatrix} = e^{-i\theta} \begin{bmatrix} y(1) & \dots & y(n) & | & v(1) & \dots & v(p) \end{bmatrix}$$

The product $e^{-i\theta}y(1)$ transforms the imaginary part of $y(1)$ to 0 and produces the real value $y'(1) = \text{abs}(y(1))$.

The multiplication by the complex exponential $e^{-i\theta}$ is carried out with Givens rotations implemented using CORDIC.

$$e^{-i\theta}[y|v] = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} \text{real}(y) & | & \text{real}(v) \\ \text{imag}(y) & | & \text{imag}(v) \end{bmatrix}$$

where

$$\theta = \tan^{-1} \left(\frac{\text{imag}(y(1))}{\text{real}(y(1))} \right)$$

and $y'(1) = \text{abs}(y(1))$ is real-valued.

The MATLAB code to rotate the first element to real is the following.

```
function [y,v] = rotateFirstElementToReal(y,v,j,niter,Kn)
    [yr,yi,vr,vi] = fixed.qr.cordicgivens(real(y),imag(y),...
                                         real(v),imag(v),j,niter,Kn);
    y = complex(yr,yi);
    v = complex(vr,vi);
end
```

Function `cordicgivens` is defined by the following MATLAB code. See the derivation of `cordicgivens` in Chapter 5 “Converting CORDIC Algorithm to Fixed Point”, on page 123, and in the following example in MathWorks documentation [Perform QR Factorization Using CORDIC](#). Functions `fixed.fimathLike`, `setfimath`, `removefimath` are used to control the fixed-point math operations to use Floor rounding, Wrap overflow, and to perform addition and multiplication without growing bits.

```
function [x,y,u,v] = cordicgivens(x,y,u,v,j,niter,Kn)
%cordicgivens CORDIC Givens rotations for QR.
% [x,y,u,v] = cordicgivens(x,y,u,v,j,niter,Kn) performs
% Givens rotations to vectors x and y with j as the
% pivot and niter CORDIC iterations. Kn is the inverse
% CORDIC gain factor. The same Givens rotations are
% applied to u and v.
F_xy = fixed.fimathLike(x);
F_uv = fixed.fimathLike(u);
x = setfimath(x,F_xy);
y = setfimath(y,F_xy);
u = setfimath(u,F_uv);
v = setfimath(v,F_uv);
Kn = removefimath(Kn);
if x(j)<0
    % Compensation for 3rd and 4th quadrants
    x(:) = -x;
    y(:) = -y;
    u(:) = -u;
    v(:) = -v;
end
for i=0:niter-1
    x0 = x;
    u0 = u;
    if y(j)<0
        % Counter-clockwise rotation
        u(:) = u - bitsra(v,i);
        v(:) = v + bitsra(u0,i);
        % Rotate over j:end
        for k = j:length(y)
            x(k) = x(k) - bitsra(y(k), i);
            y(k) = y(k) + bitsra(x0(k),i);
        end
    else
        % Clockwise rotation
        u(:) = u + bitsra(v,i);
        v(:) = v - bitsra(u0,i);
        % Rotate over j:end
        for k = j:length(y)
```

```
        x(k) = x(k) + bitsra(y(k), i);
        y(k) = y(k) - bitsra(x0(k), i);
    end
end
end
% Set y(1) to exactly zero so R will be upper triangular
% without roundoff showing up in the lower triangle.
y(j) = 0;
% Normalize the CORDIC gain
x(:) = Kn * x;
y(:) = Kn * y;
u(:) = Kn * u;
v(:) = Kn * v;
x = removefimath(x);
y = removefimath(y);
u = removefimath(u);
v = removefimath(v);
end
```

7.4.3 Rotate Sub-Diagonal Elements to Zero

After rotating the pivot element to real, loop over each of the pairs of rows of A and B to zero-out the sub-diagonal elements of A to form $R = Q'A$ using CORDIC Givens transformations. Apply the same transformation to the elements of B to form $C = Q'B$.

```
function [C,R] = qr_zero_sub_diagonal_example(A,B)
    [niter, Kn] = fixed.cordicConstants(A);
    [m,n] = size(A);
    p = size(B,2);
    C = zeros(n,p,'like',B);
    R = zeros(n,n,'like',A);
    for i = 1:m
        for j = 1:n
            if ~isreal(A(i,:))
                % Rotate first element to real
                [A(i,:),B(i,:)] = ...
                    fixed.qr.rotateFirstElementToReal(...
                        A(i,:),B(i,:),j,niter,Kn);
            end
            % Rotate sub-diagonal element of A to zero.
            % Apply the same transformation to B.
            [R(j,:),A(i,:),C(j,:),B(i,:)] = ...
                fixed.qr.cordicgivens(...
                    R(j,:),A(i,:),C(j,:),B(i,:),j,niter,Kn);
        end
    end
end
```

7.5 Solve $(A'A)X = B$ given A

Matrix equation $(A'A)X = B$ can be solved by computing the Cholesky factorization of $A'A$ and forward- and backward-substituting the Cholesky factor into B .

Open and step through the following examples.

```
CholeskyAndQR.mlx
QlessQRExample.mlx
QlessQRMatrixSolveExample.mlx
```

The Cholesky factorization of $A'A$ is an upper- or lower-triangular matrix R with positive diagonal elements such that $R'R = A'A$. By default, the MATLAB `chol` function returns an upper-triangular matrix. For example,

```
rng('default')
A = randn(10,3);
R = chol(A'*A)

R =
    5.6648    2.3256   -0.8496
         0    3.5967   -0.9131
         0         0    2.4822
```

If you are given matrix A directly without computing the covariance matrix $A'A$, for example as rows of readings from a sensor array, then you can compute the Cholesky factorization of A using the QR decomposition. This is more accurate than computing the Cholesky factorization from $A'A$. To see why this works, let $QR = A$ be the economy-size QR decomposition of A . Then

$$A'A = R'Q'QR = R'R.$$

To solve $(A'A)X = B$, you don't need Q and you don't need to apply transformations to B , so you can use function `qlessQR`, which always returns non-negative diagonal elements of R . For example, compute the upper-triangular QR factor of A and the Cholesky factorization of $A'A$, and you can see that they are identical.

```
>> rng('default')
>> A = randn(10,3);
>> R = qlessQR(A)

R =
    5.6648    2.3256   -0.8496
         0    3.5967   -0.9131
         0         0    2.4822
```

```
>> R_chol = chol(A'*A)

R_chol =
    5.6648    2.3256   -0.8496
         0    3.5967   -0.9131
         0         0    2.4822
```

Function `qllessQR` works with fixed-point inputs and supports C code generation.

Once you have R , then you can solve the matrix equation using forward-and backward substitution into B . For example,

```
>> rng('default')
>> A = randn(10,3);
>> B = randn(3,1);
>> R = qllessQR(A);
>> C = R' \ B; % Forward substitution
>> X = R \ C   % Backward substitution

X =
    0.0659
   -0.1712
   -0.2141
```

Compare to solving $(A'A)X = B$ directly in MATLAB, and you can see that the answers are identical.

```
>> rng('default')
>> A = randn(10,3);
>> B = randn(3,1);
>> X = (A'*A) \ B

X =
    0.0659
   -0.1712
   -0.2141
```

Computing the Q-less QR decomposition coupled with forward- and backward substitution is encapsulated in function `qllessQRMatrixSolve`. For example,

```
>> rng('default')
>> A = randn(10,3);
>> B = randn(3,1);
>> X = qllessQRMatrixSolve(A,B)

X =
    0.0659
   -0.1712
   -0.2141
```

The QR decomposition is always defined, and the relationship $R'R = A'A$ is always true (within roundoff) where R is the upper-triangular QR factor of A . For example,

```
>> A = ones(10,3);
>> R = qllessQR(A)
R =

    3.1623    3.1623    3.1623
         0    0.0000    0.0000
         0         0    0.0000

>> R' * R

ans =

   10.0000   10.0000   10.0000
   10.0000   10.0000   10.0000
   10.0000   10.0000   10.0000

>> A' * A

ans =

    10    10    10
    10    10    10
    10    10    10
```

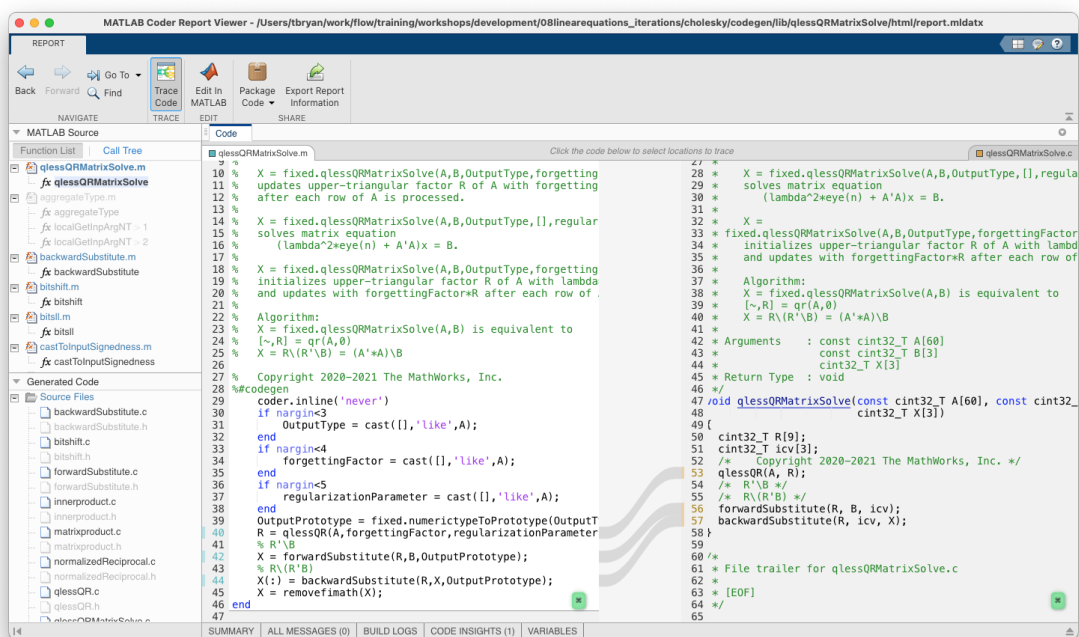
However, the Cholesky factorization of a matrix is only defined when it is symmetric and has all positive eigenvalues, which is the definition of a positive-definite matrix. For example, the following matrix ($A'A$) is symmetric, but is not full rank and so has eigenvalues that are zero.

```
>> A = ones(10,3);  
>> chol(A'*A)  
  
Error using chol  
Matrix must be positive definite.
```

7.5.1 Fixed-Point Code Generation

Run the following MATLAB script in this directory to see the results of datatype visualization and code generation with fixed-point types.

```
QlessQRExample.mlx
QlessQRMatrixSolveExample.mlx
```



7.5.2 Minimum-Variance Distortionless Response (MVDR) Beamformer

The Minimum-Variance Distortionless Response (MVDR) Beamformer example on the following reference page uses the method described in this section. You can generate efficient HDL from the Simulink implementation. The CORDIC math is implemented in MATLAB Function blocks. Parallelization, timing, latency, and pipelining are implemented in the surrounding Simulink diagram.

<https://www.mathworks.com/help/fixedpoint/ug/fixed-point-mvdr-beamformer.html>

The MVDR beamformer is computed by the following equations. Compute the MVDR beamformer response by solving the following equation for x , where b is a steering vector pointing in the direction of the desired signal.

$$(A^H A)x = b$$

The MVDR weight vector w is computed from x and b using the following equation, which normalizes x to preserve the gain in the direction of arrival of the desired signal.

$$w = \frac{x}{b^H x}$$

The MVDR system response is the inner product between the MVDR weight vector w and a current sample from the sensor array $a(t)$.

$$y = w^H a(t)$$

You can read about this example by navigating to it from MATLAB documentation running the following in the MATLAB command line.

```
web(fullfile(docroot, 'fixedpoint/ug/fixed-point-mvdr-beamformer.html'))
```

You can open this example by running the following from the MATLAB command line.

```
openExample('fixedpoint/MVDRBeamformerHDLOptimizedExample')
```

Fixed-Point HDL-Optimized Minimum-Variance Distortionless-Response (MVDR) Beamformer

Documentation Search R2021b Documentation

All Examples Functions Blocks Apps Trial Software Product Updates

Fixed-Point HDL-Optimized Minimum-Variance Distortionless-Response (MVDR) Beamformer R2021b

This example shows how to implement a fixed-point HDL-optimized minimum-variance distortionless-response (MVDR) beamformer, also known as Capon's method [5] and robust adaptive beamforming (RAB) [6]. For more information on beamformers, see [Conventional and Adaptive Beamformers](#) (Phased Array System Toolbox).

MVDR Objective

The MVDR beamformer preserves the gain in the direction of arrival of a desired signal and attenuates interference from other directions [1], [2].

Given readings from a sensor array, such as the uniform linear array (ULA) in the following diagram, form data matrix A from samples of the array, where $a(t)$ is an n -by-1 column vector of readings from the array sampled at time t , and $a(t)^H$ is one row of matrix A . Many more samples are taken than there are elements in the array. This results in the number of rows in A being much greater than the number of columns. An estimate of the covariance matrix is $A^H A$, where A^H is the Hermitian or complex-conjugate transpose of A .

This example uses:
[Phased Array System Toolbox](#)
[Fixed-Point Designer](#)
[Simulink](#)

[Open Script](#)

Compute the MVDR beamformer response by solving the following equation for x , where b is a steering vector pointing in the direction of the desired signal.

$$(A^H A)x = b$$

The MVDR weight vector w is computed from x and b using the following equation, which normalizes x to preserve the gain in the direction of arrival of the desired signal.

$$w = \frac{x}{b^H x}$$

The MVDR system response is the inner product between the MVDR weight vector w and a current sample from the sensor array $a(t)$.

7.6 Solve $(A'A)X = B$ given $(A'A)$

If your system gives you the covariance matrix $A'A$ already formed, then you can solve the system of equations $(A'A)X = B$ by computing the QR decomposition of $Y = A'A$ to obtain upper-triangular $R = Q'Y$ and applying the same transformations to B to obtain $C = Q'B$. Then you can compute X by back-substituting R into C . This operation has been encapsulated in function `qrMatrixSolve`. For example

```
>> rng('default')
>> A = randn(10,3);
>> Y = A'*A;
>> B = randn(3,1);
>> X = qrMatrixSolve(Y,B)

X =
    0.0659
   -0.1712
   -0.2141
```

And you can see that you get the same answer as before.

7.6.1 Fixed-Point Code Generation

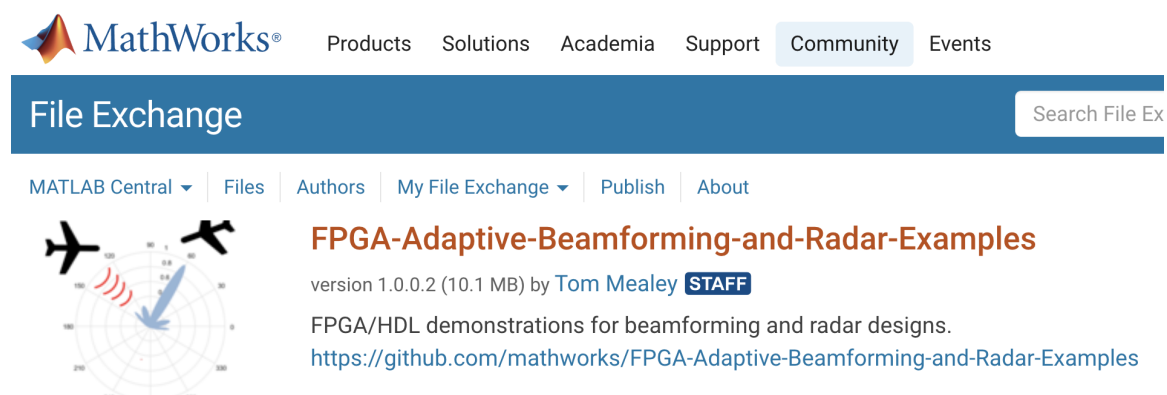
Run the following MATLAB script in this directory to see the results of datatype visualization and code generation with fixed-point types.

```
SymmetricQRMatrixSolveExample.mlx
```

7.6.2 MVDR Beamformer Solving $(A'A)X = B$ given $(A'A)$

The MVDR beamformer defined in section 7.5.2 is implemented on Xilinx's Zynq(R) UltraScale+(TM) Radio Frequency System-on-Chip (RFSoc) ZCU111 and can be downloaded from the following pages.

- <https://www.mathworks.com/matlabcentral/fileexchange/89619-fpga-adaptive-beamforming-and-radar-examples>
- <https://github.com/mathworks/FPGA-Adaptive-Beamforming-and-Radar-Examples>



If you download the github repository, you can run the model by navigating to the following directory in the repository.

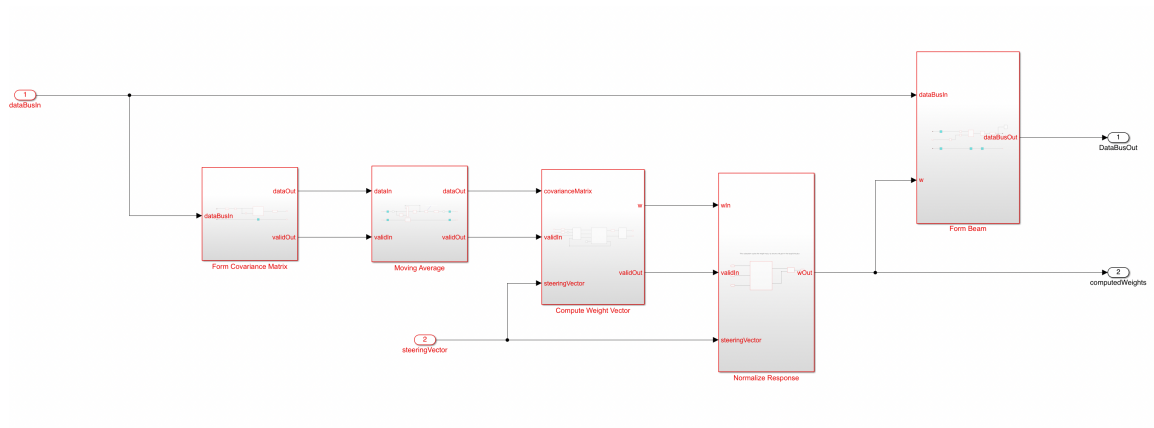
```
mvdr_demo/zcu111_mvdr_demo
```

Open the model.

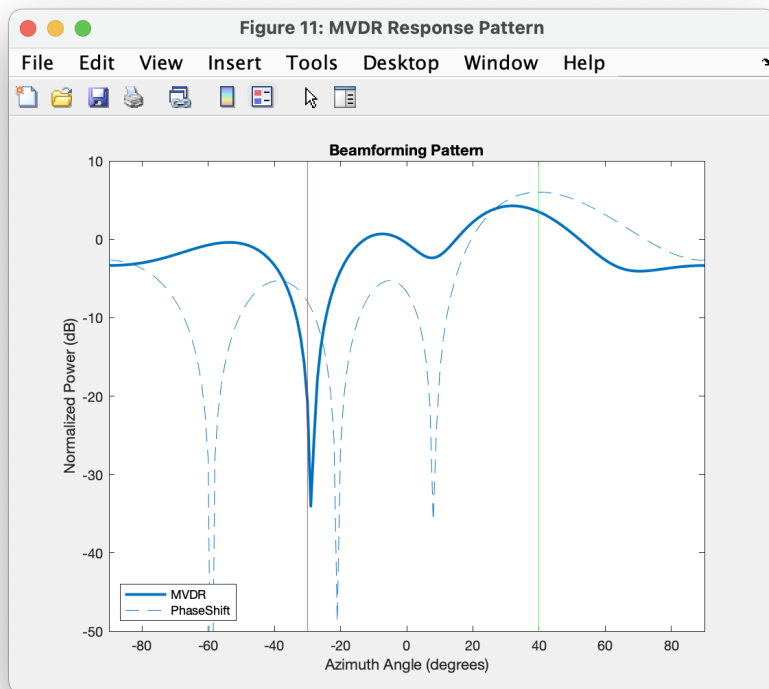
```
TxSteering_RxMVDR_4x4_HDL_IQ.slx
```

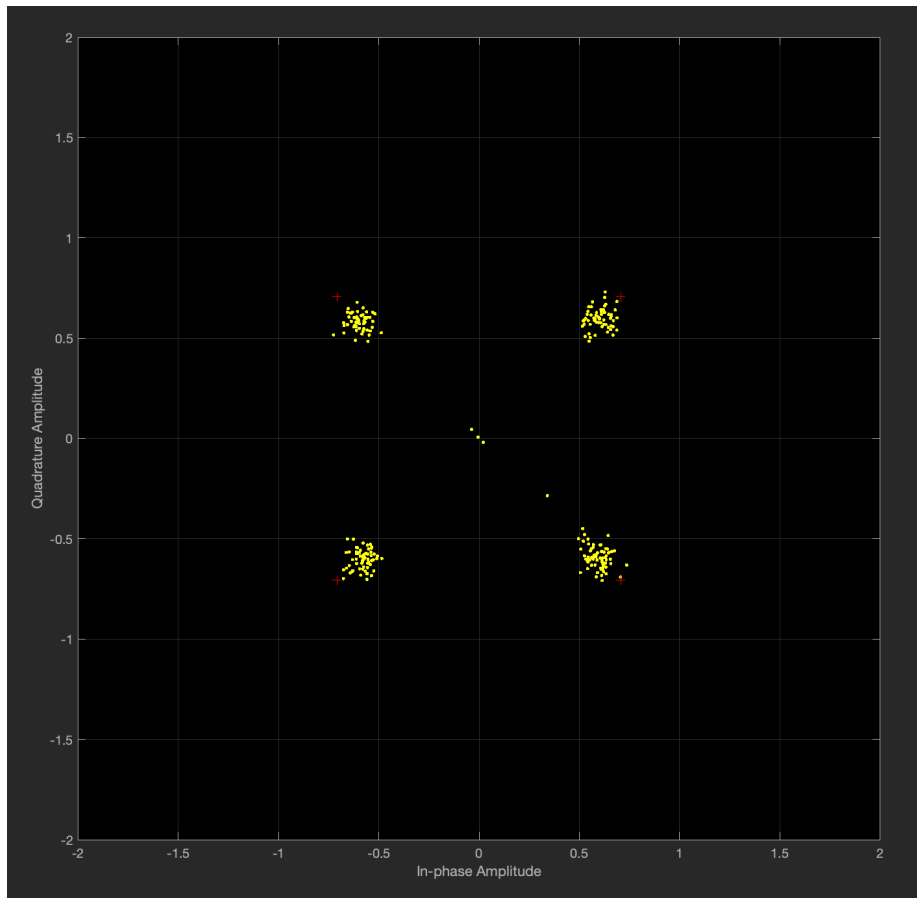
Navigate to the MVDR subsystem by running the following from the MATLAB Command line.

```
open_system('TxSteering_RxMVDR_4x4_HDL_IQ/DUT/Rx Beamforming/MVDR')
```



Run the model to see the beam pattern and constellation evolve over time.





7.7 Solve $AX = B$, Classical Least-Squares

The so-called method of normal equations to solve $AX = B$ is

$$X = (A'A)^{-1}A'B.$$

This has appeal because you can write it in closed-form, and you often see it written this way. For this reason, we often get requests to support fixed-point matrix inverse. However, using the inverse is less efficient and produces worse results than the following method using QR decomposition.

To solve $AX = B$, compute the QR decomposition of A to obtain $R = Q'A$ and apply the same transformations to B to obtain $C = Q'B$.

This operation has been encapsulated in function `qrMatrixSolve`. For example

```
>> rng('default')
>> [A,B] = fixed.example.realRandomLeastSquaresMatrices(10,3,1);
>> X = qrMatrixSolve(A,B)

X =
   -0.5855
    1.1440
    0.3084
```

where function `fixed.example.realRandomLeastSquaresMatrices` produces B in the range of A . Compare to builtin MATLAB and you can see that the results are the same.

```
>> X_builtin = A \ B

X_builtin =
   -0.5855
    1.1440
    0.3084
```

7.7.1 Fixed-Point Code Generation

Run the following MATLAB script in this directory to see the results of datatype visualization and code generation with fixed-point types.

```
QRMatrixSolveExample.mlx
```

7.8 Tikhonov Regularization

The following Tikhonov regularization equations are solved by appending λI_n and solving the equations as in the previous sections.

$$\begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = \begin{bmatrix} 0_{n,p} \\ B \end{bmatrix}, \text{ least-squares with Tikhonov regularization} \quad (7.5)$$

$$\begin{bmatrix} \lambda I_n \\ A \end{bmatrix}' \begin{bmatrix} \lambda I_n \\ A \end{bmatrix} X = (\lambda^2 I_n + A' A) X = B, \text{ beamforming with regularization} \quad (7.6)$$

Each of the MATLAB functions and Simulink blocks has an optional “Regularization Parameter”. When it is non-zero, then the regularized solution is produced. The regularized solution is biased, but has the advantage of making the problem well-conditioned (i.e. you won’t divide by very small numbers).

7.9 Combine MATLAB and Simulink for efficient HDL

Creating efficient hardware requires specifying pipelining, parallelization, and resource-sharing that is easier to do in Simulink than in MATLAB alone. The following examples show how to combine MATLAB for the math and Simulink for these elaborations.

7.9.1 HDL-Optimized Linear Algebra Library

You can see our HDL-optimized linear algebra library in the following reference pages.

<https://www.mathworks.com/help/fixedpoint/fixed-point-matrix-operations-in-simulink.html>

Or by executing the following in the MATLAB Command line.

```
web(fullfile(docroot, 'fixedpoint/fixed-point-matrix-operations-in-simulink.html'))
```

7.9.2 HDL-Optimized Linear Algebra Examples

Complex matrix solve with systolic array.

```
openExample('fixedpoint/HardwareEfficientSystemsOfComplexValuedLinearEquationsExample')
```

Partial-systolic complex QR decomposition

```
openExample('fixedpoint/ComplexPartialSystolicQRExample')
```

Resource-shared complex QR decomposition.

```
openExample('fixedpoint/ComplexBurstQRExample')
```

Real QR with systolic array.

```
openExample('fixedpoint/QRDecompositionUsingCORDICInASystolicArrayExample')
```

Partial-systolic real QR decomposition

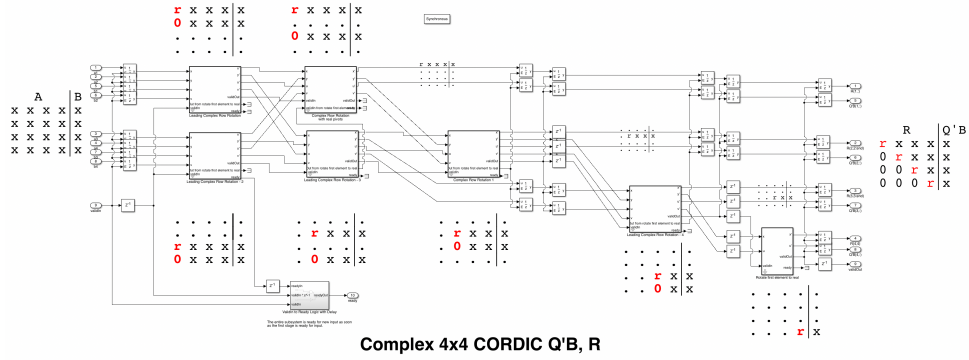
```
openExample('fixedpoint/RealPartialSystolicQRExample')
```

Resource-shared real QR decomposition.

```
openExample('fixedpoint/RealBurstQRExample')
```

7.9.3 Implementation Comparisons

The following diagrams compare the resources used by each of the implementations.



<i>Method</i>	<i>Input</i>	<i>Throughput</i>	<i>Latency</i>	<i>Area</i>	<i>Introduced in</i>
Systolic	Matrix	High	$\mathcal{O}(n \log_2(m))$	$\mathcal{O}(mn^2)$	R2019a
Partial-Systolic	Row	Medium	$\mathcal{O}(mn)$	$\mathcal{O}(n^2)$	R2020b
Burst	Row	Medium	$\mathcal{O}(mn)$	$\mathcal{O}(n)$	R2020a

Chapter 8

Euler to North-East-Down (NED) Coordinate Transformation

The examples for this chapter are in directory

```
exercises/08euler_transform
```

We use the CORDIC technique to compute the Euler to North-East-Down (NED) transformation without explicitly performing any multiplications, or calculating any trigonometric functions directly. The NED transformation is used, for example, in aerospace coordinate systems [10].

Arbitrary rotational transformations can be expensive for embedded applications. They often require the calculation of the basic trigonometric functions on the fly, which must then be multiplied in order to form up the full coordinate transformation of interest. Finally, a matrix multiplication must be carried out on input data to get the desired quantity. This is costly, and can lead to prohibitively large designs that may not fit on a target FPGA.

Here, we use the CORDIC technique to compute the Euler to North-East-Down (NED) transformation without explicitly performing any multiplications, or calculating any trigonometric functions directly. This technique can be coded in an HDL style that allows for deployment entirely in the FPGA fabric, and does not consume any DSP slices. Moreover, it is flexible enough that it can be programmed as either a pipelined module, or a shared computational unit, depending on design constraints.

8.1 The Euler-NED Transformation

The Euler-NED transformation is carried out by the successive application of the three rotation matrices below

$$R_\phi = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix}$$

$$R_\theta = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

$$R_\psi = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiplying these matrices together gives the total transformation given below:

$$\begin{aligned} A &= R_\psi R_\theta R_\phi \\ &= \begin{bmatrix} \cos(\psi) \cos(\theta) & \cos(\psi) \sin(\phi) \sin(\theta) - \cos(\phi) \sin(\psi) & \sin(\phi) \sin(\psi) + \cos(\phi) \cos(\psi) \sin(\theta) \\ \cos(\theta) \sin(\psi) & \cos(\phi) \cos(\psi) + \sin(\phi) \sin(\psi) \sin(\theta) & \cos(\phi) \sin(\psi) \sin(\theta) - \cos(\psi) \sin(\phi) \\ -\sin(\theta) & \cos(\theta) \sin(\phi) & \cos(\phi) \cos(\theta) \end{bmatrix} \end{aligned}$$

Thus, the transformation between two frames related by these angles can be applied by multiplying a vector in an initial frame by the matrix above. However, this formulation can become large in FPGA designs. This involves not only calculating the explicit transformation matrix for all desired transformation angles, but also requires multiplication with the input vector afterward. This approach is shown in figure [8.1](#).

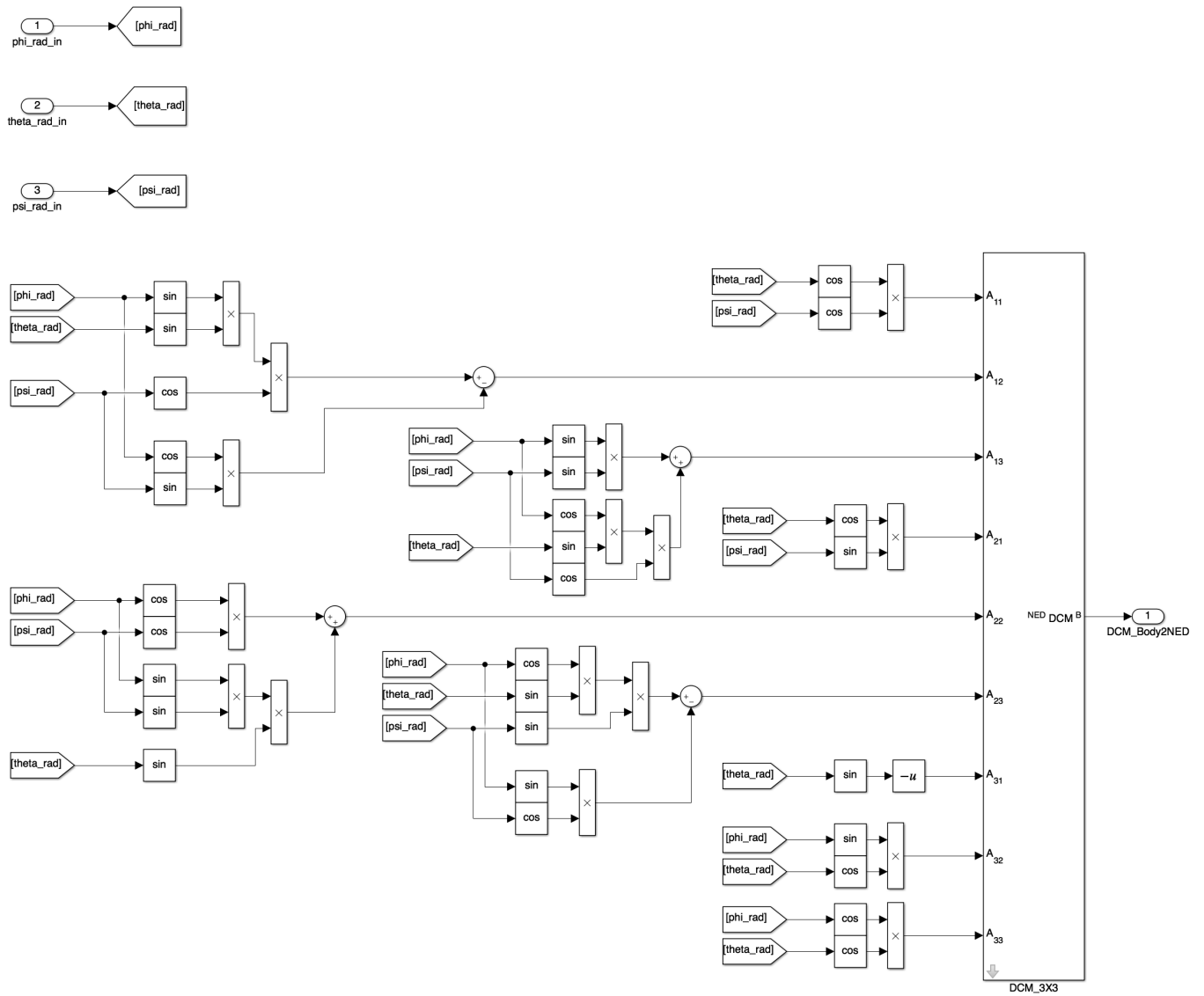


Figure 8.1: SIMULINK[®] MODEL THAT EXPLICITLY FORMS THE EULER-NED TRANSFORMATION MATRIX

One suitable alternative is to use the CORDIC technique to apply the transformations. This technique applies each rotation in succession without explicitly calculating the trigonometric functions, or perform any multiplications.

8.2 The CORDIC Technique

The CORDIC (COordinate Rotation Digital Computer) technique was developed by Jack Volder when working on digital electronics for the B-58 bomber [23]. Though originally used to compute sine, cosine, and tangent [24], the technique has been extended to cover both linear and hyperbolic functions in a unified framework [26]. Using the rotation mode employed by Volder to calculate sine and cosine, it is possible to perform arbitrary planar rotations without computing the rotation matrix explicitly, or performing any multiplications.

There are three central observations from which the CORDIC algorithm can be constructed. The first is that a planar rotation can be constructed by consecutive rotations of constituent angles. For example, a two dimensional Givens rotations is described as

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (8.1)$$

Multiplying this matrix by non-zero 2 x 1 vector rotates the vector by θ in the counterclockwise direction. Note that this rotation can also be described as a rotation of the x-y axes by θ in the clockwise direction. Note that, given angles θ_1 and θ_2 , $R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$, which can be verified using equation 8.1.

The second observation necessary to construct the CORDIC algorithm is that any angle between approximately -99° and 99° can be written using a series

$$\theta = \sum_{i=0}^{\infty} \sigma_i \alpha_i \quad (8.2)$$

such that $\sigma_i = \pm 1$ and $\alpha_i = \tan^{-1} 2^{-i}$. The third observation necessary to construct CORDIC is that the rotation matrices for the angles α_i have a special form that require no multiplications, up to a fixed scale factor. Plugging α_i into 8.1 gives

$$\begin{aligned} R(\alpha_i) &= \begin{bmatrix} \cos(\alpha_i) & -\sin(\alpha_i) \\ \sin(\alpha_i) & \cos(\alpha_i) \end{bmatrix} \\ &= \cos(\alpha_i) \times \begin{bmatrix} 1 & -\tan(\alpha_i) \\ \tan(\alpha_i) & 1 \end{bmatrix} \\ &= \frac{1}{\sqrt{1 + 2^{-2i}}} \begin{bmatrix} 1 & -2^{-i} \\ 2^i & 1 \end{bmatrix}. \end{aligned} \quad (8.3)$$

Note that ignoring the scale factor $1/\sqrt{1 + 2^{-2i}}$ equation 8.3 allows us to compute pseudorotations by the angles α_i using only addition and bitshifts. The overall scale factor can be computed after

all rotations have been performed. Since the scale factor is independent of the direction of rotation, the overall scaling can be applied after all rotations have been performed. This multiplication can be optimized using the canonical signed digit representation, which allows it to be computed entirely with shifts and adds [30].

To use CORDIC to solve the Euler-NED transformation, we apply CORDIC Givens rotations sequentially in the appropriate subspaces of the initial space. Here, we first rotate by ϕ in the yz plane, then by $-\theta$ in the xz plane, and finally ψ in the xy plane, all using CORDIC. This can be optimized to either share computational resources for a minimal FPGA footprint, or can be pipelined for maximum throughput. This approach is shown by the model in figure 8.2.

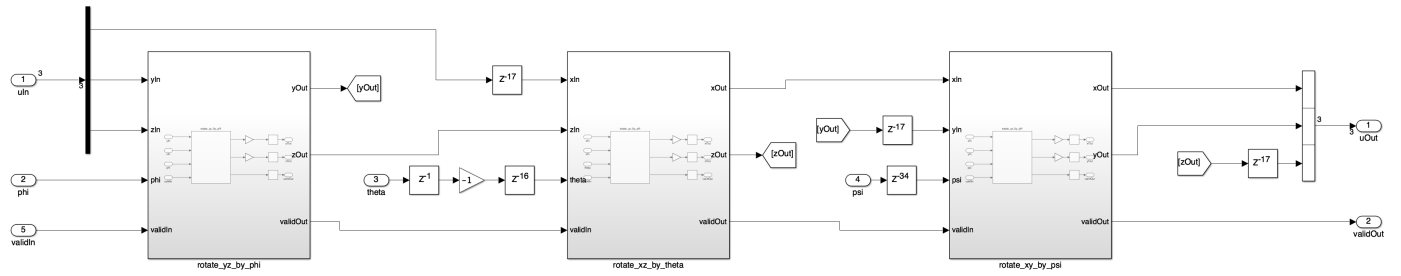


Figure 8.2: SIMULINK[®] MODEL THAT CALCULATES THE EULER-NED TRANSFORMATION USING CORDIC.

8.3 Benchmarks

All benchmarks were done on a Virtex 7 Part. Synthesis is done out of context to ignore I/O Constraints

8.3.1 Resource Usage

Algorithm	Flip Flops	LUT	LUTRAM	DSPs
Pipelined CORDIC (sfix14En10)	3141	86	3973	0
Resource Shared CORDIC (sfix14En10)	337	659	0	0
Multiplication with fxpopt (sfix18En14)	219	304	0	18

8.3.2 Static Timing Analysis

Algorithm	Clock Frequency (MHz)	Latency (Cycles)	Latency (ns)
Pipelined CORDIC (sfix14En10)	347	63	181
Resource Shared CORDIC (sfix14En10)	347	57	164
Multiplication with fxpopt (sfix18En14) ¹	192	5	26

¹These estimates are obtained using some pipelining in the original model. Actual results may vary slightly

Chapter 9

Oblique Orthographic Projection Using CORDIC

The examples for this chapter are in directory

`exercises/09oblique_orthographic_projection`

9.1 The Oblique Orthographic Projection

The Oblique Orthographic Projection is a geometric transformation that maps a half-sphere onto the equatorial plane of the sphere that is used, for example, in Landsat and other near-polar-orbiting satellites [22]. The mapping can be further generalized so that the projection can be carried out onto any plane that intersects the center of the sphere. This requires choosing a reference latitude and longitude, which are denoted as ϕ_0 and λ_0 respectively. These angles specify the normal to the plane, as well as the hemisphere projected onto the plane. This generalization is called the oblique orthographic projection [22, p. 145]. A schematic of the orthographic projection is shown in figure 9.1.

The full oblique orthographic projection is given by equation 9.1. The latitude and longitude ϕ and λ are transformed into map coordinates by a series of four rotations. First, the three dimensional vector corresponding to the north pole is rotated by $90^\circ - \phi$ about the X Axis. Then, the result is rotated by λ degrees about the north pole (the Z-axis). This gives the three dimensional coordinates of the point on the sphere that we wish to project. Thus, to compute the orthographic projection, all that remains is to rotate the axes so that the reference point is aligned along the Z-axis. To do this, we rotate about the Z-axis by $-\lambda_0$, and finally rotate about the X axis by $90^\circ - \phi_0$. The map coordinates are then obtained by taking the X and Y elements of the resultant vector, and multiplying each by the radius of the earth.

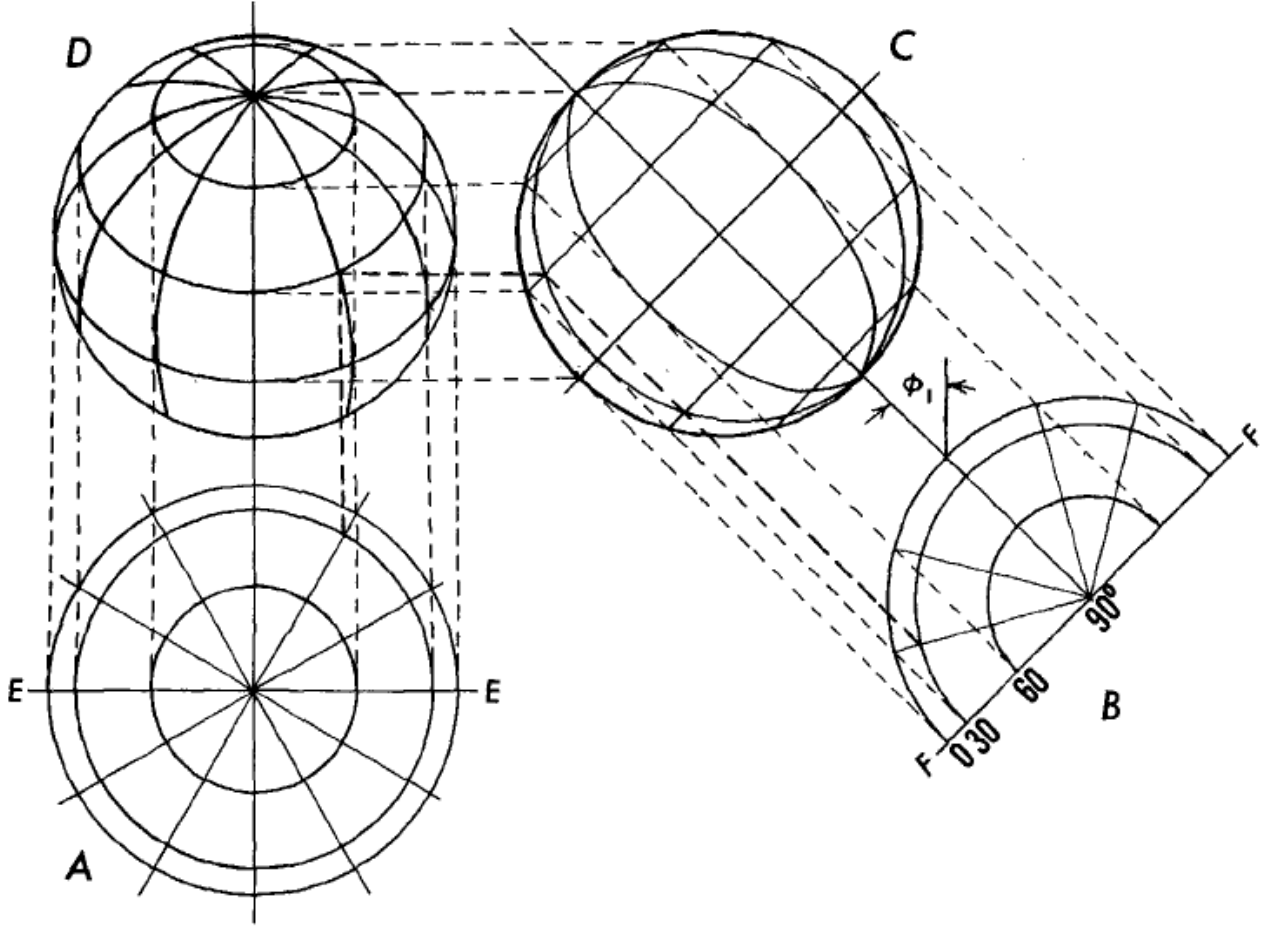


Figure 9.1: SCHEMATIC OF THE ORTHOGRAPHIC PROJECTION[22, P. 148].

$$\begin{aligned}
 \begin{bmatrix} X \\ Y \\ H \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\pi/2 - \phi_0) & \sin(\pi/2 - \phi_0) \\ 0 & -\sin(\pi/2 - \phi_0) & \cos(\pi/2 - \phi_0) \end{bmatrix} \times \\
 &\quad \begin{bmatrix} \cos(\lambda_0) & \sin(\lambda_0) & 0 \\ -\sin(\lambda_0) & \cos(\lambda_0) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \\
 &\quad \begin{bmatrix} \cos(\lambda) & -\sin(\lambda) & 0 \\ \sin(\lambda) & \cos(\lambda) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \\
 &\quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\pi/2 - \phi) & \sin(\pi/2 - \phi) \\ 0 & -\sin(\pi/2 - \phi) & \cos(\pi/2 - \phi) \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}
 \end{aligned} \tag{9.1}$$

Each individual transformation in equation 9.1 is efficiently computable using the CORDIC technique. This technique allows the rotations to be performed using a series of shift and add operations, while neither explicitly computing the trigonometric functions necessary to form the rotation matrices, synnor explicitly multiplying. Hardware implementations of CORDIC are efficient in terms of both resource usage and maximum frequency. It is possible to synthesize CORDIC elements that operate at over 300 MHz using only fabric logic (LUTs and flip flops).

9.2 The CORDIC Technique

The CORDIC (COordinate Rotation Digital Computer) technique was developed by Jack Volder when working on digital electronics for the B-58 bomber [23]. Though originally used to compute sine, cosine, and tangent [24], the technique has been extended to cover both linear and hyperbolic functions in a unified framework [26]. Using the rotation mode employed by Volder to calculate sine and cosine, it is possible to perform arbitrary planar rotations without computing the rotation matrix explicitly, or performing any multiplications.

There are three central observations from which the CORDIC algorithm can be constructed. The first is that a planar rotation can be constructed by consecutitive rotations of constituent angles. For example, a two dimensional Givens rotations is described as

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (9.2)$$

Multiplying this matrix by non-zero 2 x 1 vector rotates the vector by θ in the counterclockwise direction. Note that this rotation can also be described as a rotation of the x-y axes by θ in the clockwise direction. Note that, given angles θ_1 and θ_2 , $R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$, which can be verified using equation 9.2.

The second observation necessary to construct the CORDIC algorithm is that any angle between approximately -99° and 99° can be written using a series

$$\theta = \sum_{i=0}^{\infty} \sigma_i \alpha_i \quad (9.3)$$

such that $\sigma_i = \pm 1$ and $\alpha_i = \tan^{-1} 2^{-i}$. The third observation necessary to construct CORDIC is that the rotation matrices for the angles α_i have a special form that require no multiplications, up to a fixed scale factor. Plugging α_i into 9.2 gives

$$\begin{aligned} R(\alpha_i) &= \begin{bmatrix} \cos(\alpha_i) & -\sin(\alpha_i) \\ \sin(\alpha_i) & \cos(\alpha_i) \end{bmatrix} \\ &= \cos(\alpha_i) \times \begin{bmatrix} 1 & -\tan(\alpha_i) \\ \tan(\alpha_i) & 1 \end{bmatrix} \\ &= \frac{1}{\sqrt{1+2^{-2i}}} \begin{bmatrix} 1 & -2^{-i} \\ 2^i & 1 \end{bmatrix}. \end{aligned} \quad (9.4)$$

Note that ignoring the scale factor $1/\sqrt{1+2^{-2i}}$ equation 9.4 allows us to compute pseudorotations by the angles α_i using only addition and bitshifts. The overall scale factor can be computed after all rotations have been performed. Since the scale factor is independent of the direction of rotation, the overall scaling can be applied after all rotations have been performed. This multiplication can be optimized using the canonical signed digit representation [30].

9.3 Designing the Oblique Orthogonal Transformation

As we briefly discussed in section 9.1, the CORDIC method allows us to perform the oblique orthogonal projection in a hardware efficient manner. This stems from the fact that we can decompose the rotations involved in the total transformation into a series of rotations in single planes. While a fully pipelined approach is possible, maximal resource efficiency requires reusing the most expensive pieces of the design to perform time multiplexed computations. While this does cost resources in scheduling logic, the savings are far greater due to the need for fewer computational cores. This section will briefly detail the architectural design decisions for a maximally efficient model.

9.3.1 The Valid-Ready Interface

The HDL Efficient Orthographic Projection block is shown in figure 9.2. The inputs `phi`, `phi0`, `lambda`, and `lambda0` are each defined in the same manner as the corresponding variable in equation 9.1. The input `validIn` is used to signal that the current input to the block is valid, and should be used for the computation. As the block is currently programmed, inputs are ignored if the block's internal `ready` register is not set. However, it may be possible to provide alternatives in future iterations.

The outputs `X_Out`, `Y_Out`, and `H_Out` correspond to the X , Y , and H in equation 9.1. The `validOut` port signals that the data at these ports is a valid computational result. Finally, `readyOut` indicates that the block is ready to receive inputs. If the data is sent on this signal, samples should not be dropped.

9.3.2 Architecture

The computation is computed by two pieces. The memory controller `memctrl` is a Mealey state machine that schedules Givens rotations via CORDIC. This block stores data, and communicates with the other major block, the CORDIC `Rotation`. The architecture of the block is shown in figure 9.3.

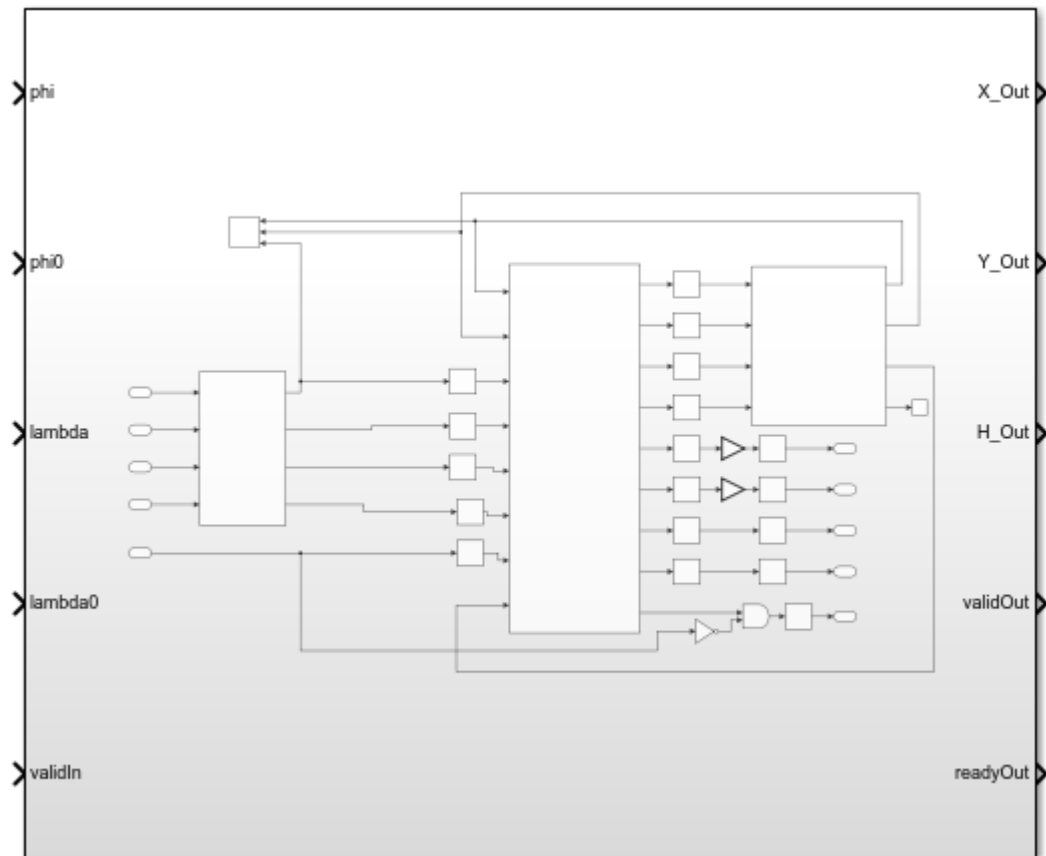


Figure 9.2: THE ORTHOGRAPHIC PROJECTION BLOCK.

9.3.3 Choosing Datatypes

Simulink_Euler_NED

All internal operations in the oblique orthogonal projection cast their results to the input datatype. Thus, it is necessary that the fixed-point datatype has a sufficient fraction length to get the needed accuracy, but also has enough headroom such that no overflows occur internally. The CORDIC correction factor is the only source of internal growth. This factor approaches approximately 1.64676, so providing an additional bit of growth for the initial data is sufficient. This means that, if you are manipulating data initially on the unit sphere, it should have at least 2 integer bits. A good rule of thumb for your normalized dataset (that is, X and Y are given in a fraction of the earth's radius) is that the datatype should be signed with the fraction length 3 bits fewer than the wordlength. This should be scaled accordingly as the data grows in magnitude.

9.4 Resource Usage

The resource usage for this model is given in the table 9.4. It is clear that the CORDIC approach is more resource efficient than the floating point calculation. This benchmark was synthesized for a Xilinx Virtex 7 board with a speed grade of -2.

The CORDIC Model meets timing closure at 250 MHz for 32 bit inputs, while the Floating Point Matrix multiplication exhibited significant negative slack (about 0.8 ns). Thus, a reasonable upperbound for the frequency of the floating point model is likely about 200 MHz.

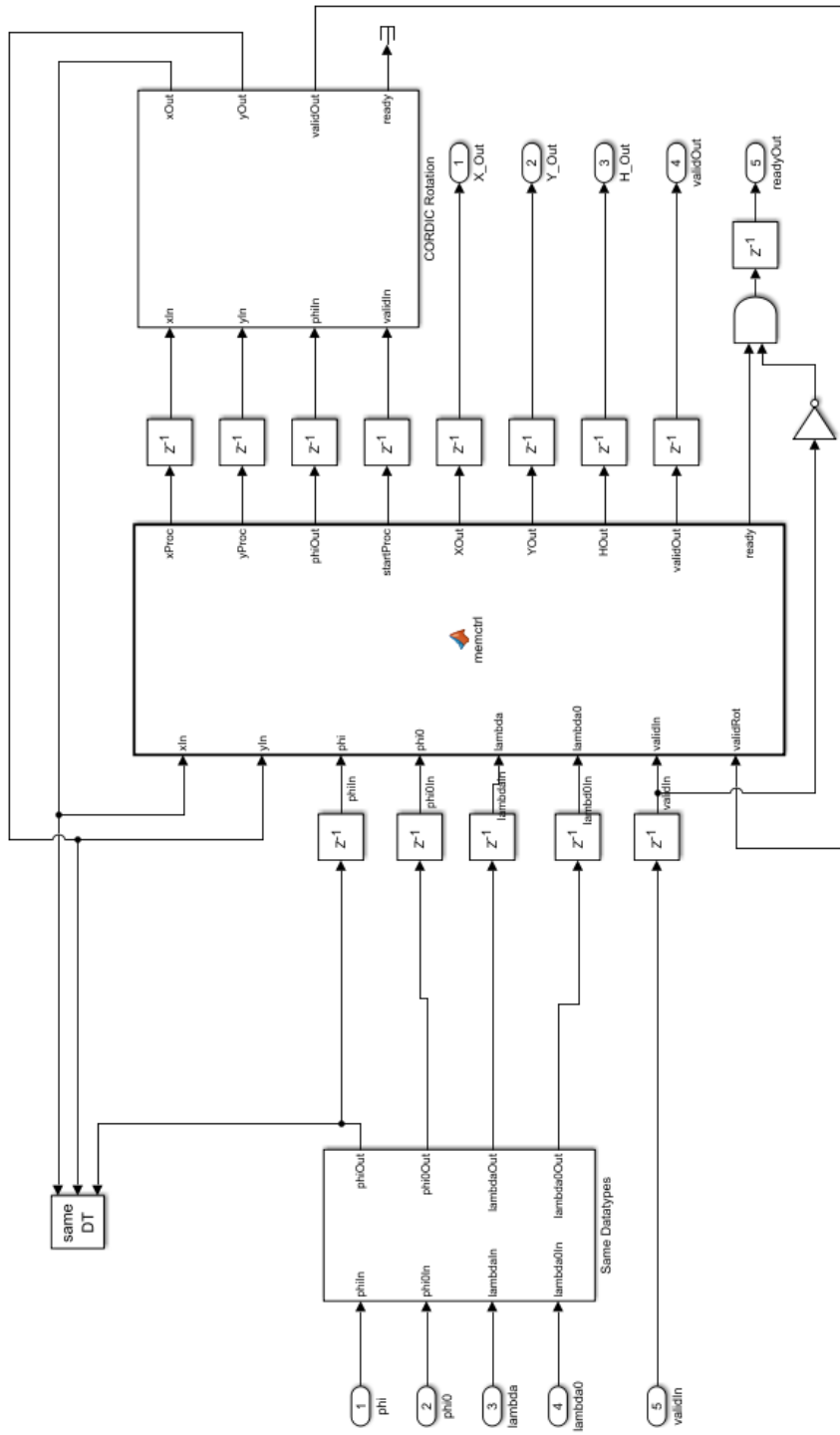


Figure 9.3: ARCHITECTURE OF THE HDL EFFICIENT OBLIQUE ORTHOGONAL TRANSFORMATION

Resource	CORDIC: sfix24En18	CORDIC: sfix32En24	NFP: Matrix Multiplication
LUT	1334	2238	10367
LUTRAM	0	0	799
FF	618	820	13575
DSP	0	0	154

Figure 9.4: RESOURCE ESTIMATION FOR THE TWO METHODS OF COMPUTATION.

Chapter 10

Future Horizons

We began with the fundamental principles of fixed-point arithmetic, moved through the details of digital hardware representation, and eventually applied these concepts to complex real-world systems. My hope is that the hands-on exercises have not only sharpened your technical proficiency but also given you the confidence to architect efficient, high-performance fixed-point systems in MATLAB.

I want to extend my sincere gratitude to you, the reader, for your persistence and dedication. This is no small feat. It requires a blend of mathematical rigor and engineering intuition. Your commitment to working through these examples is what transforms theory into practical expertise, and I am honored to have been part of your learning journey.

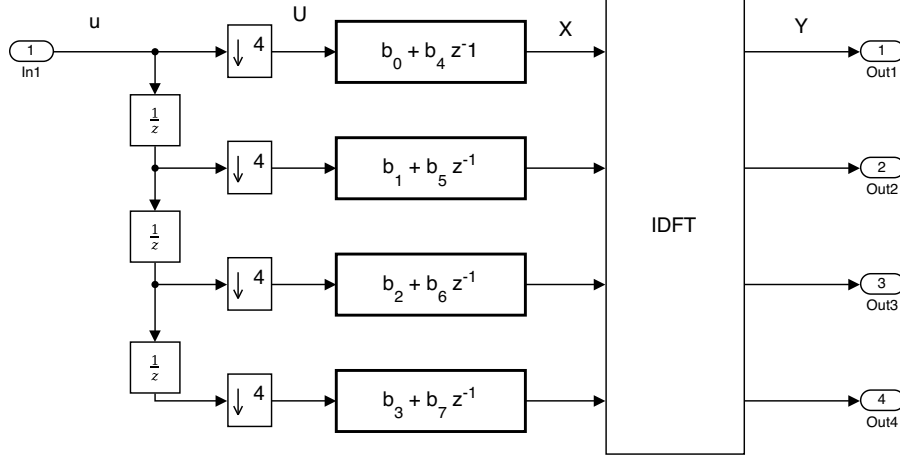
While this volume concludes our initial workshop series, the exploration of fixed-point systems is far from over. I am excited to announce that work is already underway on the next installment of this series. The upcoming volume will adopt a comprehensive *monograph style*. We will delve deeper into advanced topics. Stay tuned for this new chapter in our journey toward hardware-efficient design.

Until then, keep iterating, keep optimizing, and thank you once again for sticking with us until the very end.

Appendix A

Derivation of the DFT Polyphase Filter Bank

The following is a derivation of the DFT Polyphase Filter bank [8]. It shows that the relationship between the input and output is a phase-shifted version of the full-length filter, which turns the low-pass filter into band-pass filters for each successive channel. The advantage of this structure is that the sample-rate is reduced by $1/M$, where M is the number of channels, before the computations begin, and there is no decimation after the computations have been made (i.e. you don't throw away anything you've computed).



$$M = 4, \quad U = \text{flipud}(\text{reshape}(u, M, \text{length}(u)/M)) = \begin{bmatrix} u_4 & u_8 & u_{12} & u_{16} & u_{20} & u_{24} & \dots \\ u_3 & u_7 & u_{11} & u_{15} & u_{19} & u_{23} & \dots \\ u_2 & u_6 & u_{10} & u_{14} & u_{18} & u_{22} & \dots \\ u_1 & u_5 & u_9 & u_{13} & u_{17} & u_{21} & \dots \end{bmatrix}$$

$$N = 2, \quad B = (\text{reshape}(b, M, N)) = \begin{bmatrix} b_0 & b_4 \\ b_1 & b_5 \\ b_2 & b_6 \\ b_3 & b_7 \end{bmatrix}$$

$$X(i, :) = \text{filter}(B(i, :), 1, U(i, :))$$

$$X(:, k/M) = \begin{bmatrix} b_0 u_k & + & b_4 u_{k-4} \\ b_1 u_{k-1} & + & b_5 u_{k-5} \\ b_2 u_{k-2} & + & b_6 u_{k-6} \\ b_3 u_{k-3} & + & b_7 u_{k-7} \end{bmatrix}$$

IFFT

$$W = e^{2\pi i/M}, \quad F = \text{ifft}(\text{eye}(M)) = (1/M) \cdot \begin{bmatrix} W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 \\ W^0 & W^2 & W^4 & W^6 \\ W^0 & W^3 & W^6 & W^9 \end{bmatrix}$$

$$Y(:, k/M) = \text{ifft}(X(:, k/M)) = F \cdot X(:, k/M) = (1/M) \cdot$$

$$\begin{bmatrix} (b_0 u_k + b_4 u_{k-4})W^0 + (b_1 u_{k-1} + b_5 u_{k-5})W^0 + (b_2 u_{k-2} + b_6 u_{k-6})W^0 + (b_3 u_{k-3} + b_7 u_{k-7})W^0 \\ (b_0 u_k + b_4 u_{k-4})W^0 + (b_1 u_{k-1} + b_5 u_{k-5})W^1 + (b_2 u_{k-2} + b_6 u_{k-6})W^2 + (b_3 u_{k-3} + b_7 u_{k-7})W^3 \\ (b_0 u_k + b_4 u_{k-4})W^0 + (b_1 u_{k-1} + b_5 u_{k-5})W^2 + (b_2 u_{k-2} + b_6 u_{k-6})W^4 + (b_3 u_{k-3} + b_7 u_{k-7})W^6 \\ (b_0 u_k + b_4 u_{k-4})W^0 + (b_1 u_{k-1} + b_5 u_{k-5})W^3 + (b_2 u_{k-2} + b_6 u_{k-6})W^6 + (b_3 u_{k-3} + b_7 u_{k-7})W^9 \end{bmatrix}$$

$$\begin{array}{cccccccc} W^0 & = & W^4 & = & W^8 & = & W^{12} & = & W^{16} & = & W^{20} & = & 1 \\ W^1 & = & W^5 & = & W^9 & = & W^{13} & = & W^{17} & = & W^{21} & & \\ W^2 & = & W^6 & = & W^{10} & = & W^{14} & = & W^{18} & = & W^{22} & & \\ W^3 & = & W^7 & = & W^{11} & = & W^{15} & = & W^{19} & = & W^{23} & & \end{array}$$

$$Y(:, k/M) = (1/M) \cdot$$

$$\begin{bmatrix} b_0 u_k W^0 + b_1 u_{k-1} W^0 + b_2 u_{k-2} W^0 + b_3 u_{k-3} W^0 + \dots \\ b_0 u_k W^0 + b_1 u_{k-1} W^1 + b_2 u_{k-2} W^2 + b_3 u_{k-3} W^3 + \dots \\ b_0 u_k W^0 + b_1 u_{k-1} W^2 + b_2 u_{k-2} W^4 + b_3 u_{k-3} W^6 + \dots \\ b_0 u_k W^0 + b_1 u_{k-1} W^3 + b_2 u_{k-2} W^6 + b_3 u_{k-3} W^9 + \dots \\ b_4 u_{k-4} W^0 + b_5 u_{k-5} W^0 + b_6 u_{k-6} W^0 + b_7 u_{k-7} W^0 \\ b_4 u_{k-4} W^4 + b_5 u_{k-5} W^5 + b_6 u_{k-6} W^6 + b_7 u_{k-7} W^7 \\ b_4 u_{k-4} W^8 + b_5 u_{k-5} W^{10} + b_6 u_{k-6} W^{12} + b_7 u_{k-7} W^{14} \\ b_4 u_{k-4} W^{12} + b_5 u_{k-5} W^{15} + b_6 u_{k-6} W^{18} + b_7 u_{k-7} W^{21} \end{bmatrix}$$

If the impulse response $h(n)$ is multiplied by a factor $e^{in\Omega_0}$, then the frequency response $H(e^{i\Omega})$ of the prototype is shifted to the right by Ω_0 :

$$h(n) \cdot e^{in\Omega_0} \Leftrightarrow H(e^{i(\Omega-\Omega_0)}).$$

N.J. Fliege, *Multirate Digital Signal Processing*, Wiley, 1994, ISBN 0 471 93976 5, pp. 229, 227–237.

```

M = 4; % Number of channels
N = 16; % Number of coefficients in each channel
Wn = 1/M; % Normalized frequency
b = fir1(M*N-2,Wn); % Filter coefficients
b = [b,zeros(1,M*N-length(b))];
B = reshape(b,M,N);

% Input: Complex linear chirp
n_out = 1024;
n = M*n_out;
fs = n; % Sampling frequency
f1 = fs; % Target frequency
f0 = 0; % Initial frequency
phi0 = 0; % Initial phase offset
t = (0:n-1)/fs;
phi = pi*f1*t.^2 + 2*pi*f0*t + phi0;
u = exp(1i*phi);

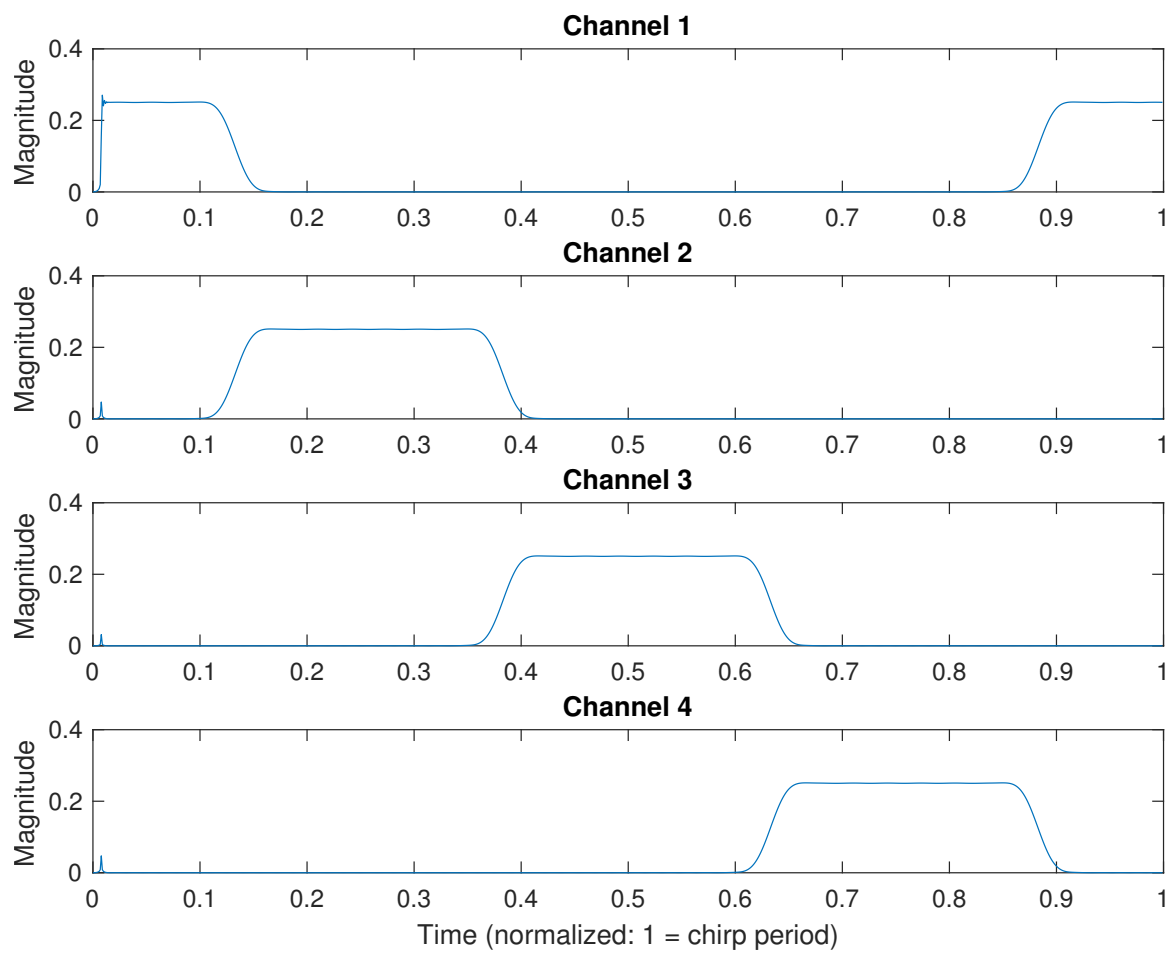
% "Parallelize and Downsample"
U = flipud(reshape(u,M,length(u)/M));
U = [zeros(M,1), U(:,1:end-1)];

% FIR Filter Bank
X = zeros(size(U));
for i = 1:M
    X(i,:) = filter(B(i,:),1,U(i,:));
end

% IFFT
Y = ifft(X);

% Slower clock on output
t2 = (0:size(Y,2)-1)/(fs/M);
clf
for i = 1:M
    subplot(M,1,i)
    plot(t2,abs(Y(i,:)))
    ylabel('Magnitude')
    title(['Channel ',int2str(i)])
end
xlabel('Time (normalized: 1 = chirp period)')

```



Appendix B

Equivalent Toolbox Functions and Blocks

The following MathWorks products are replacements for the functions used in this workshop, and generally provide more features and better code generation. In general, functions provide an easy MATLAB interface, and System objects and blocks provide more options for efficient code generation.

MATLAB Coder or Embedded Coder are required for C code generation.

HDL Coder is required for HDL code generation.

- Functions work in MATLAB, and MATLAB Function blocks in Simulink.
- System objects work in MATLAB, MATLAB System Block in Simulink, and MATLAB Function Block in Simulink.
- Blocks work in Simulink.

FIR Filter

- **filter** function. Supports fixed-point inputs for FIR filter only (denominator coefficients == 1). MATLAB, Fixed-Point Designer.
- **dsp.FIRFilter** System object. Supports fixed-point inputs and C and HDL code generation. DSP System Toolbox.
- **Discrete FIR Filter** block. Supports fixed-point inputs and C and HDL code generation. DSP System Toolbox.

IFFT

- `ifft` function. Does not support fixed-point inputs. MATLAB.
- `dsp.IFFT` System object. Supports fixed-point inputs and C code generation. DSP System Toolbox.
- `dsp.HDLIFFT` System object. Supports fixed-point inputs. Optimized for HDL Code generation. DSP System Toolbox.
- `IFFT HDL Optimized` block. Supports fixed-point inputs. Optimized for HDL Code generation. DSP System Toolbox.

CORDIC magnitude and phase

- `cordiccart2pol` function. Supports fixed-point inputs. Fixed-Point Designer.
- `dsp.HDLComplexToMagnitudeAngle` System object. Supports fixed-point inputs and C and HDL code generation. DSP System Toolbox.
- `Complex to Magnitude-Angle HDL Optimized` block. Supports fixed-point inputs and C and HDL code generation. DSP System Toolbox.

Channelizer HDL Optimized

- `Channelizer HDL Optimized` block. Supports fixed-point inputs and HDL code generation. DSP System Toolbox. An example is in `solutions/06dftpolyphasefilterbank/dft_polyphase_8_channel_new_hdl_using_buffer_block.slx`

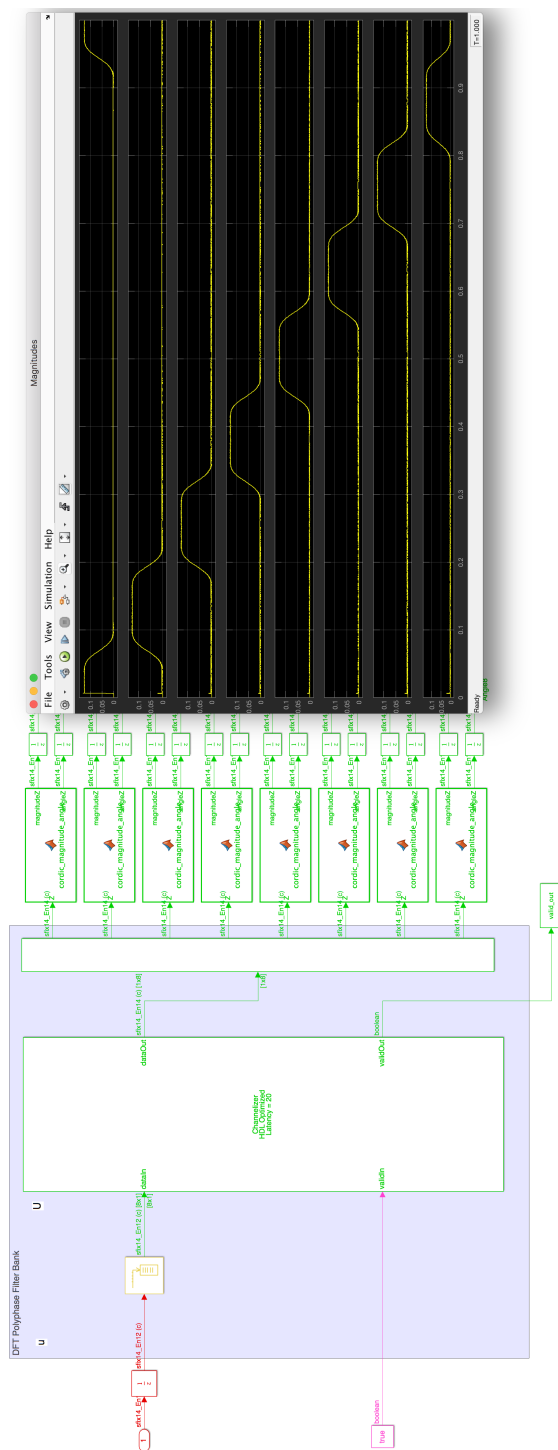


Figure B.1: CHANNELIZER HDL OPTIMIZED
The full DFT Polyphase Filter is implemented as a single block in DSP System Toolbox

Appendix C

HEX Read and Write

The `fi` object has methods to get and set hex values. You can use this feature to create files of test vectors in hex.

For example, the following files included in this workshop write and read hex data to and from text files.

```
solutions/read_write_hex/read_hex_from_file.m  
solutions/read_write_hex/write_hex_to_file
```

Function `write_hex_to_file` adds the prefix `0x` to the hex word, and saves them in columns in a text file.

```
function write_hex_to_file(a,filename)  
    % write_hex_to_file Write hex values from fi object to file  
    %  
    % write_hex_to_file(a,filename) writes hex values from fi  
    % object a to file named filename.  
    assert(isfi(a),'Input must be a fi object');  
    % Create hex column vector  
    h = hex(a(:));  
    fid = fopen(filename,'w');  
    % Write hex to the file one row at a time  
    for i = 1:numel(a)  
        fprintf(fid, '0x%s\n', h(i,:));  
    end  
    fclose(fid);  
end
```

If you want a different prefix, then change `0x` in the following line to your preferred prefix.

```
fprintf(fid, '0x%s\n', h(i,:));
```

Function `read_hex_from_file` reads hex data from the file generated by `write_hex_to_file` and returns a `fi` object with values corresponding to that hex data. Since hex data doesn't have information about signedness, wordlength, or fractionlength, you need to give it a prototype `fi` object to fill.

```
function b = read_hex_from_file(prototype, filename)
    % read_hex_from_file Reads hex values from file
    %
    % read_hex_from_file(prototype, filename) reads hex values
    % from file filename and casts them to a fi object with
    % the properties of prototype.
    assert(isfi(prototype), 'Input must be a fi object');

    fid = fopen(filename, 'r');
    h = textscan(fid, '0x%s');
    fclose(fid);
    b = zeros(size(h{1}), 'like', prototype);
    b.hex = char(h{1});
end
```

The following example `test_read_write_hex.m` creates a hex file and reads it back.

```
%% Create a fi object and write hex to file
a = fi(-16:16, 1, 8, 0);
write_hex_to_file(a, 'hex_file.txt');

%% Read fi object from hex file
prototype = cast([], 'like', a);
b = read_hex_from_file(prototype, 'hex_file.txt');

%% Verify that the write-read round trip worked
is_equal_write_read_round_trip = isequal(a(:), b(:))
```

The generated file `hex_file.txt` contains the following, corresponding to the integers -16:16 in signed 8-bit fixed-point with 0 fraction length.

```
0xf0
0xf1
0xf2
0xf3
0xf4
0xf5
0xf6
0xf7
0xf8
0xf9
0xfa
0xfb
0xfc
0xfd
0xfe
0xff
0x00
0x01
0x02
0x03
0x04
0x05
0x06
0x07
0x08
0x09
0x0a
0x0b
0x0c
0x0d
0x0e
0x0f
0x10
```


Appendix D

ac_fixed, sc_fixed, and fi

The syntax for constructing a MATLAB fixed-point (`fi`) object, is

```
a = fi(x, s, w, f, 'RoundingMethod', q, 'OverflowAction', o);
```

The syntax for constructing an HLSLibs algorithmic C fixed-point (`ac_fixed`) object [7] from <https://hlslibs.org> is

```
ac_fixed<W, I, S, Q, 0> A = X;
```

The syntax for constructing a signed fixed-point object in SystemC from <https://systemc.org> is

```
sc_fixed<W, I, Q, 0> A = X;
```

and for an unsigned fixed-point object is

```
sc_ufixed<W, I, Q, 0> A = X;
```

The equivalence between the properties is described in the tables below.

Property	fi	ac_fixed	sc_fixed, sc_ufixed
Variable name	a	A	A
Value	x	X	X
Signedness (true = signed, false = unsigned)	s	S	
Word length in bits	w	W	W
Fraction length in bits	f	W - I	W - I
Integer length in bits	w - f	I	I
Rounding method	q	Q	Q
Overflow action	o	O	O

Rounding method	fi RoundingMethod	ac_fixed ac_q_mode	sc_fixed, sc_ufixed q_mode
Round toward positive infinity. The opposite of Floor.	Ceiling		
Round toward nearest. Ties round to the nearest even stored integer (least biased).	Convergent	AC_RND_CONV	SC_RND_CONV
Round toward negative infinity.	Floor	AC_TRN (default)	SC_TRN
Round toward nearest. Ties round toward positive infinity. This is a two's-complement round-to-nearest.	Nearest (default)	AC_RND	SC_RND
Round toward nearest. Ties round toward negative infinity for negative numbers, and toward positive infinity for positive numbers. This is a sign/magnitude round-to-nearest and is the behavior of MATLAB's <code>round</code> function on double-precision floating-point numbers.	Round	AC_RND_INF	SC_RND_INF
Round toward zero.	Zero	AC_TRN_ZERO	SC_TRN_ZERO
Round toward nearest. Ties round toward zero.		AC_RND_ZERO	SC_RND_ZERO
Round toward nearest. Ties round to the nearest odd stored integer.		AC_RND_CONV_ODD	

Overflow action	fi OverflowAction	ac_fixed ac_o_mode	sc_fixed, sc_ufixed o_mode
Saturate to maximum or minimum value of the fixed-point range on overflow. This is two's complement saturation overflow.	Saturate (default)	AC_SAT	SC_SAT
Wrap on overflow. This mode is also known as two's complement overflow.	Wrap	AC_WRAP (default)	SC_WRAP
Saturate to zero on overflow.		AC_SAT_ZERO	SC_SAT_ZERO
Symmetric saturation on overflow. The most negative value of a signed type saturates to the value one least-significant bit toward zero so that the most positive and most negative value have the same magnitude. This is sign/magnitude saturation overflow.		AC_SAT_SYM	SC_SAT_SYM

Tom Bryan chose fraction length as a property of the `fi` object instead of integer length because he was influenced by the “Q15” format from Texas Instruments where “15” is the number of fraction bits ([https://en.wikipedia.org/wiki/Q_\(number_format\)](https://en.wikipedia.org/wiki/Q_(number_format))), and because industry notation is not consistent with respect to whether the sign bit is included in the integer length (the sign bit is counted as an integer bit in `ac_fixed`, but not in Q number format).

Floor rounding and Wrap overflow usually produce more efficient code in C and HDL, but Tom

chose the defaults of **Nearest** rounding and **Saturate** overflow for `fi` because they would most likely produce correct (or least-surprising) answers, and it is consistent with the behavior of MATLAB's built-in integers (`int8`, `int16`, ..., `int64`, `uint8`, `uint16`, ..., `uint64`). For example, the accumulated errors using **Floor** rounding in the recursive computation of the index of the Vancouver Stock Exchange caused the index to appear to lose about half its value over the course of two years (Section 1.4, page 26, [17]).

Example

The following are equivalent.

Create a MATLAB fixed-point (`fi`) object using default signed, 16-bit word length, best-precision fraction length for the value, **Nearest** rounding, and **Saturate** overflow.

```
>> format long
>> a = fi(pi)

a =

    3.141601562500000

        DataTypeMode: Fixed-point: binary point scaling
           Signedness: Signed
          WordLength: 16
       FractionLength: 13
```

Create an an HLSLibs fixed-point (`ac_fixed`) object with the same properties as the above `fi` object.

```
#include "ac_fixed.h"
#include <iostream>
int main(){
    // Equivalent to a = fi(pi)
    ac_fixed<16,3,true,AC_RND,AC_SAT> a = 3.141592653589793;
    std::cout << "a = " << a << std::endl;
}
```

Output:

```
a = 3.1416015625
```

Bibliography

- [1] Harshita Bhurat, Tom Bryan, and Jared Schuler. *Best Practices for Converting MATLAB Code to Fixed Point Webinar*. The MathWorks, Inc. 2013. URL: <http://www.mathworks.com/videos/best-practices-for-converting-matlab-code-to-fixed-point-using-fixed-point-designer-86835.html>.
- [2] Harshita Bhurat, Tom Bryan, and Julia Wall. *Best Practices for Converting MATLAB Code to Fixed Point Technical Article*. The MathWorks, Inc. 2013. URL: <https://www.mathworks.com/company/newsletters/articles/best-practices-for-converting-matlab-code-to-fixed-point.html>.
- [3] Thomas A. Bryan. *Perform QR Factorization Using CORDIC*. Derivation of the bound on growth when computing QR. MathWorks. 2010. URL: <https://www.mathworks.com/help/fixedpoint/examples/perform-qr-factorization-using-cordic.html>.
- [4] Thomas A. Bryan and Jenna L. Warren. “Systems and Methods for Design Parameter Selection”. Patent pending. U.S. Patent Application No. 16/947,130. 2020.
- [5] Zizhong Chen and Jack J. Dongarra. “Condition Numbers of Gaussian Random Matrices”. In: *SIAM J. Matrix Anal. Appl.* 27.3 (July 2005), pp. 603–620. ISSN: 0895-4798. DOI: [10.1137/040616413](https://doi.org/10.1137/040616413). URL: <http://dx.doi.org/10.1137/040616413>.
- [6] *convergent function reference page*. The MathWorks, Inc. 2006. URL: <https://www.mathworks.com/help/fixedpoint/ref/convergent.html>.
- [7] Michael Fingeroff. *High-Level Synthesis Blue Book*. Mentor Graphics Corporation, 2019. ISBN: 978-1-4500-9724-6.
- [8] N.J. Fliege. *Multirate Digital Signal Processing: multirate systems, filter banks, wavelets*. Chichester: John Wiley & Sons, 1994. ISBN: 0-471-93976-5.
- [9] *Fuzzy Logic Toolbox: Foundations of Fuzzy Logic*. The MathWorks, Inc. URL: <https://www.mathworks.com/help/fuzzy/foundations-of-fuzzy-logic.html>.
- [10] Don Koks. *Using Rotations to Build Aerospace Coordinate Systems*. Technical note ADA484864. Retrieved 23 November 2022. Salisbury, Australia: Defence Science and Technology Organisation Salisbury (Australia) Systems Sciences LAB, Aug. 2008. URL: <https://apps.dtic.mil/sti/citations/ADA484864>.
- [11] James C. Kolanek and Thomas A. Bryan. “Uniform Discrete Fourier Transform Filter Parameter Encoder”. Patent Number: [US 5,870,436](https://patents.google.com/patent/US5870436). Feb. 2020.

- [12] Claire Masterson. *Massive MIMO and Beamforming: The Signal Processing Behind the 5G Buzzwords*. Analog Devices. 2017. URL: <https://www.analog.com/media/en/analog-dialogue/volume-51/number-3/articles/massive-mimo-and-beamforming-the-signal-processing-behind-the-5g-buzzwords.pdf>.
- [13] Cleve Moler. *Floating Point Numbers*. The MathWorks, Inc. 2014. URL: <https://blogs.mathworks.com/cleve/2014/07/07/floating-point-numbers/>.
- [14] Cleve Moler. *Floating Points: IEEE Standard Unifies Arithmetic Model*. The MathWorks, Inc. 1996. URL: <https://www.mathworks.com/company/newsletters/articles/floating-points-ieee-standard-unifies-arithmetic-model.html>.
- [15] Cleve Moler. *Jim Wilkinson*. The MathWorks, Inc. 2013. URL: <https://blogs.mathworks.com/cleve/2013/02/18/jim-wilkinson/>.
- [16] *nearest function reference page*. The MathWorks, Inc. 2006. URL: <https://www.mathworks.com/help/fixedpoint/ref/nearest.html>.
- [17] Kevin Quinn. “Ever Had Problems Rounding Off Figures? This Stock Exchange Has”. In: *The Wall Street Journal* (Nov. 1983), p. 37.
- [18] C.M. Rader. “VLSI Systolic Arrays for Adaptive Nulling”. In: *IEEE Signal Processing Magazine* (July 1996), pp. 29–49.
- [19] J. R. Ragazzini and L. A. Zadeh. “The analysis of sampled-data systems”. In: *Electrical Engineering* 71.12 (1952), pp. 1102–1102. DOI: [10.1109/EE.1952.6437836](https://doi.org/10.1109/EE.1952.6437836).
- [20] J. R. Ragazzini and L. A. Zadeh. “The analysis of sampled-data systems”. In: *Transactions of the American Institute of Electrical Engineers* 71 (II) (1952), pp. 225–234.
- [21] *round function reference page*. The MathWorks, Inc. 2006. URL: <https://www.mathworks.com/help/fixedpoint/ref/round.html>.
- [22] John P. Snyder. *Map Projections - A Working Manual*. US Geological Survey. 1987.
- [23] Jack E. Volder. “The Birth of CORDIC”. In: *Journal of VLSI Signal Processing* 25 (2 2000), pp. 101–105.
- [24] Jack E. Volder. “The CORDIC Trigonometric Computing Technique”. In: *IRE Trans. Electron. Comput. EC* 8 (1959), pp. 330–334.
- [25] Jack E. Volder. “The CORDIC trigonometric computing technique”. In: *IRE Transactions on Electronic Computers* (now called *IEEE Transactions on Computers*) 8 (Sept. 1959), pp. 330–334.
- [26] J.S. Walther. “A Unified Algorithm for Elementary Functions”. In: *Proceedings of the May 18-20, 1971, Spring Joint Computer Conference* (1971), pp. 379–385.
- [27] B. Widrow. “A Study of Rough Amplitude Quantization by Means of Nyquist Sampling Theory”. In: *IRE Transactions on Circuit Theory* 3.4 (Dec. 1956), pp. 266–276.
- [28] B. Widrow and I. Kollár. *Quantization Noise – Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge, UK: Cambridge University Press, 2008.
- [29] J.H. Wilkinson. *Rounding Errors in Algebraic Processes*. Englewood Cliffs, New Jersey: Prentice-Hall, 1963.

- [30] Kyle York. *Multiplication by a Fixed Point Constant*. 2009. URL: <https://www.embedded.com/design/mcus-processors-and-socs/4008308/Multiplication-by-a-Fixed-Point-Constant> (visited on 11/21/2018).
- [31] Lotfi A. Zadeh. “Fuzzy sets”. In: *Information and Control* 8.3 (1965), pp. 338–353. ISSN: 0019–9958. DOI: [10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X). URL: <https://www.sciencedirect.com/science/article/pii/S001999586590241X>.
- [32] Daniel R. Zahirniak, David L. Sharpin, and Timothy W. Fields. “A Hardware-Efficient, Multirate, Digital Channelized Receiver Architecture”. In: *IEEE Transactions on Aerospace and Electronic Systems* 34.1 (Jan. 1998), pp. 137–152.

Author Biographies



Tom Bryan has been a developer at MathWorks since 1997 where he created Fixed-Point Toolbox. He worked at Raytheon Electromagnetics Systems Division in Goleta, California from 1984 to 1996 where he worked on radar applications, and received a PhD in Electrical and Computer Engineering at University of California, Santa Barbara on a Raytheon Miccioli Scholarship. Tom has been awarded 10 patents and is the inventor of the `fi` object, `cast(a, 'like', b)`, and datatype visualization features that are used in this workshop.



Brenda Zhuang is a software engineering manager and leads a team that develops software tools for automatic deployment of embedded applications in microprocessors and FPGAs. Brenda has contributed to the development and evolution of many new features in the MATLAB and Simulink product families. She received her Ph.D. in Systems Engineering from Boston University and M.S. in Electrical and Electronics Engineering from Hong Kong University of Science and Technology. She served on technical program committees for the international conferences in modeling and simulation.



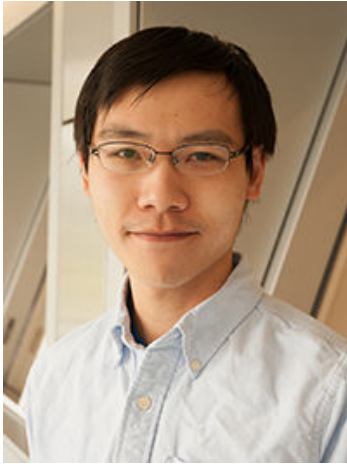
Jenna Warren has been with The MathWorks since 2013, where she works as a team lead of the code generation documentation team. She has a B.S. in Math and a Masters in Electrical Engineering with a focus on biomedical signal processing applications. In her spare time she enjoys rowing with the MathWorks rowing team and running Ragnar relay races with coworkers.



Jim Joy has been with the MathWorks since 2017. Since then, he has worked on fixed-point math, with a focus on embedded applications utilizing CORDIC algorithms. Prior to working at the MathWorks, he finished a Ph.D. in Physics from Brown University, where he studied the properties of Quantum phase transitions in superconducting systems.



Bharath Venkataraman is the Senior Manager of the Image & HDL IP group at MathWorks. Bharath's teams have primary focus on image processing, computer vision, automated driving, and HDL code generation for signal processing, communications, and video applications. Prior to joining MathWorks in 2003, Bharath worked on several ASIC and FPGA designs, primarily related to coding algorithms over satellite channels. Bharath has an M.S. in Electrical Engineering from the University of Virginia.



Rui Zhang has been with MathWorks since 2021. Since then, he has worked on fixed-point algorithms and implementations that focus on HDL optimization. Prior to working at MathWorks, he worked at Analog Devices Inc, where he worked on embedded systems and digital design.